

LÃ THỊ LINH

KHOA HỌC MÁY TÍNH

BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA



PHENIKAA
UNIVERSITY

ĐỒ ÁN TỐT NGHIỆP

PHÁT HIỆN HÀNH VI BẤT THƯỜNG TRONG THI CỬ

Sinh viên: Lã Thị Linh

Mã số sinh viên: 22014063

Khóa: 16

Ngành: Khoa học máy tính

Hệ: Đại học chính quy

Giảng viên hướng dẫn: TS. Bùi Huy Toàn

Hà Nội - Năm 2026

LÃ THỊ LINH

KHOA HỌC MÁY TÍNH

BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA



PHENIKAA
UNIVERSITY

ĐỒ ÁN TỐT NGHIỆP

PHÁT HIỆN HÀNH VI BẤT THƯỜNG TRONG THI CỬ

Sinh viên: Lã Thị Linh

Mã số sinh viên: 22014063

Khóa: 16

Ngành: Khoa học máy tính

Hệ: Đại học chính quy

Giảng viên hướng dẫn: TS. Bùi Huy Toàn

Hà Nội - Năm 2026

LỜI CAM ĐOAN

Họ và tên sinh viên: Lã Thị Linh

Mã sinh viên: 22014063

Lớp : K16-KHMT (TN)

Tôi đã thực hiện đồ án/khóa luận tốt nghiệp với đề tài: Phát hiện hành vi bất thường trong thi cử

Tôi xin cam đoan đây là đề tài nghiên cứu của riêng tôi và được sự hướng dẫn của: TS. Bùi Huy Toàn.

Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa được các tác giả khác công bố dưới bất kỳ hình thức nào. Nếu phát hiện có bất kỳ hình thức gian lận nào tôi xin hoàn toàn chịu trách nhiệm trước pháp luật.

Hà nội, ngày 10 tháng 08 năm 2026

GIẢNG VIÊN HƯỚNG DẪN

(Ký, ghi rõ họ tên)

SINH VIÊN

(Ký, ghi rõ họ tên)

Bùi Huy Toàn

Lã Thị Linh

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc tới TS. Bùi Huy Toàn – giảng viên đã trực tiếp hướng dẫn, tận tình chỉ bảo và định hướng cho em trong suốt quá trình thực hiện đồ án tốt nghiệp. Thầy đã dành nhiều thời gian và tâm huyết để hướng dẫn em từ việc xác định đề tài, xây dựng phương pháp thực hiện, triển khai các nội dung nghiên cứu cho đến quá trình hoàn thiện báo cáo. Những kiến thức, kinh nghiệm chuyên môn cùng những góp ý quý báu của thầy là nền tảng quan trọng giúp em hoàn thành đồ án và tích lũy thêm nhiều kinh nghiệm hữu ích cho quá trình học tập, nghiên cứu và làm việc sau này.

Em xin chân thành cảm ơn Ban Giám hiệu cùng toàn thể quý thầy cô ngành Khoa học máy tính, Trường Đại học Phenikaa đã tận tình giảng dạy, truyền đạt cho em những kiến thức nền tảng và chuyên môn trong suốt quá trình học tập tại trường. Những kiến thức và kỹ năng được trang bị trong quá trình học tập không chỉ giúp em có cơ sở để thực hiện đồ án tốt nghiệp mà còn là hành trang quý báu cho công việc và định hướng phát triển của em trong tương lai.

Em cũng xin gửi lời cảm ơn tới các thầy cô, bạn bè và những người đã luôn sẵn sàng trao đổi, chia sẻ kinh nghiệm, góp ý và hỗ trợ em trong quá trình học tập cũng như thực hiện đồ án. Những ý kiến đóng góp và sự động viên đó đã giúp em có thêm kiến thức, kinh nghiệm và động lực để hoàn thiện nội dung đồ án một cách tốt nhất.

Mặc dù đã dành nhiều thời gian và nỗ lực để thực hiện đồ án, song do kiến thức và kinh nghiệm của bản thân còn hạn chế nên báo cáo chắc chắn không thể tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến nhận xét và đóng góp quý báu từ quý thầy cô để đồ án được hoàn thiện hơn, đồng thời giúp em có thêm kinh nghiệm cho quá trình học tập và nghiên cứu sau này.

Em xin chân thành cảm ơn!

TÓM TẮT

Giám sát thi trực tuyến bằng thị giác máy tính là một nhu cầu ngày càng tăng nhằm đảm bảo tính liêm chính trong các kỳ thi từ xa. Các hệ thống hiện tại như Proctorio hay Honorlock tồn tại nhưng thường là các sản phẩm thương mại đóng kín. Các dự án tham khảo mở như exam-cheating-detection chỉ sử dụng những thuật toán phát hiện đơn giản dựa trên if-elif threshold-based, thiếu các kỹ thuật tiên tiến như ước lượng góc đầu chính xác, xác thực danh tính xuyên suốt kỳ thi, và khả năng tổng hợp đa tín hiệu thông minh. Các hạn chế này dẫn đến tỷ lệ báo động giả cao và khó phát hiện các hành vi gian lận phức tạp như đổi người thi hộ giữa chừng.

Đề án này chọn hướng tiếp cận xây dựng một ứng dụng giám sát CV độc lập, chạy trên một máy tính cục bộ, với toàn bộ pipeline được tự thiết kế từ đầu. Hướng này được lựa chọn vì tập trung vào chất lượng cao của sản phẩm CV thay vì mở rộng tính năng quá nhanh, cho phép chứng minh cải tiến bằng số liệu định lượng, và phù hợp với khung thời gian đề án.

Giải pháp gồm ba tầng chính: tầng Perception trích xuất đặc trưng dùng MediaPipe, MTCNN và YOLOv8, tầng Signal Extractors tính toán bảy tín hiệu gồm có ước lượng góc đầu bằng solvePnP, xác thực danh tính bằng FaceNet embedding, và tầng Risk Fusion Engine tự thiết kế dùng state machine theo cửa sổ thời gian trượt kết hợp với weighted scoring và hysteresis để giảm báo động giả.

Đóng góp chính của đề án bao gồm: ước lượng góc đầu chính xác sử dụng solvePnP thay cho cách pixel-based đơn giản, khả năng phát hiện đổi người thông qua xác thực danh tính xuyên suốt, Risk Fusion Engine tự thiết kế thay thế if-elif, và đánh giá định lượng trên bộ test 26 clip (gần 2 tiếng) tự quay và thu thập với ground truth được gán nhãn, so sánh Precision, Recall, F1 với baseline đơn giản. Kết quả đạt được cho thấy hệ thống cải thiện đáng kể so với baseline, đồng thời tạo nền tảng cho mở rộng sang nền tảng đa người dùng trong tương lai.

MỤC LỤC

LỜI CAM DOAN	iii
LỜI CẢM ƠN	iv
TÓM TẮT	v
Danh mục hình vẽ.....	xii
Danh mục bảng.....	xiv
Danh mục các từ viết tắt	xv
Danh mục thuật ngữ	xvi
Mở đầu	1
Chương 1 Giới thiệu đề tài	3
1.1 Mục tiêu nghiên cứu	3
1.1.1 Mục tiêu tổng quát.....	3
1.1.2 Mục tiêu cụ thể	4
1.2 Đối tượng và phạm vi nghiên cứu	4
1.2.1 Đối tượng nghiên cứu.....	4
1.2.2 Phạm vi thực hiện.....	5
1.2.3 Nội dung ngoài phạm vi	5
1.3 Định hướng giải pháp	6
1.4 Các đóng góp chính	6
1.5 Bố cục báo cáo	7
Chương 2 Cơ sở lý thuyết.....	9
2.1 Thị giác máy tính	9
2.1.1 Định nghĩa và mục tiêu	9
2.1.2 Biểu diễn dữ liệu hình ảnh.....	10
2.1.3 Các phương pháp trong thị giác máy tính	11

2.2 Phát hiện khuôn mặt	12
2.2.1 Tầm quan trọng và định nghĩa.....	12
2.2.2 Lịch sử phát triển.....	13
2.2.3 Multi-task Cascaded Convolutional Networks.....	14
2.2.4 So sánh các phương pháp phát hiện khuôn mặt	17
2.2.5 Lựa chọn phương pháp cho đồ án	17
2.3 Nhận diện điểm mốc khuôn mặt với MediaPipe	18
2.4 Ước lượng góc quay đầu bằng Perspective-n-Point	18
2.5 Xác thực danh tính bằng face embedding.....	19
2.5.1 Biểu diễn véc-tơ đặc trưng	19
2.5.2 Cosine similarity và quyết định có biên	19
2.5.3 Liveness cơ bản	20
2.6 Phát hiện trạng thái mắt bằng EAR	20
2.7 Phát hiện trạng thái miệng	20
2.8 Phát hiện vật thể.....	21
2.9 Xử lý tín hiệu theo thời gian	21
2.9.1 Debounce dựa trên thời lượng.....	21
2.9.2 Cửa sổ thời gian trượt.....	21
2.9.3 Máy trạng thái hữu hạn.....	22
2.10 Kết hợp đa tín hiệu và vùng trề.....	22
2.11 Giao tiếp thời gian thực và đồng bộ dữ liệu	23
2.11.1 Phân quyền người dùng.....	23
2.11.2 Lưu trữ sự kiện và khả năng truy vết.....	24
2.12 Các chỉ số đánh giá	24
Chương 3 Khảo sát và phân tích yêu cầu	26
3.1 Khảo sát các giải pháp thương mại.....	26
3.2 Khảo sát dự án mã nguồn mở	27

3.3 So sánh và phân tích hạn chế	28
3.4 Xác định tác nhân của hệ thống	28
3.5 Yêu cầu chức năng.....	29
3.6 Yêu cầu phi chức năng.....	31
3.7 Ràng buộc và giả định	32
Chương 4 Thiết kế hệ thống	33
4.1 Mô hình nhận diện với kiến trúc đa tầng	33
4.2 Chi tiết các tín hiệu nhận diện	34
4.3 Tổng hợp và đánh giá rủi ro.....	35
4.4 Giải pháp kết hợp đa tín hiệu theo thời gian.....	36
4.4.1 Giải pháp	36
4.4.2 Kết quả đạt được.....	38
4.5 Giải pháp giám sát thời gian thực và kiểm chứng	38
4.5.1 Giải pháp	39
4.5.2 Kết quả đạt được.....	40
4.6 Giải pháp mô hình dữ liệu kết hợp và luồng bằng chứng.....	40
4.6.1 Giải pháp	41
4.6.2 Kết quả đạt được.....	44
4.7 Giải pháp bảo vệ phiên, chính sách và dữ liệu nhạy cảm	44
4.7.1 Giải pháp	45
4.7.2 Kết quả đạt được.....	45
4.8 Đánh giá tổng hợp các đóng góp	46
Chương 5 Cài đặt hệ thống	47
5.1 Công nghệ sử dụng	47
5.2 Cấu trúc kho mã nguồn.....	47
5.3 Cấu hình tập trung.....	50

5.4 Cài đặt ứng dụng thị giác máy tính trên máy tính để bàn	51
5.4.1 Vòng đời ứng dụng	51
5.4.2 Tầng nhận thức	51
5.4.3 Đồng bộ trích xuất tín hiệu	51
5.4.4 Đăng ký khuôn mặt và kiểm tra	52
5.4.5 Bộ điều phối và tổng hợp rủi ro.....	52
5.4.6 Gửi dữ liệu đến dịch vụ phía máy chủ	53
5.5 Cài đặt tiện ích mở rộng trình duyệt	53
5.5.1 Tập khai báo và quyền.....	53
5.5.2 Luồng thiết lập.....	53
5.5.3 Tiến trình nền	54
5.5.4 Thành phần giám sát nội dung	54
5.6 Cài đặt dịch vụ phía máy chủ và bảng điều khiển	54
5.6.1 Khởi tạo FastAPI.....	54
5.6.2 Mô hình dữ liệu	55
5.6.3 Xác thực và phân quyền	55
5.6.4 Xác định chính sách hiệu lực	56
5.6.5 WebSocket và kiểm tra dữ liệu	56
5.6.6 Giao diện web.....	56
5.7 Giao diện và luồng vận hành của hệ thống đã triển khai.....	57
5.7.1 Nguyên tắc tổ chức giao diện	57
5.7.2 Giao diện web dành cho quản trị và giám thị.....	59
5.7.3 Giao diện tiện ích mở rộng dành cho thí sinh	65
5.7.4 Luồng vận hành đầu-cuối	66
5.7.5 Luồng lỗi và chế độ suy giảm	66
5.8 Lưu trữ bằng chứng và sinh báo cáo.....	67
5.8.1 Tạo thư mục dữ liệu phiên.....	67
5.8.2 Chuỗi xử lý báo cáo.....	67

5.8.3 Tác vụ báo cáo và chính sách lưu trữ	67
5.9 Triển khai	68
5.10 Các quyết định cài đặt đáng chú ý	70
Chương 6 Kiểm thử và đánh giá.....	71
6.1 Mục tiêu và phạm vi đánh giá.....	71
6.2 Kế hoạch thực nghiệm mô hình phát hiện hành vi bất thường	71
6.2.1 Bộ dữ liệu đánh giá.....	71
6.2.2 Phương pháp gán nhãn Ground Truth	72
6.2.3 Kết quả thực nghiệm	72
6.2.4 Cải tiến đạt được	74
6.2.5 So sánh với các phương pháp cơ sở	75
6.2.6 Độ trễ phát hiện	76
6.2.7 Các Trường hợp bỏ sót.....	76
6.2.8 Các trường hợp báo động giả	77
6.2.9 Cải tiến đã áp dụng và kết quả	77
6.3 Kiểm thử phần mềm	78
6.3.1 Phương pháp.....	78
6.3.2 Kiểm thử chuỗi xử lý thị giác máy tính và thuật toán	78
6.3.3 Kiểm thử dịch vụ phía máy chủ và bảo mật.....	79
6.3.4 Kiểm thử tiện ích mở rộng	79
6.4 Đánh giá hoạt động của hệ thống.....	80
6.4.1 Phương pháp và tiêu chí đánh giá	80
6.4.2 Đánh giá các kịch bản đầu-cuối	80
6.4.3 Đánh giá luồng dữ liệu và tính nhất quán	81
6.4.4 Đánh giá giao diện và khả năng vận hành.....	82
6.4.5 Tổng hợp mức độ hoạt động.....	83
6.4.6 Kịch bản nghiệm thu thủ công khi chạy hệ thống.....	83
6.5 Đánh giá hiệu năng chuỗi xử lý	85

Chương 7 Kết luận	86
7.1 Tổng kết đề tài	86
7.2 Kết quả chính đạt được	86
7.2.1 Chuỗi xử lý thị giác máy tính có cấu trúc	86
7.2.2 Hai cải tiến kỹ thuật so với phương pháp cơ sở	87
7.2.3 Bộ tổng hợp rủi ro có trạng thái và khả năng giải thích	87
7.2.4 Giám sát trình duyệt và nền tảng thời gian thực	87
7.2.5 Phân quyền và bảo vệ dữ liệu.....	87
7.2.6 Dữ liệu và báo cáo	88
7.3 Kết quả kiểm thử và thực nghiệm.....	88
7.4 Những hạn chế còn tồn tại	89
7.4.1 Ứng dụng thị giác máy tính và tiện ích mở rộng chưa hợp nhất hoàn toàn	89
7.4.2 Đồng bộ khi mất mạng chưa đầy đủ.....	89
7.4.3 Giới hạn của mô hình thị giác máy tính	89
7.4.4 Giới hạn của tiện ích mở rộng trình duyệt.....	89
7.4.5 Giới hạn về quy mô và vận hành.....	90
7.4.6 Khả năng tái lập thực nghiệm.....	90
7.4.7 Quyền riêng tư và công bằng.....	90
7.5 Hướng phát triển	90
7.6 Kết luận.....	91
Tài liệu tham khảo	92

DANH MỤC HÌNH VẼ

Hình 1 Tổng quan về thị giác máy tính	10
Hình 2 Kiến trúc MTCNN	14
Hình 3 State machine độc lập của một tín hiệu	37
Hình 4 Hysteresis ở cấp phiên thi	38
Hình 5 Sequence diagram của một phiên thi được giám sát.....	40
Hình 6 ERD rút gọn của lớp nền tảng.....	41
Hình 7 Sơ đồ luồng dữ liệu mức ngữ cảnh của hệ thống.....	42
Hình 8 Sơ đồ luồng dữ liệu chi tiết của một phiên thi	43
Hình 9 Sơ đồ luồng dữ liệu mức 1 của hệ thống	44
Hình 10 Giao diện đăng ký tài khoản và khởi tạo tổ chức.....	59
Hình 11 Giao diện đăng nhập của người dùng.	59
Hình 12 Giao diện xác thực MFA của người dùng.....	60
Hình 13 Giao diện thiết lập tài khoản và bảo mật cá nhân.	60
Hình 14 Giao diện tổng quan quản trị nền tảng dành cho quản trị viên hệ thống.	61
Hình 15 Giao diện chính sách, bảo mật và nhật ký cấp hệ thống	61
Hình 16 Giao diện quản lý tổ chức, thành viên và quyền truy cập.....	62
Hình 17 Giao diện chính sách, quyền truy cập của tổ chức.....	62
Hình 18 Giao diện danh sách và không gian làm việc của kỳ thi	63
Hình 19 Giao diện cấu hình và kiểm tra trạng thái sẵn sàng của kỳ thi.	63
Hình 20 Bảng điều khiển giám sát các phiên thi theo thời gian thực.	64
Hình 21 Giao diện chi tiết phiên thi, diễn biến theo thời gian và bằng chứng sự cố.....	64

Hình 22 Giao diện thiết lập tiện ích mở rộng trước khi tham gia kỳ thi.....	65
Hình 23 Sơ đồ triển khai các máy khách, dịch vụ và vùng dữ liệu	69
Hình 24 Đánh giá các metric sau khi fine-tune YOLOv8	74
Hình 25 Thay đổi các giá trị metric theo số lượng epoch.....	75

DANH MỤC BẢNG

Bảng 1 So sánh các phương pháp phát hiện khuôn mặt	17
Bảng 2 So sánh định hướng giải pháp	28
Bảng 3 Các tác nhân hệ thống	29
Bảng 4 Yêu cầu phi chức năng	31
Bảng 5 Nguyên tắc tổ chức giao diện	57
Bảng 6 Docker Compose	68
Bảng 7 Các quyết định cài đặt đáng chú ý.....	70
Bảng 8 Kết quả kiểm thử	78
Bảng 9 Đánh giá các kịch bản đầu-cuối	80
Bảng 10 Tổng hợp mức độ hoạt động.....	83
Bảng 11 Kịch bản nghiệm thu thủ công khi chạy hệ thống.....	83
Bảng 12 Đánh giá hiệu năng chuỗi xử lý.....	85
Bảng 13 Kết quả kiểm thử và thực nghiệm	88

DANH MỤC CÁC TỪ VIẾT TẮT

HTML	HyperText Markup Language Ngôn ngữ đánh dấu siêu văn bản
HTTPS	HyperText Transfer Protocol Secure Giao thức truyền tải siêu văn bản an toàn
MFA	Multi-Factor Authentication Xác thực đa yếu tố
RBAC	Role-Based Access Control Kiểm soát truy cập dựa trên vai trò
YOLOv8	You Only Look Once version 8
solvePnP	Solve Perspective-n-Point Giải Phối cảnh-n-điểm
API	Application Programming Interface Giao diện lập trình ứng dụng

DANH MỤC THUẬT NGỮ

Browser	Trình duyệt
MediaPipe	Thư viện xử lý đa phương tiện
FastAPI	Framework phát triển API
PostgreSQL	Hệ quản trị cơ sở dữ liệu
Redis	Kho dữ liệu trong bộ nhớ
Docker	Công cụ container hóa

MỞ ĐẦU

Trong những năm gần đây, thi trực tuyến đã trở thành xu hướng toàn cầu, được áp dụng trong hai bối cảnh chính: thi online từ xa (thí sinh ở nhà hoặc bất kỳ nơi nào khác) và thi online tại các phòng thi của các trung tâm thi nội bộ hoặc các trường học (thí sinh được quản lý ở một địa điểm tập trung nhưng vẫn sử dụng máy tính cá nhân thay vì giấy bút). Sự chuyển đổi này được thúc đẩy bởi nhu cầu tăng lên từ các trường đại học, các trung tâm thi chuẩn hóa quốc tế, các công ty tuyển dụng, và các tổ chức cấp chứng chỉ. Mặc dù thi trực tuyến mang lại nhiều lợi ích về hiệu quả vận hành và tiếp cận rộng hơn, nhưng nó cũng tạo ra thách thức đặc trưng cho mỗi bối cảnh.

Trong trường hợp thi online từ xa, thí sinh nằm ngoài sự kiểm soát trực tiếp của các giám thị, tạo ra rủi ro cao về gian lận. Thí sinh có thể sử dụng các tài liệu không được phép, tìm kiếm trên internet, liên lạc với người khác qua nhắn tin hoặc cuộc gọi, hoặc để người khác ngồi gần hỗ trợ. Trong các trường hợp nặng hơn, họ có thể để người khác thay mình làm bài thi hoàn toàn. Các báo cáo từ nhiều tổ chức giáo dục cho thấy tỷ lệ gian lận trong thi từ xa cao hơn đáng kể so với thi trực tiếp tại lớp học.

Tuy nhiên, ngay cả trong bối cảnh thi tại phòng thi của các trung tâm thi nội bộ, nơi các giám thị có mặt để giám sát, vẫn tồn tại những hạn chế. Giám thị trực tiếp chỉ có thể quan sát được các hành động rõ ràng và không thể ghi lại toàn bộ chi tiết hành vi của từng thí sinh trong khoảng thời gian dài. Họ cũng có thể bỏ sót những hành vi tinh vi như nhìn nhanh sang một thiết bị khác, hoặc những dấu hiệu vi phạm mà không dễ nhận thấy. Ngoài ra, cần có bằng chứng khách quan để hỗ trợ các quyết định kỷ luật, và một hệ thống ghi lại video có thể cung cấp điều đó. Hệ thống giám sát tự động có thể giúp các giám thị tập trung vào quản lý tổng thể phòng thi thay vì phải theo dõi chi tiết từng thí sinh, đồng thời cung cấp bằng chứng khách quan.

Nếu có thể phát triển một hệ thống giám sát trực tuyến thông minh, có khả năng hoạt động hiệu quả trong cả hai bối cảnh (từ xa và tại phòng thi tập trung), điều này sẽ mang lại lợi ích to lớn cho các bên liên quan. Đối với các tổ chức giáo dục, trung tâm thi và trường học, họ có thể bảo vệ tính liêm chính của các kỳ thi trong bất kỳ hình thức nào, cung cấp bằng chứng khách quan cho các quyết định kỷ luật, và nâng cao độ tin tưởng vào giá trị của bằng cấp được cấp. Đối với giáo viên và giám thị, thay vì phải giám sát thủ công từng thí sinh, họ có thể được hỗ trợ bởi các cảnh báo tự động về các hành vi nghi vấn, cho phép họ can thiệp kịp thời và hiệu quả. Đối với thí sinh, một hệ thống giám sát công bằng và nhất quán tạo ra môi

trường thi lành mạnh, nơi không ai có thể lợi dụng gian lận để đạt kết quả cao hơn những người thi chân chính.

Hơn nữa, một hệ thống giám sát như vậy có tiềm năng ứng dụng trong nhiều lĩnh vực khác ngoài giáo dục. Trong phỏng vấn trực tuyến cho tuyển dụng, các công ty có thể sử dụng để xác minh rằng ứng viên tự làm bài thử năng lực mà không nhận hỗ trợ hoặc tham khảo tài liệu trái phép. Trong các khoá học trực tuyến có bài kiểm tra hoặc các bài thi cấp chứng chỉ, các tổ chức cấp chứng chỉ có thể bảo vệ giá trị của chứng chỉ bằng cách giám sát điều kiện thi. Trong các công việc liên quan đến xác thực danh tính từ xa như mở tài khoản ngân hàng, giao dịch bảo hiểm, hoặc các giao dịch tài chính giá trị cao, các công ty có thể sử dụng công nghệ tương tự để xác minh rằng người giao dịch thực sự là người sở hữu tài khoản.

Tuy nhiên, thách thức kỹ thuật để xây dựng một hệ thống giám sát trực tuyến đáng tin cậy và linh hoạt vẫn còn rất lớn. Làm sao để phát hiện chính xác các hành vi gian lận trong vô số trường hợp khác nhau và trong cả hai môi trường thi (từ xa và tại phòng thi), từ các hành vi rõ ràng như vắng mặt hay nhìn ra ngoài, đến các hành vi tinh vi hơn như thay đổi danh tính hoặc tương tác với người khác? Làm sao để giảm thiểu số lần báo động giả, tránh gây phiền hà cho những thí sinh thi chân chính? Làm sao để các kỹ thuật này hoạt động được trên nhiều loại thiết bị, điều kiện ánh sáng khác nhau (từ căn phòng ở nhà đến phòng thi được thiết kế), và đa dạng hình dáng khuôn mặt của con người? Đây chính là những vấn đề cần được giải quyết để xây dựng một hệ thống giám sát thi trực tuyến có độ tin cậy cao và có tính ứng dụng thực tiễn trong cả hai bối cảnh thi.

CHƯƠNG 1 GIỚI THIỆU ĐỀ TÀI

Các giải pháp giám sát thi hiện có có thể được chia thành hai nhóm. Nhóm sản phẩm thương mại thường cung cấp nhiều chức năng như kiểm tra thiết bị, khóa hoặc theo dõi trình duyệt, ghi hình và hỗ trợ giám thị. Ưu điểm của nhóm này là quy trình vận hành tương đối hoàn chỉnh. Tuy nhiên, thuật toán phát hiện thường không được công bố đầy đủ, khả năng điều chỉnh, kiểm chứng độc lập và triển khai theo yêu cầu riêng có thể bị hạn chế.

Nhóm dự án mã nguồn mở và nghiên cứu học thuật tạo điều kiện khảo sát thuật toán và tái sử dụng mã nguồn. Một số dự án kết hợp OpenCV, MediaPipe, MTCNN và YOLO để nhận biết khuôn mặt, trạng thái mắt, chuyển động miệng hoặc vật thể. Tuy vậy, nhiều giải pháp dừng ở một chương trình cục bộ, xử lý từng điều kiện bằng chuỗi “if-elif” và ghi một nhãn cho mỗi khung hình. Cách làm này có bốn hạn chế chính:

1. Kết quả của một khung hình dễ bị ảnh hưởng bởi nhiễu và không biểu diễn được thời lượng hành vi.
2. Nhiều tín hiệu xảy ra đồng thời không được kết hợp theo mức độ quan trọng.
3. Một số phép đo dựa trên độ lệch điểm ảnh hoặc khoảng cách tuyệt đối phụ thuộc vào độ phân giải, tốc độ khung hình và vị trí khuôn mặt.
4. Ứng dụng thiếu lớp quản lý kỳ thi, phân quyền, theo dõi thời gian thực, lưu bằng chứng và đánh giá sau sự kiện.

Đề án tiếp cận khoảng trống trên ở cấp hệ thống thay vì chỉ ghép nối các mô hình nhận diện. Các mô hình được huấn luyện trước đóng vai trò thành phần nền. Phần thiết kế của đề án tập trung vào chuỗi xử lý dùng chung, hợp đồng dữ liệu, xử lý theo thời gian, tổng hợp rủi ro, xác thực lại danh tính trong phiên, tích hợp trình duyệt, phân quyền theo tài nguyên và vòng đời quản lý bằng chứng.

1.1 Mục tiêu nghiên cứu

1.1.1 Mục tiêu tổng quát

Mục tiêu của đề án là xây dựng một hệ thống giám sát thi trực tuyến có khả năng thu thập và tổng hợp nhiều tín hiệu bất thường. Hệ thống hỗ trợ giám thị theo dõi phiên thi theo thời gian thực và xem lại bằng chứng sau kỳ thi, đồng thời bảo đảm hệ thống có cấu trúc mô-đun, có thể kiểm thử, cấu hình và mở rộng.

1.1.2 Mục tiêu cụ thể

Để đạt mục tiêu tổng quát, đề án xác định các mục tiêu cụ thể sau:

- Xây dựng tầng nhận thức dùng chung để phát hiện khuôn mặt, trích xuất điểm mốc và phát hiện vật thể mà không chạy lặp cùng một mô hình cho từng tín hiệu.
- Cài đặt bảy tín hiệu gồm “FACE_PRESENCE”, “MULTI_FACE”, “EYE_STATE”, “MOUTH_STATE”, “OBJECT_PRESENCE”, “HEAD_POSE” và “IDENTITY” theo một giao diện lập trình thống nhất.
- Ước lượng ba góc quay đầu yaw, pitch và roll bằng bài toán Perspective-n-Point (PnP), thay cho phép so sánh độ lệch điểm ảnh đơn giản.
- Đăng ký và kiểm tra lại danh tính bằng véc-tơ đặc trưng khuôn mặt trong suốt phiên thi, kết hợp thử thách chớp mắt cơ bản ở bước đăng ký khuôn mặt.
- Xây dựng máy trạng thái theo cửa sổ thời gian cho từng tín hiệu và bộ tổng hợp rủi ro có trọng số, vùng trễ hai cấp và khả năng giải thích mức đóng góp của tín hiệu.
- Xây dựng tiện ích mở rộng trình duyệt để theo dõi trạng thái tập trung, chuyển thể, chế độ toàn màn hình, bảng tạm, camera, microphone và chia sẻ màn hình theo chính sách kỳ thi.
- Xây dựng dịch vụ FastAPI và bảng điều khiển để quản lý tổ chức, thành viên, kỳ thi, phiên thi, sự cố và báo cáo, truyền cập nhật bằng WebSocket.
- Thiết kế phân quyền theo vai trò, tổ chức và nhiệm vụ được giao trong từng kỳ thi, giới hạn quyền truy cập bằng chứng nhạy cảm và ghi nhật ký hoạt động.
- Kết hợp cơ sở dữ liệu quan hệ với JSONL và ảnh chụp bằng chứng để vừa truy vấn nhanh trạng thái mới nhất, vừa tái dựng diễn biến theo thời gian
- Đánh giá giải pháp bằng kiểm thử phần mềm, kiểm thử tích hợp và số liệu tổng hợp từ thực nghiệm 26 video có nhân tham chiếu được thực hiện trong một môi trường riêng.

1.2 Đối tượng và phạm vi nghiên cứu

1.2.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề án gồm:

- Các kỹ thuật phát hiện khuôn mặt, điểm mốc khuôn mặt, trạng thái mắt và miệng, vật thể, góc quay đầu và véc-tơ đặc trưng khuôn mặt.
- Phương pháp xử lý tín hiệu theo thời gian, máy trạng thái, tổng hợp có trọng số và vùng trễ.
- Cơ chế theo dõi tính toàn vẹn của phiên thi trong trình duyệt.

- Kiến trúc backend thời gian thực, phân quyền nhiều tổ chức, lưu bằng chứng và sinh báo cáo.
- Phương pháp đánh giá một hệ thống phát hiện sự kiện theo ground truth và phương pháp kiểm thử các thuộc tính phần mềm.

1.2.2 Phạm vi thực hiện

Các nội dung thuộc phạm vi đề án gồm:

- Webcam hướng về phía thí sinh, hình ảnh được xử lý trên ứng dụng máy tính để bàn viết bằng Python.
- Sử dụng các mô hình được huấn luyện trước gồm MTCNN, MediaPipe Face Landmarker, YOLOv8n trên COCO và InceptionResnetV1 trong facenet-pytorch.
- Phát hiện hai lớp vật thể của mô hình hiện tại là cell phone và book.
- Giám sát các sự kiện trình duyệt bằng tiện ích mở rộng, áp dụng chính sách đối với camera, microphone, chia sẻ màn hình, chế độ toàn màn hình, bảng tạm và phiên bản tiện ích.
- Hai phương thức tham gia của thí sinh: thông tin thủ công hoặc Google OIDC tùy cấu hình kỳ thi.
- Quản trị nhiều tổ chức với các vai trò System Admin, Organization Admin, Exam Manager và các nhiệm vụ owner/manager/proctor trên kỳ thi.
- Bảng điều khiển thời gian thực, xem bằng chứng, hậu kiểm sự cố, xuất báo cáo HTML/PDF, xử lý báo cáo nền và áp dụng chính sách lưu trữ.
- Chạy cục bộ hoặc triển khai dịch vụ phía máy chủ bằng Docker Compose với PostgreSQL và Redis.

1.2.3 Nội dung ngoài phạm vi

Đề án không đặt mục tiêu xây dựng một lockdown browser hoặc hệ thống remote attestation hoàn chỉnh. Các nội dung sau nằm ngoài phạm vi:

- Bảo đảm tuyệt đối client không bị sửa khi chạy trên thiết bị do thí sinh kiểm soát.
- Quan sát ứng dụng ngoài trình duyệt, điện thoại thứ hai hoặc thiết bị nằm ngoài góc nhìn camera.
- Phát hiện chắc chắn camera ảo, video replay hoặc deepfake bằng anti-spoofing chuyên dụng.
- Nhận dạng giọng nói, phân tích nội dung âm thanh hoặc truyền video liên tục đến giám thị.
- Huấn luyện lại toàn bộ các model nền từ đầu.
- Tự động kết luận hoặc áp dụng kỷ luật đối với thí sinh.

- Khẳng định khả năng triển khai quy mô lớn nhiều vùng địa lý khi chưa có đánh giá tải và hạ tầng tương ứng.

Việc xác định giới hạn này giúp kết quả đề án được diễn giải đúng: hệ thống cung cấp tín hiệu và bằng chứng hỗ trợ con người, không phải công cụ chứng minh gian lận một cách tuyệt đối.

1.3 Định hướng giải pháp

Để giải quyết những vấn đề nêu trên, tôi lựa chọn định hướng xây dựng một ứng dụng giám sát thi bằng thị giác máy tính chạy trên một máy tính cục bộ, tập trung vào chất lượng cao của pipeline computer vision thay vì mở rộng thành nền tảng đa người dùng phức tạp ở giai đoạn đầu. Lựa chọn này cho phép tập trung tài nguyên và thời gian vào việc xây dựng một sản phẩm CV có độ tin cậy cao với các kỹ thuật tiên tiến, đồng thời phù hợp với khuôn khổ thời gian thực hiện đề án tốt nghiệp có hạn. Định hướng này cũng tạo nền tảng vững chắc cho việc mở rộng sang nền tảng đa người dùng trong các giai đoạn phát triển tiếp theo.

Để triển khai định hướng này, đề án sẽ xây dựng pipeline CV theo kiến trúc ba tầng. Tầng thứ nhất, Perception Layer, sử dụng các thư viện và mô hình pretrained như MediaPipe FaceMesh, MTCNN (Multi-task Cascaded Convolutional Networks), và YOLOv8 (You Only Look Once v8) để trích xuất các đặc trưng cơ bản từ khung hình webcam, gồm các điểm mốc khuôn mặt, phát hiện và theo dõi khuôn mặt, cũng như phát hiện các vật thể trong bức ảnh. Tầng thứ hai, Signal Extractors, tính toán bảy tín hiệu từ các đặc trưng vừa trích xuất: face presence (có khuôn mặt hay không), multi-face (phát hiện nhiều khuôn mặt), eye state (mắt mở hay nhắm, dựa trên Eye Aspect Ratio), mouth state (miệng mở hay đóng), object presence (phát hiện vật thể cầm như điện thoại, sách), head pose (góc quay đầu, được tính bằng phương pháp solvePnP sử dụng mô hình khuôn mặt 3D), và identity verification (xác thực danh tính bằng face embedding từ FaceNet). Tầng thứ ba, Risk Fusion Engine, là thành phần tự thiết kế quan trọng nhất, dùng state machine theo cửa sổ thời gian trượt cho mỗi tín hiệu, kết hợp các tín hiệu thành một điểm rủi ro liên tục bằng weighted scoring, và sử dụng cơ chế hysteresis (hai ngưỡng khác nhau để phát hiện và hồi phục) để tránh báo động giả do dao động quanh ngưỡng phát hiện.

1.4 Các đóng góp chính

Các đóng góp nổi bật của đề án gồm:

1. Kiến trúc xử lý dùng chung kết quả nhận thức, giúp bảy tín hiệu không chạy lặp bộ phát hiện và sử dụng một hợp đồng dữ liệu thống nhất.

2. Ước lượng góc quay đầu bằng PnP, kiểm tra lại danh tính định kỳ và đăng ký khuôn mặt có thử thách chớp mắt cơ bản.
3. Cơ chế kết hợp hai cấp: máy trạng thái độc lập theo cửa sổ thời gian cho từng tín hiệu và bộ tổng hợp rủi ro có trọng số, vùng trễ ở cấp phiên.
4. Khả năng giải thích quyết định thông qua trạng thái, tín hiệu đóng góp, mức nghiêm trọng, diễn biến theo thời gian và ảnh chụp bằng chứng.
5. Kết hợp ứng dụng thị giác máy tính, tiện ích mở rộng trình duyệt và nền tảng quản lý thời gian thực mà không truyền video liên tục đến máy chủ.
6. Phân quyền nhiều tổ chức theo năng lực và phạm vi tài nguyên, kèm cơ chế cấp quyền đọc bằng chứng ngoại lệ có phê duyệt và thời hạn.
7. Mô hình dữ liệu lai SQL–JSONL, kết quả hậu kiểm tách khỏi sự kiện máy, tác vụ tạo báo cáo chạy nền và chính sách lưu trữ có nhật ký kiểm toán.
8. Bộ kiểm thử phần mềm nhiều tầng và phân phân tích định lượng trên bộ video có nhãn tham chiếu, kèm công bố giới hạn tái lập.

Kết quả đạt được sau cùng sẽ là một hệ thống giám sát thi trực tuyến hoàn chỉnh, hoạt động một cách độc lập trên máy tính cục bộ, có khả năng phát hiện các hành vi gian lận với độ tin cậy cao hơn đáng kể so với các phương pháp đơn giản hiện tại. Hệ thống sẽ cung cấp các báo cáo chi tiết về các vi phạm được phát hiện, kèm theo ảnh chứng minh và timeline chi tiết, giúp các giáo viên và giám thị đưa ra quyết định công bằng và có cơ sở khoa học. Ngoài ra, đồ án cũng tạo ra một kiến trúc, một tập hợp các thuật toán, và một phương pháp đánh giá định lượng rõ ràng, có thể được tái sử dụng hoặc mở rộng trong các nghiên cứu tương lai hoặc các ứng dụng thực tế khác.

Chi tiết về vấn đề, giải pháp và kết quả của từng đóng góp được trình bày tại Chương 4.

1.5 Bộ cục báo cáo

Phần còn lại của báo cáo Đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 giới thiệu về các nghiên cứu và sản phẩm hiện tại trong lĩnh vực giám sát thi trực tuyến, bao gồm các sản phẩm thương mại như Proctorio, Honorlock, ProctorU với mô tả chức năng và hạn chế, các dự án mã nguồn mở như exam-cheating-detection với phân tích chi tiết kiến trúc và các phương pháp được sử dụng, cũng như các nghiên cứu học thuật về các kỹ thuật liên quan như head pose estimation, face verification, behavioral analysis, và các phương pháp fusion đa tín hiệu trong các hệ thống bảo mật. Chương này sẽ phân tích và

so sánh chi tiết điểm mạnh, hạn chế, và lý do tại sao các phương pháp hiện tại chưa đủ để giải quyết vấn đề.

Trong Chương 3, trình bày các cơ sở lý thuyết cần thiết cho đề án, bao gồm những khái niệm cơ bản về thị giác máy tính, các kỹ thuật phát hiện và theo dõi khuôn mặt, phương pháp landmark detection để trích xuất các điểm mốc trên khuôn mặt, các phương pháp ước lượng góc đầu và đặc biệt là phương pháp solvePnP (Perspective-n-Point), kỹ thuật face embedding và similarity measurement để xác thực danh tính, những khái niệm cơ bản về xử lý tín hiệu và state machine, cũng như các phương pháp fusion dữ liệu từ nhiều nguồn khác nhau.

Chương 4 trình bày chi tiết thiết kế kiến trúc hệ thống đề xuất, bao gồm mô tả ba tầng chính (Perception Layer, Signal Extractors, Risk Fusion Engine) và lý do lựa chọn kiến trúc này, chi tiết cách tính toán từng trong số bảy tín hiệu (face presence, multi-face, eye state, mouth state, object presence, head pose, identity verification) với giải thích toán học cơ bản cho mỗi tín hiệu, mô tả thuật toán solvePnP cho ước lượng góc đầu, phương pháp FaceNet embedding cho xác thực danh tính, và đặc biệt là mô tả chi tiết Risk Fusion Engine bao gồm state machine theo cửa sổ thời gian trượt, phương pháp weighted scoring để tổng hợp đa tín hiệu, cơ chế hysteresis để giảm báo động giả, và cách tính toán điểm rủi ro liên tục theo thời gian.

Chương 5 trình bày việc cài đặt (implementation) các thành phần được thiết kế ở chương trước, bao gồm lựa chọn ngôn ngữ lập trình (Python) và các thư viện sử dụng, cách tích hợp MediaPipe, MTCNN, YOLOv8, FaceNet vào pipeline chính, xử lý các trường hợp đặc biệt (edge case) như không phát hiện khuôn mặt, nhiều khuôn mặt quá gần, điều kiện ánh sáng yếu, cấu trúc code, quản lý tài nguyên, và cách lưu trữ telemetry.

Chương 6 là phần thực nghiệm và đánh giá, trình bày kế hoạch quay bộ test (26 clip ngắn gần 2 tiếng đồng hồ) bao gồm các kịch bản khác nhau như hành vi bình thường, sử dụng điện thoại, nhìn ra ngoài, nhiều người trong khung hình, đối người giữa chừng, phương pháp gán nhãn ground truth chi tiết tại mức khung hình, cách tính toán các chỉ số Precision, Recall, F1 score, độ trễ phát hiện trung bình, kết quả thu được từ hệ thống đề xuất, và so sánh chi tiết với baseline đơn giản mô phỏng logic if-elif. Chương này cũng sẽ phân tích ảnh hưởng của các tham số như trọng số tín hiệu, ngưỡng hysteresis, và chiều dài cửa sổ thời gian lên hiệu năng.

Cuối cùng, Chương 7 đưa ra các kết luận chính từ đề án bao gồm những kết quả đạt được, những hạn chế của giải pháp hiện tại (như phụ thuộc vào điều kiện ánh sáng, khó xử lý các khuôn mặt có chướng ngại vật như kính), những giả định và điều kiện sử dụng, và các hướng phát triển tương lai có thể được thực hiện (như tích hợp backend để quản lý nhiều kỳ thi, thêm phát hiện các hành vi phức tạp khác, sử dụng deep learning cho Risk Fusion Engine, hoặc mở rộng sang các ứng dụng khác như phỏng vấn trực tuyến).

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

Chương này sẽ trình bày các cơ sở lý thuyết cần thiết như một nền tảng để hiểu các kỹ thuật được sử dụng trong đề án. Các nội dung chính bao gồm: (i) các khái niệm cơ bản về thị giác máy tính, (ii) các phương pháp phát hiện khuôn mặt, (iii) trích xuất các điểm mốc khuôn mặt (landmark detection), (iv) ước lượng góc đầu, (v) xác thực danh tính bằng face embedding, (vi) phát hiện các trạng thái của mắt và miệng, (vii) phát hiện vật thể, (viii) các kỹ thuật xử lý tín hiệu và state machine, và (ix) các phương pháp tổng hợp đa tín hiệu.

2.1 Thị giác máy tính

2.1.1 Định nghĩa và mục tiêu

Thị giác máy tính (computer vision) là một lĩnh vực của trí tuệ nhân tạo tập trung vào việc cho phép máy tính "nhìn" và "hiểu" thế giới thực qua các hình ảnh hoặc video. Mục tiêu cơ bản của thị giác máy tính là trích xuất thông tin hữu ích từ các dữ liệu hình ảnh (pixel data) thô, rồi chuyển đổi nó thành dạng mà máy tính có thể xử lý, phân tích, và ra quyết định [5]. Quá trình này thường được gọi là "quá trình hiểu ảnh" (image understanding), trong đó dữ liệu pixel một chiều được chuyển đổi thành các khái niệm ngữ nghĩa cao hơn mà con người có thể hiểu.

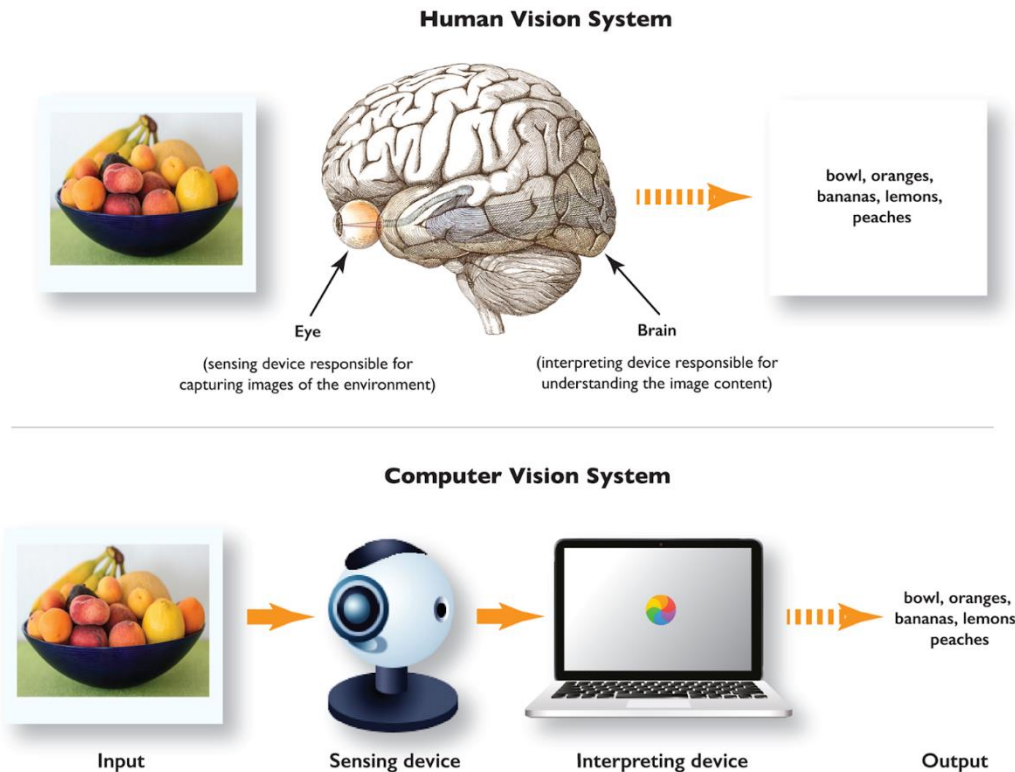
Quá trình xử lý hình ảnh trong thị giác máy tính thường bao gồm ba bước chính:

Bước 1: thu nhập dữ liệu (image acquisition). Bước này liên quan đến việc thu thập hình ảnh hoặc video từ các nguồn khác nhau như camera, webcam, hoặc ảnh tĩnh được lưu trữ. Chất lượng của dữ liệu thu nhập trực tiếp ảnh hưởng đến kết quả của các bước tiếp theo, vì vậy việc lựa chọn độ phân giải thích hợp, điều chỉnh các thông số camera, và đảm bảo điều kiện ánh sáng tốt là rất quan trọng.

Bước 2: xử lý và phân tích hình ảnh (image processing and analysis). Bước này áp dụng các kỹ thuật xử lý để trích xuất các đặc trưng quan trọng từ hình ảnh thô. Các kỹ thuật này có thể là đơn giản, như điều chỉnh độ sáng (brightness), tương phản (contrast), hoặc lọc nhiễu, hoặc phức tạp hơn, như phát hiện cạnh (edge detection), phân khúc ảnh (image segmentation), hoặc trích xuất các điểm đặc biệt (keypoint detection).

Bước 3: hiểu biết về nội dung (semantic understanding). Bước cuối cùng chuyển đổi các đặc trưng trích xuất thành các khái niệm có ý nghĩa mà con người có thể hiểu. Ví dụ, thay vì chỉ

có một tập hợp các giá trị pixel, hệ thống sẽ xác định được "có một khuôn mặt ở vị trí (x, y)", "khuôn mặt này thuộc người A", hoặc "người này đang nhắm mắt". Đây là phần khó nhất của thị giác máy tính vì yêu cầu sự kết hợp giữa các kỹ thuật học máy, toán học, và kiến thức miền (domain knowledge).



Hình 1 Tổng quan về thị giác máy tính

2.1.2 Biểu diễn dữ liệu hình ảnh

Một hình ảnh hay khung hình từ video có thể được biểu diễn dưới dạng một ma trận các số. Trong trường hợp đơn giản nhất, một ảnh đen trắng (grayscale image) có thể được biểu diễn dưới dạng một ma trận hai chiều, trong đó mỗi phần tử (pixel) chứa một giá trị từ 0 đến 255, đại diện cho mức độ sáng từ đen (0) đến trắng (255).

Tuy nhiên, hầu hết các ứng dụng thực tế làm việc với ảnh màu (color image), trong đó mỗi pixel được biểu diễn bằng ba giá trị số, tương ứng với ba kênh màu RGB (Red, Green, Blue). Do đó, một ảnh màu có thể được biểu diễn dưới dạng một ma trận ba chiều với kích thước chiều rộng $\times$ chiều cao $\times$ 3. Mỗi kênh màu có giá trị từ 0 đến 255, nên tổng cộng có $256^3 \approx 16.7$ triệu màu có thể được biểu diễn.

Ví dụ cụ thể: một hình ảnh có độ phân giải 640×480 pixel (chiều rộng $\times$ chiều cao) sẽ là một ma trận kích thước $640 \times 480 \times 3$. Tổng cộng, ma trận này chứa $640 \times 480 \times 3 = 921,600$ giá trị số. Khi xử lý ảnh hay video real-time (30 khung hình/giây), máy tính phải xử lý

$640 \times 480 \times 3 \times 30 \approx 27.6$ triệu giá trị mỗi giây, đó là lý do tại sao các kỹ thuật tối ưu hóa và sử dụng GPU (Graphics Processing Unit) là cần thiết.

2.1.3 Các phương pháp trong thị giác máy tính

Thị giác máy tính sử dụng nhiều phương pháp toán học và mô hình để trích xuất các đặc trưng có ý nghĩa từ ma trận pixel. Những phương pháp này có thể được phân thành ba nhóm lớn:

❖ *Phương pháp cổ điển (classical methods):*

Những phương pháp này đã được phát triển từ thập niên 1970-2000 và dựa trên các nguyên lý toán học rõ ràng. Chúng bao gồm:

- Xử lý ảnh cơ bản: Gaussian blur (làm mờ hình ảnh), edge detection (phát hiện cạnh) sử dụng toán tử Canny hoặc Sobel, corner detection (phát hiện góc) sử dụng toán tử Harris.
- Trích xuất đặc trưng: HOG (Histogram of Oriented Gradients) tính toán histogram các hướng của gradient tại các vị trí khác nhau trong ảnh, SIFT (Scale-Invariant Feature Transform) [6] phát hiện các điểm đặc trưng bất biến với tỷ lệ và góc quay, SURF (Speeded Up Robust Features) là một phiên bản nhanh hơn của SIFT.
- Phương pháp phát hiện: Haar Cascade [1] sử dụng cascade của các bộ phân loại Haar được huấn luyện bằng Adaboost, Deformable Part Models (DPM) mô hình vật thể thành các bộ phận có thể biến dạng.

Ưu điểm của các phương pháp cổ điển là nhanh, tiết kiệm tài nguyên tính toán, và có thể hiểu được tại sao chúng hoạt động như vậy. Nhược điểm là các phương pháp cổ điển kém chính xác hơn các phương pháp hiện đại, đặc biệt là trong các tình huống phức tạp hoặc có nhiều biến thể.

❖ *Phương pháp học máy (machine learning methods):*

Những phương pháp này sử dụng các mô hình học máy để học các mẫu từ dữ liệu huấn luyện. Chúng bao gồm:

- Support Vector Machines (SVM): Một mô hình phân loại tìm siêu phẳng tối ưu để tách các lớp dữ liệu.
- Random Forests: Một tập hợp các cây quyết định được kết hợp để đưa ra quyết định chung.
- Boosting methods: Các phương pháp như Adaboost kết hợp nhiều bộ phân loại yếu để tạo một bộ phân loại mạnh.

Các phương pháp học máy này cải thiện so với các phương pháp cổ điển nhưng vẫn kém so với các phương pháp học sâu hiện đại.

❖ **Phương pháp học sâu (deep learning methods):**

Những phương pháp này sử dụng các mạng neural sâu và là những phương pháp tiên tiến nhất hiện nay. Chúng bao gồm:

- Convolutional Neural Networks: Được phát triển từ lần đầu bởi LeCun et al. (1989), sau đó được cải tiến thông qua nhiều kiến trúc khác nhau. VGGNet [8] sử dụng các khối convolutional với kích thước filter nhỏ (3×3) nhưng nhiều lớp. ResNet giới thiệu các kết nối tắt (skip connections) để cho phép huấn luyện các mạng rất sâu. Inception sử dụng các khối convolutional song song với kích thước filter khác nhau.
- Region-based CNN: R-CNN [10] đề xuất các vùng (region proposal) và sau đó phân loại từng vùng. Fast R-CNN cải tiến việc tính toán bằng cách tính feature map một lần rồi trích xuất các tính năng cho mỗi vùng đề xuất. Faster R-CNN [11] thêm Region Proposal Network (RPN) để tạo ra các đề xuất một cách hiệu quả hơn.
- Single Shot Detection (SSD): YOLO (You Only Look Once) [3] phát hiện tất cả các vật thể trong ảnh bằng một lần forward pass duy nhất. SSD tương tự nhưng sử dụng các feature map ở nhiều tỷ lệ khác nhau. RetinaNet giới thiệu focal loss để xử lý sự mất cân bằng lớp (class imbalance) giữa các bộ phân loại dương tính (background) và âm tính (objects).
- Các mô hình khác: U-Net được sử dụng cho semantic segmentation (phân khúc ngữ nghĩa). Vision Transformer (ViT) áp dụng kiến trúc transformer, được phát triển thành công trong xử lý ngôn ngữ tự nhiên, vào thị giác máy tính.

Trong đồ án này, tôi sẽ sử dụng chủ yếu các phương pháp học sâu (deep learning) kết hợp với một số phương pháp cổ điển (như ước lượng góc đầu bằng solvePnP).

2.2 Phát hiện khuôn mặt

2.2.1 Tầm quan trọng và định nghĩa

Phát hiện khuôn mặt (face detection) là bước đầu tiên và rất quan trọng trong mọi hệ thống xử lý khuôn mặt [7]. Mục tiêu chính của phát hiện khuôn mặt là xác định xem trong một bức ảnh hay khung hình video có chứa một hoặc nhiều khuôn mặt hay không, và nếu có, hãy xác định vị trí cụ thể (bounding box) của chúng. Bounding box thường được biểu diễn dưới dạng (x, y, w, h) , trong đó (x, y) là tọa độ góc trái trên của hộp, w là chiều rộng, và h là chiều cao.

Tại sao phát hiện khuôn mặt lại quan trọng? Vì các bước tiếp theo trong pipeline xử lý khuôn mặt (như landmark detection, face recognition, emotion detection) đều phụ thuộc vào việc

đã phát hiện được khuôn mặt chính xác hay chưa. Nếu phát hiện sai lầm (bỏ sót khuôn mặt hoặc phát hiện nhầm vật thể khác là khuôn mặt), các bước tiếp theo sẽ cho kết quả sai. Hơn nữa, trong bối cảnh giám sát thi trực tuyến của đồ án, phát hiện khuôn mặt chính xác là tiền đề để có thể phát hiện được các hành vi gian lận như vắng mặt, nhiều người, hoặc đổi người.

2.2.2 Lịch sử phát triển

Phát hiện khuôn mặt là một bài toán được nghiên cứu từ thập niên 1990. Có nhiều phương pháp được phát triển qua các năm, từ những phương pháp cổ điển đến các phương pháp học sâu hiện đại [7]:

Giai đoạn 1: Phương pháp cổ điển (1990s-2000s)

- Haar Cascade Classifiers: được đề xuất bởi Viola và Jones [1] vào năm 2001, phương pháp này sử dụng cascade của các bộ phân loại Haar (Haar-like features) được huấn luyện bằng Adaboost. Đây là một phương pháp đột phá lúc bấy giờ và được sử dụng rộng rãi trong OpenCV đến ngày nay. Ưu điểm: nhanh, tài nguyên ít, phù hợp cho các ứng dụng real-time. Nhược điểm: độ chính xác kém hơn các phương pháp hiện đại, khó xử lý các tình huống khó như khuôn mặt bị quay với góc lớn, che khuất một phần, hoặc chiếu sáng không tốt.
- HOG: Một phương pháp trích xuất đặc trưng được sử dụng kết hợp với SVM để phát hiện người (thu viện dlib) [9]. Phương pháp này cũng có thể được sử dụng cho phát hiện khuôn mặt. Ưu điểm: tương đối chính xác. Nhược điểm: chậm hơn Haar Cascade, vẫn kém so với các phương pháp học sâu.

Giai đoạn 2: Phương pháp học sâu (2010s)

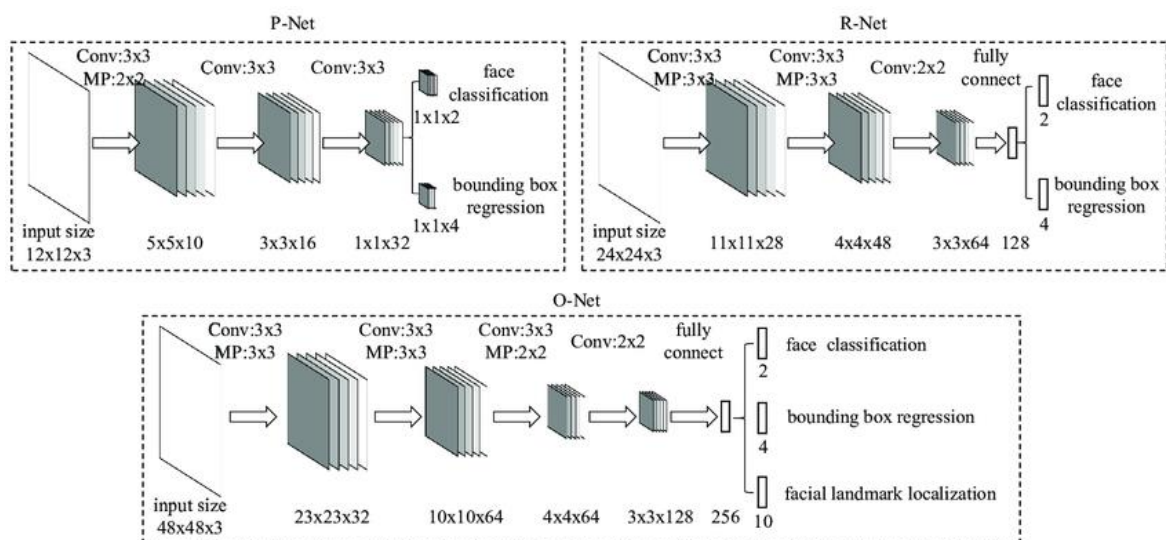
- Region-based CNN: R-CNN [10] được đề xuất bởi Girshick et al. (2014) là phương pháp đột phá đầu tiên sử dụng deep learning cho phát hiện vật thể. Phương pháp này sau đó được áp dụng cho phát hiện khuôn mặt. R-CNN sử dụng region proposal (đề xuất vùng) để tìm các vùng có khả năng chứa vật thể, rồi phân loại từng vùng bằng CNN. Ưu điểm: chính xác cao. Nhược điểm: chậm (phải chạy CNN cho mỗi region proposal).
- Fast R-CNN: Cải tiến của R-CNN bằng cách tính feature map một lần rồi trích xuất các tính năng cho mỗi region proposal, nhanh hơn nhiều so với R-CNN gốc.
- Faster R-CNN [11]: Thêm Region Proposal Network (RPN) để tạo ra các region proposal một cách hiệu quả hơn. Phương pháp này trở thành một trong những phương pháp phát hiện vật thể tốt nhất và được sử dụng rộng rãi.
- YOLO [3]: Được đề xuất bởi Redmon et al. (2016), YOLO là một phương pháp single-shot detection cho phép phát hiện nhanh tất cả các vật thể trong ảnh bằng một lần forward pass duy nhất. Mặc dù YOLO được thiết kế cho phát hiện vật thể chung

(chó, mèo, xe, v.v.), nó cũng có thể được huấn luyện cho phát hiện khuôn mặt bằng fine-tuning trên dataset khuôn mặt. Ưu điểm: rất nhanh, phù hợp cho real-time. Nhược điểm: độ chính xác kém hơn Faster R-CNN, đặc biệt là với các khuôn mặt nhỏ.

- SSD: Tương tự như YOLO, là một phương pháp single-shot, nhưng sử dụng các feature map ở nhiều tỷ lệ khác nhau để phát hiện các vật thể có kích thước đa dạng.
- MTCNN Được thiết kế đặc biệt cho phát hiện khuôn mặt, được giới thiệu bởi Zhang et al. [2] vào năm 2016. Phương pháp này sẽ được trình bày chi tiết ở mục 2.2.3.
- RetinaFace: Một phương pháp phát hiện khuôn mặt hiện đại [4] được đề xuất bởi Deng et al. (2020), kết hợp các ý tưởng từ Faster R-CNN với FPN (Feature Pyramid Network) để xử lý khuôn mặt có kích thước đa dạng.

2.2.3 Multi-task Cascaded Convolutional Networks

Multi-task Cascaded Convolutional Networks (MTCNN) [2] là một trong những phương pháp phát hiện khuôn mặt tốt nhất và được sử dụng rộng rãi. Nó được thiết kế để hoạt động tốt trong các tình huống khó như khuôn mặt bị che khuất một phần, góc quay lớn, hoặc chiếu sáng không tốt. MTCNN được phát triển với hai đặc điểm chính: (i) sử dụng cascade (tầng xếp tầng) của nhiều mạng CNN, và (ii) mỗi mạng giải quyết nhiều nhiệm vụ (multi-task) cùng lúc.



Hình 2 Kiến trúc MTCNN

MTCNN bao gồm ba giai đoạn (stage), mỗi giai đoạn là một mạng CNN riêng biệt. Ba giai đoạn này được thiết kế để dần dần tinh chỉnh các kết quả:

Giai đoạn 1: Proposal Network (P-Net)

Mục tiêu: Tạo ra các hộp đề xuất ban đầu (region proposal) có thể chứa khuôn mặt.

Quá trình hoạt động:

- Đầu vào: Hình ảnh gốc với các kích thước khác nhau (image pyramid). Tại mỗi kích thước, P-Net quét qua toàn bộ hình ảnh với sliding window kích thước 12×12 .
- Tính toán: Tại mỗi vị trí sliding window, mạng P-Net tính toán ba đầu ra: (i) xác suất có khuôn mặt tại vị trí này (face probability), (ii) tọa độ điều chỉnh bounding box (bounding box regression), (iii) tọa độ của 5 điểm mốc khuôn mặt sơ bộ (landmark regression).
- Loại bỏ: Các hộp có xác suất $<$ ngưỡng (thường 0.5) được loại bỏ. Các hộp còn lại được hợp nhất (NMS - Non-Maximum Suppression) để loại bỏ các hộp trùng lặp.
- Đầu ra: Một danh sách các hộp đề xuất có xác suất cao và các điểm mốc sơ bộ.

Giai đoạn 2: Refine Network (R-Net)

Mục tiêu: Loại bỏ các hộp đề xuất sai lầm (false positives) và tinh chỉnh vị trí cũng như các điểm mốc của các hộp còn lại.

Quá trình hoạt động:

- Đầu vào: Các hộp đề xuất từ P-Net. Các hộp này được cắt từ hình ảnh gốc và resize về kích thước chuẩn 24×24 .
- Tính toán: R-Net là một mạng CNN lớn hơn P-Net, có kiến trúc tương tự nhưng với nhiều lớp hơn. Nó tính toán ba đầu ra: face probability, bounding box regression, và landmark regression.
- Loại bỏ: Các hộp có xác suất $<$ ngưỡng được loại bỏ. Các hộp còn lại được hợp nhất bằng NMS.
- Tinh chỉnh: Bounding box và các điểm mốc được tinh chỉnh dựa trên đầu ra của R-Net.
- Đầu ra: Một danh sách nhỏ hơn các hộp có chất lượng cao.

Giai đoạn 3: Output Network (O-Net)

Mục tiêu: Tạo ra kết quả cuối cùng với độ chính xác cao nhất và trích xuất chính xác 5 điểm mốc khuôn mặt.

Quá trình hoạt động:

- Đầu vào: Các hộp đề xuất từ R-Net, được resize về kích thước chuẩn 48×48 (kích thước cao nhất trong ba giai đoạn, cho phép phát hiện chi tiết hơn).
- Tính toán: O-Net là mạng lớn nhất trong ba giai đoạn. Nó tính toán bốn đầu ra:
 - Face probability: xác suất có khuôn mặt

- Bounding box regression: điều chỉnh vị trí bounding box
- Face landmarks: 5 điểm mốc trên khuôn mặt (cụ thể là: hai mắt, mũi, hai khóe miệng)
- Face pose (tùy chọn): góc quay đầu
- Đầu ra: Danh sách final các khuôn mặt được phát hiện với: vị trí bounding box, xác suất, 5 điểm mốc, và các thông tin khác.

Ưu điểm của MTCNN:

1. Độ chính xác cao: MTCNN đạt tỷ lệ phát hiện rất cao trên nhiều benchmark, bao gồm WIDER Face dataset [18], một trong những dataset lớn nhất cho phát hiện khuôn mặt với hơn 393,703 khuôn mặt được gán nhãn.
2. Xử lý tốt các tình huống khó: Mặc dù có cascade của ba giai đoạn, MTCNN vẫn có thể xử lý tốt các khuôn mặt bị quay với góc lớn, che khuất một phần bởi các vật khác (kính, khẩu trang), hoặc chiếu sáng không tốt (thiếu sáng hoặc quá sáng).
3. Trích xuất landmark sơ bộ: Mỗi giai đoạn P-Net, R-Net, O-Net đều trích xuất 5 điểm mốc, và điểm mốc từ O-Net được sử dụng cho các bước xử lý tiếp theo (như ước lượng góc đầu, xác thực danh tính).
4. Tốc độ xử lý chấp nhận được: Nhờ cascade design (loại bỏ các false positive từ sớm ở P-Net), MTCNN đạt tốc độ xử lý tương đối nhanh so với các phương pháp khác như RetinaFace, mặc dù chậm hơn YOLO.

Nhược điểm của MTCNN:

1. Cần ba lần forward pass: Phải chạy ba mạng CNN riêng biệt (P-Net, R-Net, O-Net), tổng cộng chậm hơn single-stage methods như YOLO.
2. Phụ thuộc vào chất lượng detection của P-Net: Nếu P-Net bỏ sót một khuôn mặt (đặc biệt là khuôn mặt nhỏ hoặc bị quay với góc rất lớn), các giai đoạn R-Net và O-Net không thể phục hồi (vì chúng chỉ làm việc trên các proposal từ P-Net).
3. Nhạy cảm với kích thước khuôn mặt rất nhỏ: Khó phát hiện các khuôn mặt rất nhỏ trong ảnh (ví dụ khuôn mặt ở xa trong một ảnh phòng học toàn cảnh).
4. Cần điều chỉnh các ngưỡng: Các ngưỡng xác suất (thường 0.5) được sử dụng trong P-Net, R-Net, O-Net có thể cần điều chỉnh tùy theo bài toán cụ thể (ví dụ có thể giảm ngưỡng để phát hiện khuôn mặt nhỏ hơn, nhưng điều này có thể tăng false positive).

2.2.4 So sánh các phương pháp phát hiện khuôn mặt

Bảng dưới đây tóm tắt so sánh các phương pháp phát hiện khuôn mặt chính:

Bảng 1 So sánh các phương pháp phát hiện khuôn mặt

Phương pháp	Loại	Độ chính xác	Tốc độ	Xử lý tình huống khó	Trích xuất landmark
Haar Cascade [1]	Cổ điển	Thấp	Rất nhanh	Kém	Không
HOG + SVM [9]	Cổ điển	Trung bình	Chậm	Kém	Không
R-CNN [10]	Deep learning	Cao	Rất chậm	Tốt	Có thể
Faster R-CNN [11]	Deep learning	Rất cao	Chậm	Rất tốt	Có thể
YOLO [3]	Deep learning	Cao	Rất nhanh	Trung bình	Không
MTCNN [2]	Deep learning	Rất cao	Chậm	Rất tốt	Có (5 điểm)
RetinaFace [4]	Deep learning	Rất cao	Chậm	Rất tốt	Có (5 điểm)

2.2.5 Lựa chọn phương pháp cho đồ án

Trong đồ án này, tôi sử dụng MTCNN làm phương pháp chính để phát hiện khuôn mặt. Quyết định này dựa trên những lý do sau:

1. Độ chính xác cao và xử lý tốt tình huống khó: MTCNN đã được chứng minh hiệu quả trong các ứng dụng thực tế của face detection, đặc biệt là khi khuôn mặt có góc quay lớn hoặc bị che khuất một phần, đây là những tình huống phổ biến trong giám sát thi trực tuyến (ví dụ thí sinh quay đầu nhìn sang bên, hoặc đeo kính).
2. Có sẵn các thư viện Python: có các triển khai sẵn của MTCNN trong Python (thư viện mtcnn, facenet-pytorch) với giao diện dễ sử dụng, tiết kiệm thời gian phát triển.
3. Tốc độ xử lý chấp nhận được: mặc dù chậm hơn YOLO, MTCNN vẫn có thể xử lý ở tốc độ gần real-time (15-25 khung hình/giây trên CPU thông thường, hoặc 30+ FPS trên GPU), đủ cho ứng dụng giám sát thi.

4. Trích xuất sẵn 5 điểm landmark: MTCNN đã trích xuất 5 điểm mốc khuôn mặt chính xác, những điểm này được sử dụng trực tiếp cho các bước tiếp theo như ước lượng góc đầu (solvePnP) hoặc phát hiện trạng thái mắt (EAR).
5. Tương thích tốt với các thư viện khác: MTCNN tương thích tốt với MediaPipe FaceMesh (sẽ được trình bày ở mục 2.3) và các thư viện khác mà đồ án sử dụng, cho phép tích hợp dễ dàng.

Nếu trong tương lai có nhu cầu tăng độ chính xác (ưu tiên hơn tốc độ), có thể xem xét RetinaFace [4]. Nếu cần tốc độ rất cao (ưu tiên hơn độ chính xác), có thể sử dụng YOLOv8 [21] với fine-tuning trên dataset khuôn mặt.

2.3 Nhận diện điểm mốc khuôn mặt với MediaPipe

Điểm mốc khuôn mặt (facial landmark) là tập các điểm đặc trưng mô tả đường viền mắt, môi, mũi, cằm và bề mặt khuôn mặt. MediaPipe Face Mesh/Face Landmarker dự đoán 468 điểm bề mặt từ ảnh của một camera và có thể chạy gần thời gian thực [3], [11].

Tọa độ MediaPipe được chuẩn hóa theo kích thước ảnh. Với điểm mốc $p_i = (x_i, y_i)$ đã chuẩn hóa và khung hình có kích thước (W, H) , tọa độ điểm ảnh là:

$$p_i^{pixel} = (x_i W, y_i H)$$

Phép quy đổi này cần thiết khi tính khoảng cách Euclid. Nếu sử dụng trực tiếp tọa độ chuẩn hóa trên ảnh không vuông, hai trục có tỉ lệ khác nhau và đại lượng hình học sẽ bị sai lệch. Đồ án sử dụng tọa độ điểm ảnh cho EAR, tỉ lệ mở miệng và PnP.

Không phải mọi tín hiệu đều cần điểm mốc chi tiết của nhiều khuôn mặt. Face Landmarker chỉ theo dõi khuôn mặt chính để phục vụ tín hiệu mắt, miệng và góc quay đầu; MTCNN chịu trách nhiệm đếm khuôn mặt. Cách phân tách này giảm chi phí và làm rõ trách nhiệm của từng mô hình.

2.4 Ước lượng góc quay đầu bằng Perspective-n-Point

Ước lượng góc đầu (head pose estimation) là khả năng xác định hướng mà đầu người đang quay, thường được biểu diễn bằng ba góc: yaw (quay trái-phải), pitch (quay lên-xuống), và roll (quay cân bằng). Phát hiện các thay đổi lớn trong góc đầu có thể chỉ ra rằng thí sinh đang nhìn sang nơi khác, có thể là nhìn ra ngoài khung hình hoặc nhìn sang một thiết bị khác.

Phương pháp Perspective-n-Point (solvePnP) là một kỹ thuật cổ điển trong thị giác máy tính để ước lượng góc đầu dựa trên tương ứng giữa các điểm 2D trên ảnh và các điểm 3D của một mô hình khuôn mặt chuẩn. Cụ thể, phương pháp này sử dụng sáu điểm landmark chính

(hai mắt, mũi, hai khoé miệng, cằm) từ ảnh 2D và so khớp chúng với một mô hình khuôn mặt 3D đã được biết trước. Bằng cách giải bài toán hình học này, phương pháp solvePnP có thể tính toán chính xác các ma trận quay (rotation matrix) và vectơ chuyển dịch (translation vector), từ đó suy ra góc yaw, pitch, roll.

Ưu điểm của solvePnP so với các phương pháp khác (như chỉ dựa trên sự lệch vị trí pixel) là nó tận dụng thông tin về hình học 3D của khuôn mặt, cho phép ước lượng chính xác ngay cả khi khuôn mặt có góc quay lớn. Tuy nhiên, độ chính xác của phương pháp này phụ thuộc vào chất lượng của mô hình khuôn mặt 3D chuẩn và độ chính xác của camera intrinsic parameters (focal length, principal point, v.v.).

2.5 Xác thực danh tính bằng face embedding

2.5.1 Biểu diễn véc-tơ đặc trưng

Xác minh khuôn mặt (face verification) trả lời câu hỏi hai ảnh có thuộc cùng một người hay không. FaceNet học ánh xạ $f(x) \in R^d$ từ ảnh mặt sang véc-tơ đặc trưng bằng hàm mất mát bộ ba [5]. Với mẫu neo a , mẫu cùng người p và mẫu khác người n :

$$L = \max(0, ||f(a) - f(p)||_2^2 - ||f(a) - f(n)||_2^2 + \alpha)$$

Mục tiêu là đưa véc-tơ của cùng một người lại gần nhau và đẩy véc-tơ của người khác ra xa ít nhất một biên α .

Đồ án sử dụng InceptionResnetV1 đã được huấn luyện trước trên VGGFace2 thông qua *facenet-pytorch* để tạo véc-tơ 512 chiều. Trong bước đăng ký, véc-tơ tham chiếu là trung bình của nhiều khung hình hợp lệ nhằm giảm ảnh hưởng của một quan sát riêng lẻ.

2.5.2 Cosine similarity và quyết định có biên

Độ tương tự cosine giữa hai vector x và y là:

$$sim(x, y) = \frac{x \cdot y}{||x||_2 ||y||_2}$$

Giá trị cao cho biết hai hướng véc-tơ gần nhau. Ngưỡng xác thực phụ thuộc vào mô hình và miền dữ liệu, do đó không có một giá trị phổ quát. Thiết kế nên có vùng cảnh báo và vùng không khớp thay vì một ngưỡng duy nhất. Đồ án còn yêu cầu nhiều kết quả không khớp liên tiếp và chỉ kiểm tra lại theo chu kỳ để tránh tạo véc-tơ đặc trưng trên mọi khung hình.

Nếu không tìm thấy khuôn mặt tại thời điểm kiểm tra lại, hệ thống không được đồng nhất tình huống này với “không đúng người”. Tín hiệu hiện diện khuôn mặt chịu trách nhiệm cho sự vắng mặt; tín hiệu danh tính chỉ đánh giá khi có véc-tơ hợp lệ để so sánh.

2.5.3 Liveness cơ bản

Xác minh khuôn mặt có thể bị đánh lừa bởi ảnh tĩnh nếu không kiểm tra sự hiện diện sống. Thử thách chớp mắt *mở* $\rightarrow$ *nhắm* $\rightarrow$ *mở* xác nhận một chuyển động đơn giản trước bước đăng ký. Giải pháp này có thể ngăn một số trường hợp sử dụng ảnh in hoặc ảnh trên màn hình, nhưng không chống được phát lại video, mặt nạ hoặc nội dung giả mạo tinh vi. Vì vậy, đây chỉ là kiểm tra sự hiện diện sống cơ bản, không phải cơ chế chống giả mạo hoàn chỉnh.

2.6 Phát hiện trạng thái mắt bằng EAR

Tỉ lệ khung hình mắt (Eye Aspect Ratio – EAR) là đại lượng hình học được Soukupová và Čech đề xuất để biểu diễn độ mở mắt [4]. Với sáu điểm $P_1, \dots, P_6$ quanh một mắt:

$$EAR = \frac{\|P_2 - P_6\|_2 + \|P_3 - P_5\|_2}{2\|P_1 - P_4\|_2}$$

Từ số đo hai khoảng mở theo chiều dọc, mẫu số đo chiều rộng mắt. Vì là một tỉ lệ, EAR ít phụ thuộc hơn vào kích thước khuôn mặt trên ảnh. Khi mắt khép, khoảng cách dọc giảm và EAR giảm.

EAR không đo hướng nhìn. Một người quay đầu có thể làm một mắt bị nén phối cảnh và tạo EAR thấp giả. Do nhắm mắt tự nhiên thường ảnh hưởng cả hai mắt, đồ án chỉ coi trạng thái nhắm khi EAR của cả hai mắt cùng dưới ngưỡng. Head Pose Signal đảm nhận việc nhận biết quay đầu.

Ngưỡng EAR không phải hằng số cho mọi người và mọi camera. Nó cần được kết hợp với thời lượng nhắm và được hiệu chỉnh trên dữ liệu thực nghiệm. Một frame có EAR thấp không đủ để tạo cảnh báo.

2.7 Phát hiện trạng thái miệng

Miệng mở có thể đo bằng tỉ lệ giữa khoảng cách dọc của môi trong và khoảng cách hai khóm miệng:

$$MOR = \frac{\|P_{upper} - P_{lower}\|_2}{\|P_{left} - P_{right}\|_2}$$

Trong đó MOR là Mouth Open Ratio. Việc chia cho chiều rộng miệng giúp đại lượng ít phụ thuộc vào khoảng cách giữa thí sinh và camera.

Khác với ngáp, nói chuyện làm miệng mở và đóng xen kẽ. Nếu thuật toán yêu cầu miệng mở liên tục trong nhiều giây, hành vi nói có thể không bao giờ vượt ngưỡng. Vì vậy cần theo dõi tỉ lệ quan sát miệng mở trong cửa sổ thời gian:

$$a_{mouth}(t) = \frac{N_{open}(t - W, t)}{N_{valid}(t - W, t)}$$

Khi a_{mouth} vượt ngưỡng và cửa sổ đã có đủ độ phủ, tín hiệu mới được coi là bất thường. Cách này không phụ thuộc trực tiếp vào số FPS và phản ánh đúng hơn mẫu mở–đóng lặp lại.

Tuy nhiên, chuyển động miệng không chứng minh thí sinh trao đổi nội dung. Tín hiệu này chỉ đóng vai trò phụ và cần được giám thị đối chiếu với tín hiệu khác.

2.8 Phát hiện vật thể

Phát hiện vật thể (object detection) là khả năng xác định các vật thể cụ thể trong một ảnh và xác định loại cũng như vị trí của chúng. Trong bối cảnh giám sát thi, các vật thể quan trọng cần phát hiện là các vật thể cầm hoặc đáng ngờ như điện thoại di động, sách, máy tính bảng, v.v.

YOLOv8 [21] là một mô hình phát hiện vật thể hiện đại, nổi tiếng vì khả năng phát hiện nhanh và chính xác. YOLO sử dụng một mạng CNN duy nhất để phát hiện tất cả các vật thể trong một ảnh cùng một lúc, thay vì các phương pháp cũ dựa trên region proposal. YOLOv8 được huấn luyện trên dataset COCO [17] lớn chứa hơn 80 loại vật thể khác nhau, bao gồm các vật thể như điện thoại, sách, laptop, v.v. Ngoài ra, YOLOv8 có thể được fine-tune trên dataset riêng để phát hiện các loại vật thể cụ thể (như điện thoại bất kỳ loại nào) với độ chính xác cao hơn.

2.9 Xử lý tín hiệu theo thời gian

2.9.1 Debounce dựa trên thời lượng

Cơ chế chống dao động chỉ chấp nhận một điều kiện khi điều kiện đó duy trì đủ lâu. Nếu $q(t) \in \{0,1\}$ biểu diễn điều kiện thô và D là thời lượng tối thiểu, tín hiệu chỉ vượt ngưỡng khi $q(t) = 1$ liên tục trong khoảng D . Cách đo bằng giây ổn định hơn đếm số khung hình vì tốc độ khung hình có thể thay đổi theo thiết bị và tải xử lý.

2.9.2 Cửa sổ thời gian trượt

Với cửa sổ độ dài W , tỉ lệ bất thường của tín hiệu i tại thời điểm t là:

$$r_i(t) = \frac{\sum_{\tau \in (t-W, t]} 1[q_i(\tau) = 1]}{\sum_{\tau \in (t-W, t]} 1[q_i(\tau) \text{ hợp lệ}]}$$

Cửa sổ trượt giữ lại lịch sử gần nhất, loại bỏ ảnh hưởng lâu dài của sự kiện cũ và hỗ trợ các mẫu không liên tục như hoạt động miệng.

2.9.3 Máy trạng thái hữu hạn

Máy trạng thái hữu hạn (Finite State Machine – FSM) biểu diễn hệ thống bằng tập trạng thái và điều kiện chuyển. Ba trạng thái NORMAL, SUSPICIOUS, ALERT cho phép phân biệt quan sát ban đầu với bằng chứng kéo dài. Máy trạng thái của mỗi tín hiệu hoạt động độc lập để nhiều hành vi có thể được ghi nhận đồng thời.

2.10 Kết hợp đa tín hiệu và vùng trễ

Tổng hợp có trọng số:

Sau khi ổn định từng tín hiệu, trạng thái có thể ánh xạ thành giá trị s_i và kết hợp bằng tổng có trọng số:

$$R(t) = \sum_{i=1}^n w_i s_i(t)$$

Trọng số w_i biểu diễn mức đóng góp tương đối. Tổng hợp có trọng số có ưu điểm dễ giải thích, dễ cấu hình và không đòi hỏi tập huấn luyện lớn. Hạn chế của phương pháp là quan hệ tuyến tính không tự học được các tương tác phức tạp giữa các tín hiệu.

Vùng trễ:

Nếu chỉ dùng một ngưỡng T , nhiễu quanh T làm trạng thái bật/tắt liên tục. Vùng trễ (hysteresis) sử dụng hai ngưỡng:

$$\text{state}(t) = \begin{cases} \text{ALERT}, & \text{state}(t-1) = \text{NORMAL} \wedge R(t) \geq T_{\text{enter}} \\ \text{NORMAL}, & \text{state}(t-1) = \text{ALERT} \wedge R(t) \leq T_{\text{exit}} \\ \text{state}(t-1), & \text{các trường hợp còn lại} \end{cases}$$

với $T_{\text{exit}} < T_{\text{enter}}$. Khoảng $(T_{\text{exit}}, T_{\text{enter}})$ là vùng đệm giữ nguyên trạng thái cũ. Công thức trên minh họa quyết định hai trạng thái ở cấp phiên; máy trạng thái ba mức của từng tín hiệu sử dụng các điều kiện chuyển chi tiết hơn. Vùng trễ có thể được áp dụng ở cả cấp tín hiệu và cấp phiên.

Một cảnh báo kéo dài không nên tạo một sự kiện ở mọi khung hình. Sự kiện được sinh tại cạnh chuyển vào trạng thái cảnh báo; thời lượng có thể suy ra từ bản ghi chuyển trạng thái trở về bình thường. Cách này giảm kích thước nhật ký và tránh hiển thị nhiều bản ghi cho cùng một hành vi.

2.11 Giao tiếp thời gian thực và đồng bộ dữ liệu

REST phù hợp với thao tác yêu cầu–phản hồi như đăng nhập, tạo kỳ thi và lấy danh sách phiên. WebSocket theo RFC 6455 [9] duy trì kênh song song để máy khách gửi nhịp kết nối và dữ liệu giám sát, còn máy chủ đẩy cập nhật đến bảng điều khiển mà không cần thăm dò liên tục.

Kênh thời gian thực không loại bỏ nhu cầu xác thực và kiểm tra dữ liệu. Một thông điệp từ máy khách cần:

- Có loại và lược đồ xác định.
- Giới hạn kích thước, tần suất và số phần tử.
- Phân biệt dấu thời gian do máy khách cung cấp với thời gian máy chủ tiếp nhận.
- Gắn với đúng loại token và đúng phiên.
- Có cơ chế nhịp kết nối, thời hạn chờ khi không hoạt động và trạng thái mất kết nối.

Đồng bộ dữ liệu giúp các thành phần thống nhất ý nghĩa của trường dữ liệu. `SignalResult`, `ViolationEvent` và sự kiện trình duyệt phải dùng giá trị liệt kê cố định, số hữu hạn và siêu dữ liệu có giới hạn. Máy chủ có thể tính lại hoặc đối chiếu giá trị dẫn xuất thay vì tin trực tiếp điểm do máy khách gửi.

2.11.1 Phân quyền người dùng

Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control – RBAC) gán quyền cho vai trò thay vì từng người dùng riêng lẻ [10]. Trong nền tảng nhiều tổ chức, chỉ kiểm tra vai trò là chưa đủ, quyết định còn phụ thuộc vào tổ chức và tài nguyên cụ thể. Có thể mô hình hóa quyền hiệu lực như:

$$Allowed(u, a, r) = Authenticated(u) \wedge Active(u)$$

$$TenantMatch(u, r) \wedge Capability(u, a) \wedge ResourceScope(u, r)$$

Trong đó u là người dùng, a là hành động và r là tài nguyên. Nhiệm vụ được giao trên kỳ thi là một dạng phạm vi tài nguyên: người dùng có thể có vai trò quản lý ở kỳ thi A nhưng chỉ có vai trò giám thị ở kỳ thi B.

Nguyên tắc đặc quyền tối thiểu yêu cầu chỉ cấp các quyền cần thiết. Quản trị viên nền tảng không nên mặc nhiên được xem bằng chứng của mọi tổ chức. Quyền ngoại lệ cần có lý do,

phạm vi, phê duyệt, thời hạn và nhật ký kiểm toán. Khi tài nguyên nằm ngoài phạm vi, phần hồi 404 có thể hạn chế tiết lộ sự tồn tại của tài nguyên tốt hơn việc trả chi tiết về quyền còn thiếu.

2.11.2 Lưu trữ sự kiện và khả năng truy vết

Cơ sở dữ liệu quan hệ phù hợp với những thực thể có quan hệ và cần truy vấn như tổ chức, người dùng, kỳ thi, nhiệm vụ được giao và trạng thái phiên mới nhất. JSON Lines phù hợp với chuỗi sự kiện chỉ ghi nối tiếp vì mỗi dòng là một đối tượng JSON độc lập, có thể ghi tuần tự và đọc từng phần.

Mô hình lai cho phép:

- SQL phục vụ bảng điều khiển và kiểm soát quyền.
- JSONL lưu diễn biến chi tiết của tín hiệu, chuyển trạng thái và sự kiện.
- Ảnh chụp cung cấp bằng chứng hình ảnh tại thời điểm cần thiết.
- Báo cáo được tái tạo từ dữ liệu gốc thay vì chỉ lưu một kết luận tổng hợp.

Nhật ký kiểm toán khác với bằng chứng phiên thi. Bằng chứng mô tả điều xảy ra trong phiên; nhật ký kiểm toán mô tả ai đã thao tác trên hệ thống hoặc truy cập dữ liệu. Hai loại dữ liệu cần được tách biệt để hỗ trợ truy vết và trách nhiệm giải trình.

2.12 Các chỉ số đánh giá

Với bài toán nhị phân, confusion matrix gồm:

- True Positive (TP): vi phạm thật được phát hiện.
- False Positive (FP): hệ thống cảnh báo nhưng nhãn tham chiếu là bình thường.
- False Negative (FN): vi phạm thật không được phát hiện.
- True Negative (TN): trạng thái bình thường được nhận biết đúng.

Các chỉ số cơ bản:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

$$Specificity = \frac{TN}{TN + FP},$$

$$FPR = \frac{FP}{TN + FP}$$

Accuracy có thể gây hiểu nhầm khi dữ liệu mất cân bằng vì một hệ thống luôn dự đoán lớp phổ biến vẫn có thể đạt Accuracy cao. Do đó, cần báo cáo đồng thời Precision, Recall, F1 và ma trận nhầm lẫn.

Đơn vị đánh giá cũng phải được công bố. Đánh giá theo khung hình coi mỗi khung hình là một mẫu; đánh giá theo sự kiện so khớp sự kiện với khoảng thời gian có nhãn tham chiếu. Hai cách đánh giá trả lời những câu hỏi khác nhau và cho kết quả khác nhau. Độ trễ phát hiện của một trường hợp dương tính đúng được tính bằng:

$$Latency = t_{detected} - t_{ground\ truth\ start}$$

Đối với ROC-AUC hoặc PR-AUC, cần có score liên tục ở nhiều ngưỡng hoặc một quá trình quét ngưỡng. Một confusion matrix tại một ngưỡng duy nhất không đủ để suy ra hai diện tích này.

CHƯƠNG 3 KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này, tôi khảo sát những hướng tiếp cận đang được sử dụng trong giám sát thi trực tuyến, phân tích một dự án mã nguồn mở làm đối chứng và chuyển các khoảng trống tìm được thành yêu cầu cụ thể cho hệ thống. Phạm vi khảo sát không nhằm khẳng định một sản phẩm là “tốt nhất”, vì mỗi giải pháp phục vụ mô hình thi, chính sách và ngân sách khác nhau. Mục tiêu của chương là xác định các năng lực cần thiết, những giới hạn không thể giải quyết chỉ bằng một mô hình thi giác máy tính và vị trí đóng góp của đồ án.

3.1 Khảo sát các giải pháp thương mại

Các nền tảng thương mại thường cung cấp ba mô hình dịch vụ: giám sát tự động, ghi lại để con người xem sau và giám sát trực tiếp. Chức năng được cấu hình theo từng kỳ thi, có thể gồm xác minh người dự thi, ghi hình webcam, microphone hoặc màn hình, hạn chế thao tác trình duyệt, phát hiện sự kiện và tạo báo cáo.

❖ Proctorio

Theo tài liệu sản phẩm chính thức, Proctorio cung cấp các nhóm thiết lập ghi dữ liệu, xác minh và hạn chế môi trường thi. Tùy cấu hình của đơn vị tổ chức, sản phẩm có thể ghi video, âm thanh, màn hình hoặc lưu lượng web; xác minh danh tính; yêu cầu chế độ toàn màn hình; hạn chế chuyển thẻ, bảng tạm và một số thao tác khác. Giải pháp sử dụng tiện ích mở rộng trên trình duyệt máy tính và chỉ kích hoạt trong phiên thi đã được cấu hình [16], [17].

Điều đáng chú ý đối với đồ án là việc tách chính sách kỳ thi khỏi ứng dụng phía thí sinh. Mỗi kỳ thi có thể yêu cầu những năng lực khác nhau thay vì mọi phiên đều bật toàn bộ quyền. Tuy nhiên, đây là nền tảng đóng; tài liệu công khai không mô tả đầy đủ logic nội bộ để tái hiện thuật toán hoặc hiệu chỉnh theo mục tiêu nghiên cứu của đồ án.

❖ Honorlock

Honorlock cung cấp BrowserGuard để hạn chế hoặc đánh dấu việc truy cập trang web, chuyển thẻ, sử dụng ứng dụng, phím tắt và hành vi thu nhỏ cửa sổ; sản phẩm cũng có thể ghi màn hình trong phiên thi [18]. Mô hình này cho thấy giám sát trình duyệt là một

nguồn tín hiệu riêng, bổ sung cho webcam và không nên bị đồng nhất với kết quả thi giác máy tính.

❖ **ProctorU/Meazure Learning**

Nền tảng ProctorU của Meazure Learning cung cấp các mức dịch vụ từ ghi hình để hậu kiểm đến giám sát trực tiếp có can thiệp [19]. Khác biệt chính so với mô hình hoàn toàn tự động là con người tham gia xác minh, theo dõi và xử lý sự cố. Điều này củng cố nguyên tắc của đề án: thuật toán nên hỗ trợ ưu tiên và giải thích bằng chứng; quyết định cuối cùng thuộc về giám thị hoặc đơn vị tổ chức.

Các sản phẩm thương mại cho thấy một hệ thống hoàn chỉnh cần nhiều hơn một mô hình phát hiện hành vi. Những năng lực thường gặp gồm chính sách theo kỳ thi, kiểm tra thiết bị, kiểm soát trình duyệt, giám sát thời gian thực, hậu kiểm và báo cáo. Việc tái tạo toàn bộ phạm vi của các sản phẩm thương mại không phù hợp với nguồn lực của đề án. Vì vậy, hệ thống đề xuất lựa chọn một tập chức năng có thể kiểm chứng bằng mã nguồn và công bố rõ các giới hạn.

3.2 Khảo sát dự án mã nguồn mở

Đề án sử dụng dự án AarambhTech/exam-cheating-detection làm nguồn tham khảo và phương pháp cơ sở ở mức ý tưởng, không sao chép mã nguồn [20]. Dự án này kết hợp OpenCV, MediaPipe, MTCNN và YOLO để quan sát khuôn mặt, mắt, miệng và vật thể từ webcam.

Ưu điểm của giải pháp tham khảo là dễ tiếp cận, chạy cục bộ và minh họa được cách ghép nhiều thư viện CV. Qua rà soát mã nguồn và so sánh kỹ thuật, đề án nhận thấy các hạn chế sau:

- Một số điều kiện được xử lý bằng chuỗi if-elif, làm mất thông tin khi nhiều bất thường xảy ra đồng thời.
- Các ngưỡng dựa trên số khung hình hoặc khoảng cách điểm ảnh phụ thuộc vào tốc độ khung hình, độ phân giải và khoảng cách đến camera.
- Độ mở miệng chưa được chuẩn hóa đầy đủ; một số điểm mốc cần được xác minh lại theo API MediaPipe thực tế.
- Phần ước lượng hướng nhìn không cho ra yaw/pitch/roll theo mô hình hình học 3D.
- Không có bước tạo véc-tơ đặc trưng khuôn mặt để kiểm tra lại danh tính trong suốt phiên.
- Chưa có lớp nền tảng nhiều tổ chức, phân quyền theo kỳ thi, bảng điều khiển WebSocket và quy trình quản lý bằng chứng, hậu kiểm, báo cáo tương đương hệ thống đề xuất.

3.3 So sánh và phân tích hạn chế

Bảng dưới đây tóm tắt so sánh các giải pháp:

Bảng 2 So sánh định hướng giải pháp

Tiêu chí	Sản phẩm thương mại	Dự án mã nguồn mở đối chứng	Hệ thống của đề án
Minh bạch thuật toán	Hạn chế do nền tảng đóng	Có thể đọc mã nguồn	Mã nguồn và cấu hình được quản lý trong kho mã nguồn
Webcam/CV	Có, tùy sản phẩm và chính sách	Một số tín hiệu đơn lẻ	Bảy tín hiệu và chuỗi xử lý dùng chung
Xử lý theo thời gian	Không công bố đầy đủ	Chủ yếu ngưỡng/bộ đếm đơn giản	Debounce, cửa sổ trượt, state machine hai cấp
Xác thực trong phiên	Có thể có, chi tiết tùy sản phẩm	Không kiểm tra liên tục bằng véc-tơ đặc trưng	Đăng ký khuôn mặt, kiểm tra sự hiện diện sống cơ bản và xác minh lại định kỳ
Toàn vẹn trình duyệt	Thường có	Không phải trọng tâm	Tiện ích mở rộng và điểm toàn vẹn tách khỏi rủi ro CV
Nhiều tổ chức và phân quyền	Có	Không	RBAC, phạm vi tổ chức và nhiệm vụ theo kỳ thi
Giám sát thời gian thực	Có	Chủ yếu hiển thị cục bộ	Bảng điều khiển WebSocket theo kỳ thi
Bảng chứng và hậu kiểm	Có	Báo cáo cơ bản	JSONL, ảnh chụp, hậu kiểm sự cố và tác vụ báo cáo
Khả năng triển khai quy mô lớn	Sản phẩm hoàn chỉnh	Không đặt mục tiêu	Mức đề án/triển khai có kiểm soát; còn giới hạn
Khả năng kiểm thử và thay đổi thuật toán	Không kiểm soát mã nguồn	Có nhưng kiến trúc đơn giản	Giao diện lập trình, cấu hình tập trung và kiểm thử nhiều tầng

Từ bảng so sánh, đề án không đặt mục tiêu cạnh tranh về quy mô với nền tảng thương mại. Giá trị chính nằm ở kiến trúc có thể giải thích, kiểm thử và phục vụ nghiên cứu, đồng thời mở rộng một chương trình webcam đơn máy bằng lớp nền tảng vận hành.

3.4 Xác định tác nhân của hệ thống

Hệ thống có năm nhóm tác nhân chính:

Bảng 3 Các tác nhân hệ thống

Tác nhân	Phạm vi trách nhiệm
System Admin	Quản lý nền tảng, tổ chức, hạn mức, chính sách cơ sở, vận hành và nhật ký toàn cục, không mặc nhiên được xem bằng chứng
Organization Admin	Quản lý hồ sơ, thành viên, lời mời, chính sách và nhật ký kiểm toán của một tổ chức
Exam Manager	Tạo kỳ thi, với nhiệm vụ có thể cấu hình, phân công và vận hành kỳ thi tương ứng
Proctor	Theo dõi bảng điều khiển, xem bằng chứng, hậu kiểm sự cố, kết thúc phiên và xuất báo cáo theo nhiệm vụ được giao
Thí sinh	Xác thực/tham gia đúng kỳ thi, cấp quyền cần thiết và gửi dữ liệu của chính phiên

3.5 Yêu cầu chức năng

Yêu cầu đối với xác thực tài khoản:

- Hệ thống cho phép nhân sự đăng nhập và đăng xuất bằng phiên được bảo vệ.
- System Admin phải hoàn tất MFA trước khi vai trò hệ thống có hiệu lực.
- Hệ thống phân biệt token nhân sự, token phiên thí sinh, device token và WebSocket ticket.
- Thay đổi trạng thái tài khoản, membership hoặc assignment phải làm quyền cũ mất hiệu lực.
- Hệ thống hỗ trợ thí sinh tham gia theo chế độ thủ công hoặc Google OIDC tùy policy kỳ thi.

Yêu cầu đối với quản lý tổ chức và chính sách:

- System Admin quản lý tổ chức, trạng thái, hạn mức phiên đồng thời và thời hạn lưu trữ trong phạm vi được cấp.
- Organization Admin quản lý hồ sơ tổ chức, thành viên, vai trò, lời mời và nhật ký kiểm toán.

- Người dùng thuộc nhiều tổ chức có thể chuyển tổ chức đang hoạt động một cách an toàn.
- Chính sách hiệu lực phải được xác định theo thứ tự chính sách cơ sở của nền tảng → tổ chức → kỳ thi, cấp dưới không được làm yếu yếu cầu bắt buộc.
- System Admin chỉ được đọc bằng chứng khi có quyền ngoại lệ đã được phê duyệt, thuộc đúng tổ chức, còn hiệu lực và chỉ cho phép đọc.

Yêu cầu đối với quản lý kỳ thi:

- Exam Manager tạo kỳ thi và trở thành owner của kỳ thi đó.
- Kỳ thi hỗ trợ vòng đời draft, scheduled, open, closed, archived và chỉ cho phép chuyển trạng thái hợp lệ.
- Mã tham gia phải có thời hạn, có thể xoay và không tự thay đổi lifecycle ngoài ý muốn.
- Người sở hữu hoặc quản lý phân công owner, manager, proctor với trạng thái và thời hạn cụ thể.
- Hành động trên giao diện và máy chủ phải dựa trên năng lực được cấp trong từng kỳ thi.
- Cập nhật kỳ thi phải hỗ trợ khóa lạc quan để phát hiện chỉnh sửa đồng thời.
- Hệ thống cung cấp danh sách kiểm tra mức độ sẵn sàng dựa trên lịch, chính sách, mã tham gia, nhân sự và hạn mức.

Yêu cầu đối với thí sinh tham gia và hoạt động giám sát phiên thi:

- Thí sinh chỉ được tham gia kỳ thi đang mở, bằng mã còn hạn và với chính sách hiệu lực đã được xác định.
- Tiện ích mở rộng kiểm tra phiên bản, sự đồng ý và quyền camera/microphone/chia sẻ màn hình theo chính sách trước khi giám sát.
- Ứng dụng máy tính để bàn xử lý webcam thành đủ bảy SignalResult theo giao diện chung.
- Bước đăng ký thu thập nhiều khung hình, yêu cầu kiểm tra sự hiện diện sống cơ bản nếu được cấu hình và tạo véc-tơ tham chiếu.
- Mỗi tín hiệu phải ổn định dữ liệu theo thời gian trước khi thay đổi trạng thái.
- Bộ tổng hợp rủi ro tính risk_score từ cấu hình, áp dụng vùng trễ cấp phiên và chỉ sinh sự kiện tại cạnh chuyển vào cảnh báo.
- Tiện ích mở rộng ghi nhận trạng thái tập trung/chuyển thể, toàn màn hình, bảng tạm, camera, microphone, chia sẻ màn hình và trạng thái giám sát.

- Máy chủ tự xác định mức nghiêm trọng và điểm toàn vẹn từ sự kiện trình duyệt theo chính sách, không tin điểm do máy khách tự khai báo.
- Hệ thống hỗ trợ heartbeat, trạng thái pending-active-disconnected-ended và lý do mất kết nối.

Yêu cầu đối với bảng điều khiển và báo cáo:

- Dashboard nhận cập nhật phiên theo thời gian thực qua WebSocket trong đúng phạm vi kỳ thi.
- Máy chủ kiểm tra lược đồ, sự hiện diện của đủ bảy tín hiệu, thời gian, rủi ro, trạng thái, mức nghiêm trọng và mức đóng góp trước khi chấp nhận dữ liệu CV.
- Hệ thống lưu siêu dữ liệu, tín hiệu, chuyển trạng thái, diễn biến điểm rủi ro, sự kiện trình duyệt, vi phạm và ảnh chụp theo phiên.
- Ảnh chụp phải được giới hạn định dạng và kích thước, kiểm tra nội dung và lưu bằng tên do máy chủ sinh.
- Giám thị có thể ghi trạng thái và nhận xét hậu kiểm mà không sửa sự kiện máy gốc.
- Hệ thống sinh báo cáo HTML/PDF từ dữ liệu phiên, kể cả khi thiếu một phần nhật ký.
- Tác vụ báo cáo chạy nền và cho biết trạng thái xử lý, lỗi hoặc kết quả.
- Tác vụ lưu trữ hỗ trợ chế độ chạy thử trước khi xóa hoặc ẩn danh dữ liệu hết hạn và phải ghi nhật ký kiểm toán.

3.6 Yêu cầu phi chức năng

Bảng 4 Yêu cầu phi chức năng

Nhóm	Yêu cầu
Bảo mật	Mọi REST, WebSocket và file download phải kiểm tra authentication, tenant và resource scope tại backend.
	Không đặt bearer token dài hạn trong URL WebSocket; ticket browser phải ngắn hạn và dùng một lần.
	Message, ảnh và metadata từ client phải được coi là đầu vào không tin cậy và được kiểm tra chặt.
	Giao diện phải hạn chế XSS, clickjacking, MIME sniffing và rò rỉ referrer bằng CSP/security header phù hợp.
Riêng tư	Không truyền video liên tục; chỉ gửi telemetry theo lô và snapshot tại sự kiện cần bằng chứng.
	Không lưu mã truy cập/mã làm mới của Google; chỉ lưu thuộc tính OIDC tối thiểu và dạng băm của mã xác thực khi có thể.
Tin cậy	Một detector lỗi tạm thời không được làm crash toàn bộ phiên CV.

	Client mất mạng phải được đánh dấu disconnected; desktop CV vẫn có khả năng ghi log/báo cáo cục bộ.
Hiệu năng	Detector nền dùng chung chỉ chạy một lần cho mỗi frame; YOLO được throttle theo thời gian.
	Telemetry mạng được gửi theo lô, mặc định khoảng một giây, không commit database ở mọi frame.
Bảo trì	Ngưỡng và trọng số thuật toán phải tập trung trong cấu hình có kiểm tra hợp lệ.
	Signal Extractor, data schema và authorization phải có interface/quy tắc tập trung và kiểm thử hồi quy.
Giải thích	Mỗi cảnh báo phải truy được risk score, state, tín hiệu đóng góp, thời gian và evidence liên quan.
Triển khai	Backend có thể chạy local development hoặc Docker Compose; dữ liệu bền vững qua volume phù hợp.
Kiểm thử	Chức năng lõi phải có unit/integration test; metric CV phải công bố dataset, ground truth và đơn vị đánh giá.

3.7 Ràng buộc và giả định

Các ràng buộc chính của đề án gồm:

- Các mô hình CV nền đã được huấn luyện trước; nguồn lực của đề án không cho phép huấn luyện toàn bộ mô hình từ đầu.
- Tham số nội tại của camera không được hiệu chuẩn riêng cho mọi thiết bị, vì vậy PnP sử dụng ma trận camera xấp xỉ.
- Ứng dụng CV và tiện ích mở rộng là hai nhánh phía máy khách; cầu nối native messaging để hợp nhất hoàn toàn hai nhánh trong cùng một phiên chưa thuộc phiên bản hiện tại.
- Connection manager WebSocket hoạt động in-process khi không cấu hình Redis; triển khai nhiều worker phải bật Redis lease/pub-sub như cấu hình Docker Compose hoặc dùng message broker tương đương.
- Dữ liệu tổng hợp dùng để kiểm thử bảng điều khiển và báo cáo không thay thế video thật trong đánh giá độ chính xác.
- Bộ 26 video, nhãn tham chiếu và tệp kết quả đánh giá nằm ở môi trường thực nghiệm khác, không có trong kho mã nguồn bản giao hiện tại.
- Kết quả tự động chỉ là chỉ báo phục vụ hậu kiểm, không phải kết luận kỹ luật.

CHƯƠNG 4 THIẾT KẾ HỆ THỐNG

Các chương trước đã trình bày cơ sở lý thuyết của MTCNN, MediaPipe Face Landmarker, YOLOv8, FaceNet, bài toán ước lượng góc quay đầu và nguyên lý kết hợp đa tín hiệu, đồng thời xác định các yêu cầu của hệ thống. Vì vậy, chương này sẽ tập trung vào các vấn đề kỹ thuật đã xuất hiện khi xây dựng một hệ thống giám sát thi hoàn chỉnh và những giải pháp đã được thiết kế, cài đặt để giải quyết các vấn đề đó. Đóng góp của đề án không nằm ở việc huấn luyện một mô hình thị giác máy tính mới, mà ở cách tổ chức các mô hình có sẵn thành chuỗi xử lý có trạng thái, kết hợp kết quả thị giác máy tính với dữ liệu toàn vẹn trình duyệt, và xây dựng lớp nền tảng nhiều tổ chức để truyền trạng thái, quản lý bằng chứng và tạo báo cáo có thể truy vết.

4.1 Mô hình nhận diện với kiến trúc đa tầng

Mô hình nhận diện được thiết kế theo ba tầng sau:

➤ Tầng 1: nhận thức

Mục tiêu của tầng này là trích xuất các đặc trưng cơ bản từ mỗi khung hình video. Tầng này sử dụng các mô hình pretrained hiện có:

- MTCNN [2] phát hiện khuôn mặt và trích xuất 5 điểm mốc sơ bộ.
- MediaPipe FaceMesh [13] trích xuất 468 điểm mốc 3D chi tiết trên khuôn mặt.
- YOLOv8 [21] phát hiện các vật thể cầm như điện thoại, sách.

Đầu ra của tầng này là một tập hợp các đặc trưng cơ bản (vị trí khuôn mặt, các điểm mốc, các vật thể phát hiện được) được truyền tới tầng tiếp theo.

➤ Tầng 2: trích xuất tín hiệu

Tầng này tính toán bảy tín hiệu từ các đặc trưng được trích xuất ở tầng 1:

- Face Presence: có khuôn mặt hay không (Boolean: true/false)
- Multi-face Detection: có nhiều khuôn mặt hay không
- Eye State: trạng thái mắt (mở/nhắm, dựa trên EAR)
- Mouth State: trạng thái miệng (mở/đóng)
- Object Presence: có vật thể cầm hay không
- Head Pose: góc đầu (yaw, pitch, roll, tính bằng solvePnP [15])

- Identity Verification: danh tính có khớp hay không (sử dụng FaceNet [12])

Mỗi tín hiệu có một giá trị hoặc điểm số (score) từ 0 đến 1, đại diện cho mức độ "bất thường" của hành vi. Ví dụ, face presence có giá trị 1.0 nếu không phát hiện khuôn mặt (bất thường), 0.0 nếu có khuôn mặt (bình thường).

➤ Tầng 3: tổng hợp rủi ro

Tầng này tổng hợp bảy tín hiệu trên thành một điểm rủi ro cuối cùng (risk score) từ 0 đến 1, và quyết định có cảnh báo (violation) hay không. Tầng này sử dụng state machine, weighted scoring, và hysteresis (sẽ được trình bày chi tiết ở mục 4.3).

4.2 Chi tiết các tín hiệu nhận diện

❖ Face Presence

Tín hiệu này kiểm tra xem có khuôn mặt được phát hiện trong khung hình hay không. Nếu không phát hiện được khuôn mặt, điểm số là 1.0 (bất thường), nếu phát hiện được, điểm số là 0.0 (bình thường). Tín hiệu này rất quan trọng vì nếu không có khuôn mặt, các tín hiệu khác (như Eye State, Head Pose) không thể được tính toán.

❖ Multi-face Detection

Tín hiệu này phát hiện xem có nhiều hơn một khuôn mặt trong khung hình. Nếu phát hiện nhiều hơn một khuôn mặt, điểm số là 1.0 (bất thường—có thể có người khác trong khung hình), nếu chỉ có một khuôn mặt, điểm số là 0.0 (bình thường). Tín hiệu này giúp phát hiện khi thí sinh có người khác ngồi gần hoặc có ai đó bước vào khung hình.

❖ Eye State

Tín hiệu này tính Eye Aspect Ratio (EAR) [14] dựa trên 6 điểm mốc quanh mỗi mắt từ MediaPipe. Nếu $EAR < 0.1$, mắt được coi là nhắm. Một state machine theo dõi liệu mắt đã nhắm trong bao lâu. Nếu mắt nhắm liên tục > 3 giây, điểm số được tăng dần từ 0.0 lên 1.0, nếu mắt mở lại, điểm số giảm dần. Phương pháp này tránh báo động giả cho những cái nhắm mắt ngắn bình thường.

❖ Mouth State

Tương tự Eye State, tín hiệu này tính MOR (Mouth Open Ratio) dựa trên các điểm mốc quanh miệng. Nếu $MOR > 0.3$, miệng được coi là mở. State machine theo dõi liệu miệng mở trong bao lâu. Nếu miệng mở liên tục > 4 giây, điểm số tăng lên.

❖ Object Presence

Tín hiệu này sử dụng YOLOv8 [21] để phát hiện các vật thể cấm. Nếu phát hiện được các vật thể như "cell phone", "book" trong một vùng gần thí sinh, điểm số là 1.0 (bất thường), nếu không, điểm số là 0.0. Độ tin cậy của phát hiện vật thể (confidence score từ YOLOv8) cũng được xem xét—chỉ xem là bất thường nếu độ tin cậy > 0.7 .

❖ Head Pose

Tín hiệu này tính yaw, pitch, roll bằng solvePnP [15] sử dụng 6 điểm landmark (hai mắt, mũi, hai khoé miệng, cằm) từ MTCNN hoặc MediaPipe. Nếu yaw $> 30^\circ$ (quay trái hoặc phải) hoặc pitch $> 40^\circ$ (quay lên hoặc xuống) liên tục > 2 giây, điểm số tăng lên. State machine theo dõi thời lượng và mức độ quay đầu.

❖ Identity Verification

Tín hiệu này sử dụng FaceNet [12] để trích xuất face embedding từ khuôn mặt hiện tại và so sánh với embedding được lưu từ lúc bắt đầu thi (enrollment). Nếu cosine similarity < 0.6 (ngưỡng được điều chỉnh), điểm số là 1.0 (đổi người). Tín hiệu này rất quan trọng vì là cách duy nhất phát hiện đổi người thi hộ. Để tránh false positive do thay đổi góc quay hoặc ánh sáng, nếu similarity trong khoảng 0.55-0.65, một cảnh báo "nhẹ" được ghi (điểm số 0.5) thay vì báo động ngay.

4.3 Tổng hợp và đánh giá rủi ro

Mỗi tín hiệu được quản lý bởi một state machine riêng. State machine này có hai trạng thái: "normal" (bình thường) và "alert" (cảnh báo). Máy chỉ chuyển sang trạng thái alert khi tín hiệu bất thường trong một khoảng thời gian nhất định (sliding time window), giảm báo động giả cho những sự kiện ngắn.

Tổng điểm rủi ro được tính bằng:

$$\text{risk_score} = w_1 \times \text{signal}_1 + w_2 \times \text{signal}_2 + \dots + w_7 \times \text{signal}_7$$

Trong đó w_i là trọng số của tín hiệu i . Các trọng số được thiết kế dựa trên mức độ nghiêm trọng của hành vi:

- Identity Verification: $w = 0.35$ (rất nghiêm trọng—đổi người)
- Object Presence: $w = 0.25$ (nghiêm trọng—có vật thể cấm)
- Face Presence: $w = 0.20$ (quan trọng—vắng mặt)
- Head Pose: $w = 0.10$ (trung bình—nhìn chuyển tab)
- Mouth State: $w = 0.05$ (ít—có thể nói chuyện)
- Eye State: $w = 0.03$ (ít—có thể mệt)

- Multi-face: $w = 0.02$ (ít—có thể người khác bước qua)

Các trọng số này sẽ được tinh chỉnh dựa trên kết quả thực nghiệm ở Chương 6.

Thay vì sử dụng một ngưỡng duy nhất để quyết định có vi phạm hay không, hệ thống sử dụng hai ngưỡng:

- Ngưỡng "lên": 0.70 — nếu $\text{risk_score} > 0.70$, chuyển sang trạng thái "alert" (có vi phạm)
- Ngưỡng "xuống": 0.50 — nếu $\text{risk_score} < 0.50$, chuyển lại trạng thái "normal" (không vi phạm)

Cơ chế hysteresis này tránh tình huống risk_score dao động quanh một ngưỡng duy nhất, gây báo động giả liên tục.

Luồng xử lý tại mỗi khung hình:

- Tầng Perception tính các đặc trưng
- Tầng Signal Extractors tính 7 tín hiệu và điểm số của mỗi tín hiệu
- Risk Fusion Engine tổng hợp: tính $\text{risk_score} = \sum(w_i \times \text{signal}_i)$
- So sánh risk_score với hai ngưỡng hysteresis
- Nếu có vi phạm, lưu lại thông tin: loại vi phạm, thời điểm, ảnh chụp, risk_score
- Tiếp tục khung hình tiếp theo

4.4 Giải pháp kết hợp đa tín hiệu theo thời gian

Kết quả của một frame đơn lẻ không đủ để kết luận hành vi bất thường. Người dùng có thể chớp mắt, nhìn sang bên trong thời gian ngắn hoặc một detector có thể nhận nhầm vật thể ở đúng một frame. Nếu ghi một vi phạm cho mọi frame vượt ngưỡng, log sẽ bị ngập bởi các sự kiện lặp và giám thị khó phân biệt hành vi kéo dài với nhiều tức thời. Một ngưỡng duy nhất cũng làm trạng thái liên tục bật/tắt khi điểm số dao động gần biên. Ngoài ra, các hành vi không có mức nghiêm trọng như nhau: không khớp danh tính hoặc có điện thoại cần đóng góp nhiều hơn một lần nhắm mắt.

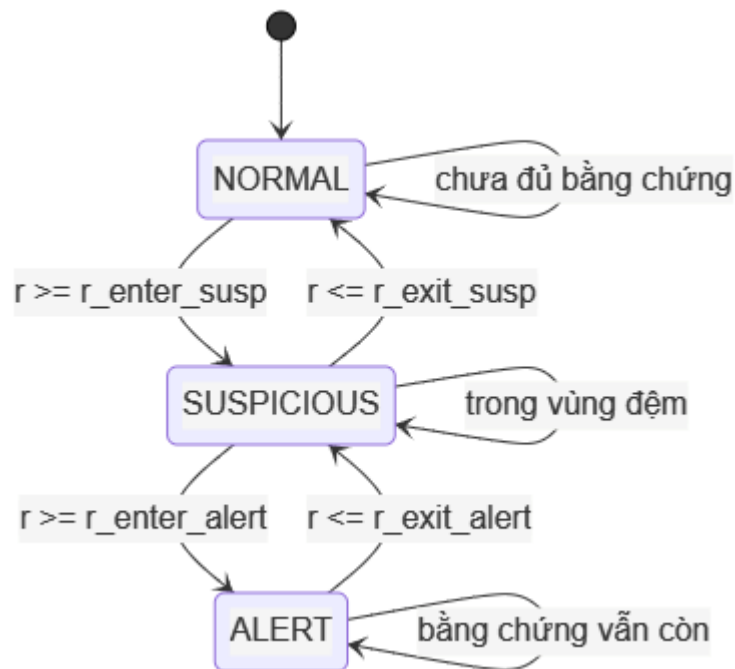
4.4.1 Giải pháp

Đồ án áp dụng hai cấp trạng thái:

- Cấp thứ nhất là một state machine độc lập cho từng tín hiệu. Trong cửa sổ trượt mặc định bốn giây, hệ thống tính:

$$r_i(t) = \frac{N_{i,\text{vượt ngưỡng}}(t - W, t)}{N_{i,\text{hợp lệ}}(t - W, t)}$$

với W là độ dài cửa sổ và $r_i(t)$ là tỉ lệ quan sát bất thường của tín hiệu i . State machine có ba trạng thái “NORMAL”, “SUSPICIOUS” và “ALERT”. Ngưỡng đi vào trạng thái cao hơn lớn hơn ngưỡng đi ra, tạo hysteresis ngay bên trong từng tín hiệu.

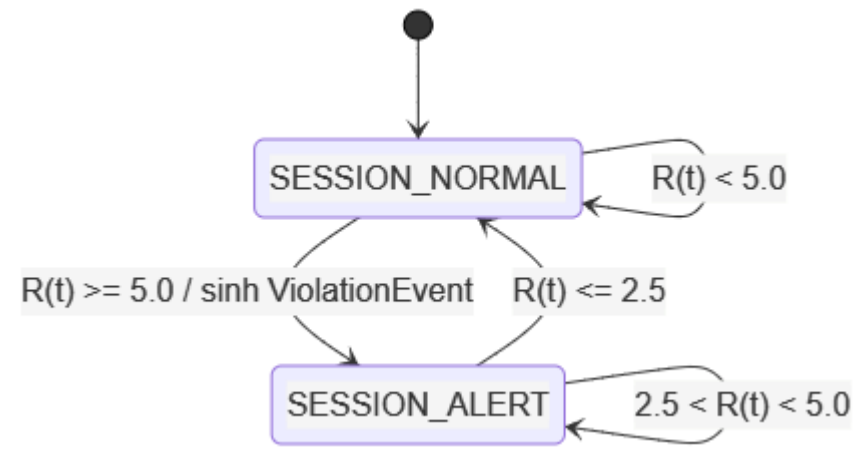


Hình 3 State machine độc lập của một tín hiệu

- Ở cấp thứ hai, các trạng thái được ánh xạ thành “NORMAL = 0”, “SUSPICIOUS = 1”, “ALERT = 2”. Điểm rủi ro phiên được tính bằng:

$R(t) = \sum_{i=1}^7 w_i s_i(t)$ trong đó w_i là trọng số tín hiệu đọc từ tệp cấu hình và $s_i(t)$ là giá trị trạng thái. Cấu hình hiện tại sử dụng trọng số: Identity 3,0; Object Presence 2,5; Face Presence 2,0; Multi-face 2,0; Eye State 1,0; Mouth State 1,0; Head Pose 1,0. Các giá trị này không được hard-code trong thuật toán mà được tập trung tại “config/fusion.yaml” để có thể hiệu chỉnh có kiểm soát.

Trạng thái phiên sử dụng hai ngưỡng: chuyển từ “SESSION_NORMAL” sang “SESSION_ALERT” khi $R(t) \geq 5,0$ và chỉ trở về bình thường khi $R(t) \leq 2,5$. Vùng giữa hai ngưỡng giữ nguyên trạng thái trước đó. Một “ViolationEvent” chỉ được sinh tại cạnh lên, tức thời điểm phiên vừa chuyển sang cảnh báo; hệ thống không tạo lại sự kiện ở mỗi frame trong khi cảnh báo còn kéo dài.



Hình 4 Hysteresis ở cấp phiên thi

Khi tạo sự kiện, engine lưu điểm rủi ro, mức nghiêm trọng, tín hiệu đóng góp, loại vi phạm chính và đường dẫn ảnh chụp. Vi phạm chính là tín hiệu có tích $sw_i \times s_i$ lớn nhất tại thời điểm cạnh lên. Cách chọn này vẫn hoạt động khi cảnh báo được tạo bởi nhiều tín hiệu “SUSPICIOUS” cộng dồn mà chưa có tín hiệu riêng lẻ ở “ALERT”.

4.4.2 Kết quả đạt được

Giải pháp loại bỏ việc sinh cảnh báo lặp theo frame và cho phép nhiều tín hiệu tác động đồng thời đến một quyết định. State machine của bảy tín hiệu được cập nhật độc lập nên một tín hiệu trở lại bình thường không xóa lịch sử của tín hiệu khác. Tất cả transition, timeline điểm rủi ro và sự kiện vi phạm đều có thể được lưu để giải thích quyết định sau kỳ thi.

Các kiểm thử đã bao phủ các tình huống: chưa vượt ngưỡng không sinh sự kiện; cạnh lên sinh đúng một sự kiện; trạng thái cảnh báo kéo dài không tạo bản ghi trùng; phiên có thể trở lại bình thường và cảnh báo lại; nhiều tín hiệu cùng đóng góp; mức nghiêm trọng cao; ảnh snapshot và transition được ghi đúng. Tuy nhiên, trọng số và ngưỡng hiện tại là cấu hình khởi tạo đã được kiểm chứng về logic, chưa phải kết quả tối ưu hóa trên một bộ dữ liệu thực lớn có ground truth.

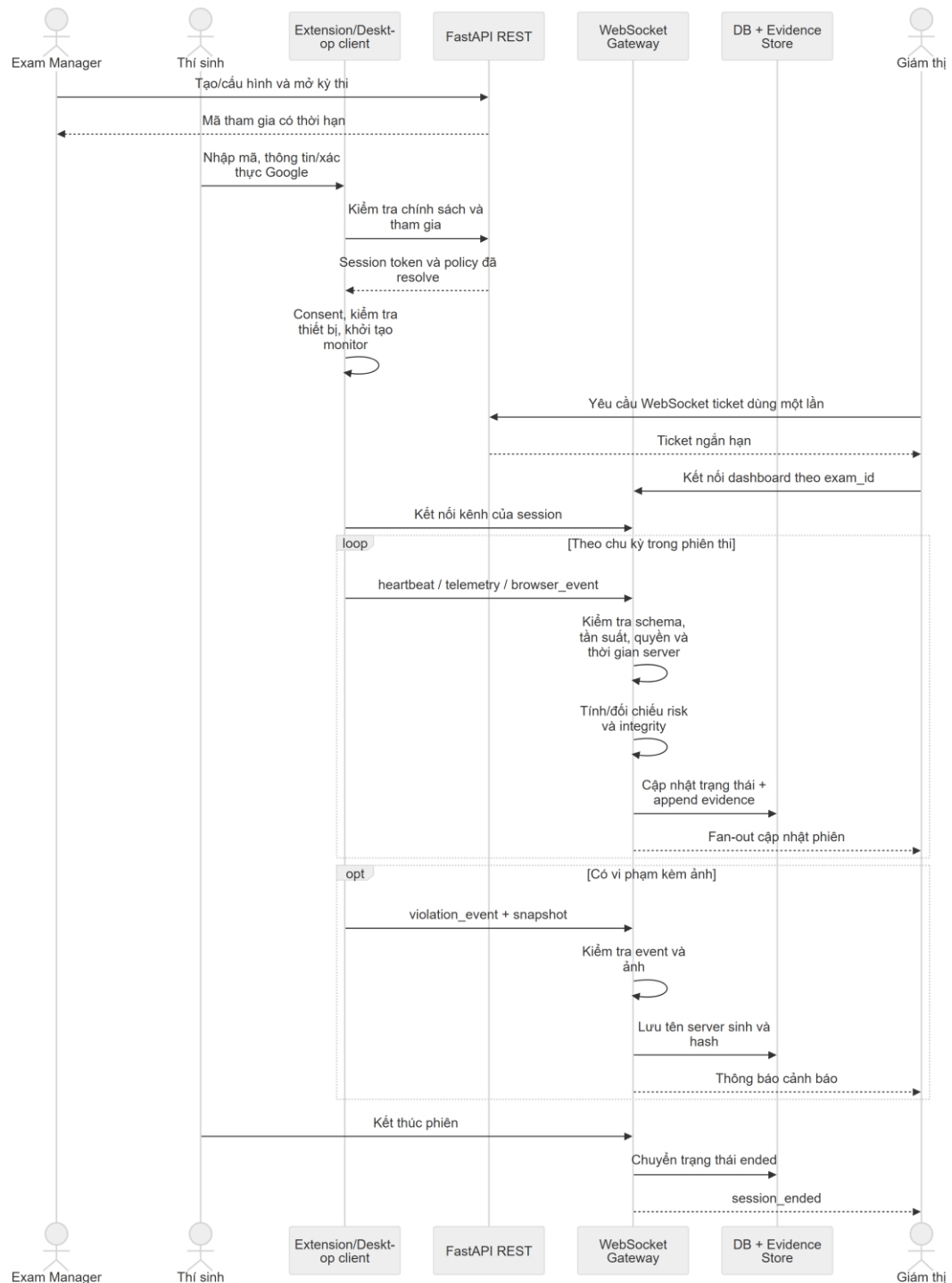
4.5 Giải pháp giám sát thời gian thực và kiểm chứng

Trong lúc sử dụng hệ thống, giám thị cần thấy thay đổi của nhiều phiên gần như tức thời. Polling REST liên tục tạo độ trễ và tải thừa, trong khi WebSocket cho phép cập nhật hai chiều nhưng phát sinh rủi ro khác: token có thể bị lộ trong URL, message có thể bị giả mạo hoặc gửi quá nhanh, timestamp của client có thể sai và kết nối có thể mất mà dashboard không nhận biết.

4.5.1 Giải pháp

Hệ thống sử dụng hai kênh WebSocket tách biệt: kênh client của từng phiên và kênh dashboard theo kỳ thi. Browser WebSocket không truyền JWT trong query string. Dashboard trước tiên gọi REST để nhận một ticket dùng một lần, tồn tại ngắn, sau đó đưa ticket qua subprotocol khi mở kết nối. Client desktop sử dụng token phiên riêng.

Trình tự xử lý một phiên được mô tả trong hình sau:



Hình 5 Sequence diagram của một phiên thi được giám sát

Mọi message được ánh xạ vào schema chặt, giới hạn kích thước và tần suất. Backend dùng timestamp nhận tại server làm thời gian chuẩn; timestamp client chỉ phục vụ truy vết. Với telemetry CV, server yêu cầu đủ bảy signal và đối chiếu lại risk/state/severity/contribution. Với sự kiện trình duyệt, server xác định mức nghiêm trọng theo chính sách kỳ thi và tự tính “integrity_score”, không nhận điểm toàn vẹn do extension tự khai báo.

Heartbeat và idle timeout được dùng để phân biệt các trạng thái “pending”, “active”, “disconnected” và “ended”. Khi nhận cập nhật hợp lệ, backend vừa lưu bản sao trạng thái mới nhất vào SQL để dashboard tải nhanh, vừa append dữ liệu chi tiết vào evidence store. Connection manager fan-out bản cập nhật đến các dashboard đúng kỳ thi.

Ảnh bằng chứng chỉ chấp nhận JPEG hoặc PNG, giới hạn 2 MB, kiểm tra magic bytes, kích thước ảnh và hash. Tên file do server sinh, không sử dụng đường dẫn client gửi. Việc tải ảnh và báo cáo cũng phải đi lại qua kiểm tra quyền của đúng tổ chức và đúng phiên.

4.5.2 Kết quả đạt được

Giám thị có thể theo dõi risk CV, integrity trình duyệt, thiết bị, kết nối và phiên thi theo thời gian thực mà không cần tải lại trang. Khi client mất heartbeat, trạng thái phiên được chuyển thành “disconnected” cùng lý do; khi kết nối lại, hệ thống tiếp tục nhận dữ liệu theo phiên hợp lệ.

Backend không còn là bộ chuyển tiếp mù quáng dữ liệu do client khai báo. Các kiểm thử tích hợp đã kiểm tra ticket một lần, schema sự kiện, tính lại điểm ở server, cô lập WebSocket giữa các tổ chức, upload snapshot và kết thúc phiên. Dù vậy, một client chạy trên thiết bị do thí sinh toàn quyền kiểm soát vẫn có thể bị sửa để phát chuỗi telemetry giả nhưng nhất quán. Giải pháp hiện tại làm tăng khả năng phát hiện sai lệch và truy vết, không thay thế remote attestation hoặc lockdown browser.

4.6 Giải pháp mô hình dữ liệu kết hợp và luồng bằng chứng

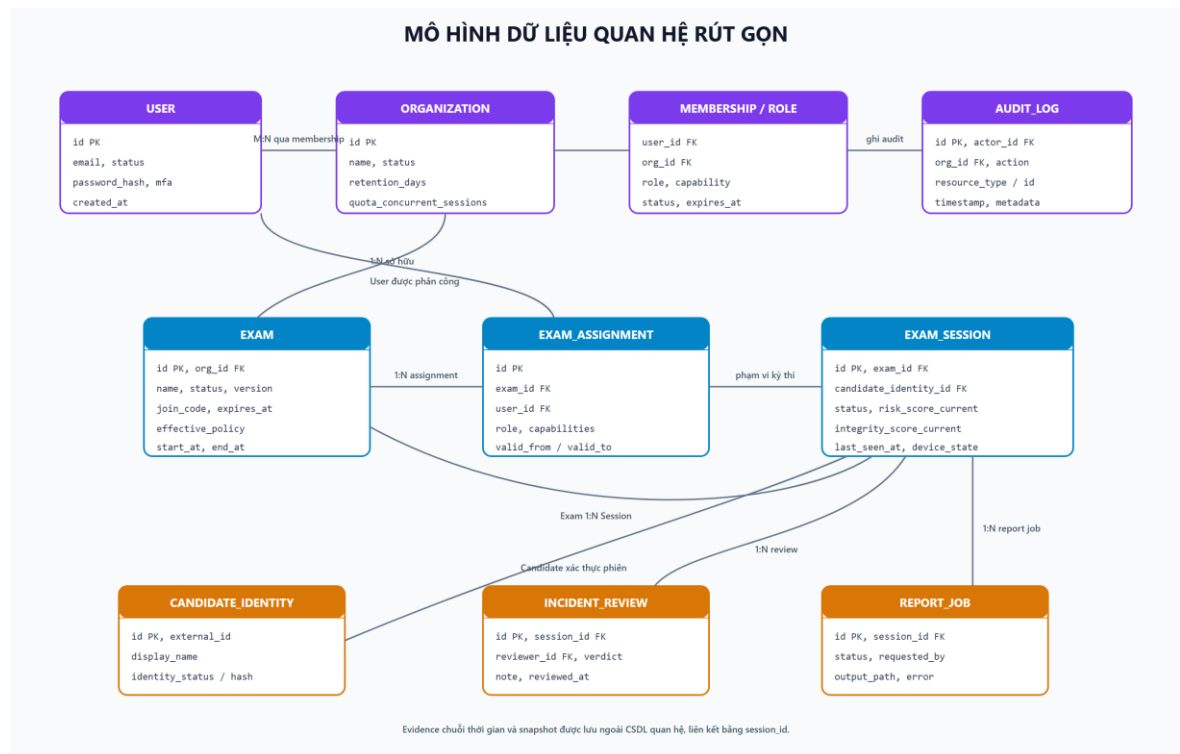
Dashboard cần truy vấn nhanh danh sách kỳ thi, người dùng và trạng thái mới nhất của phiên. Ngược lại, tín hiệu theo thời gian, transition, sự kiện trình duyệt và vi phạm là dữ liệu append-only có số lượng lớn, phù hợp với xử lý tuần tự và sinh báo cáo. Nếu đưa toàn bộ frame-level telemetry vào cơ sở dữ liệu quan hệ, schema trở nên nặng và mỗi frame có thể gây một transaction. Nếu chỉ lưu file, việc lọc theo tổ chức, phân quyền và tải trạng thái ban đầu cho dashboard trở nên khó khăn.

4.6.1 Giải pháp

Đồ án sử dụng mô hình dữ liệu kết hợp:

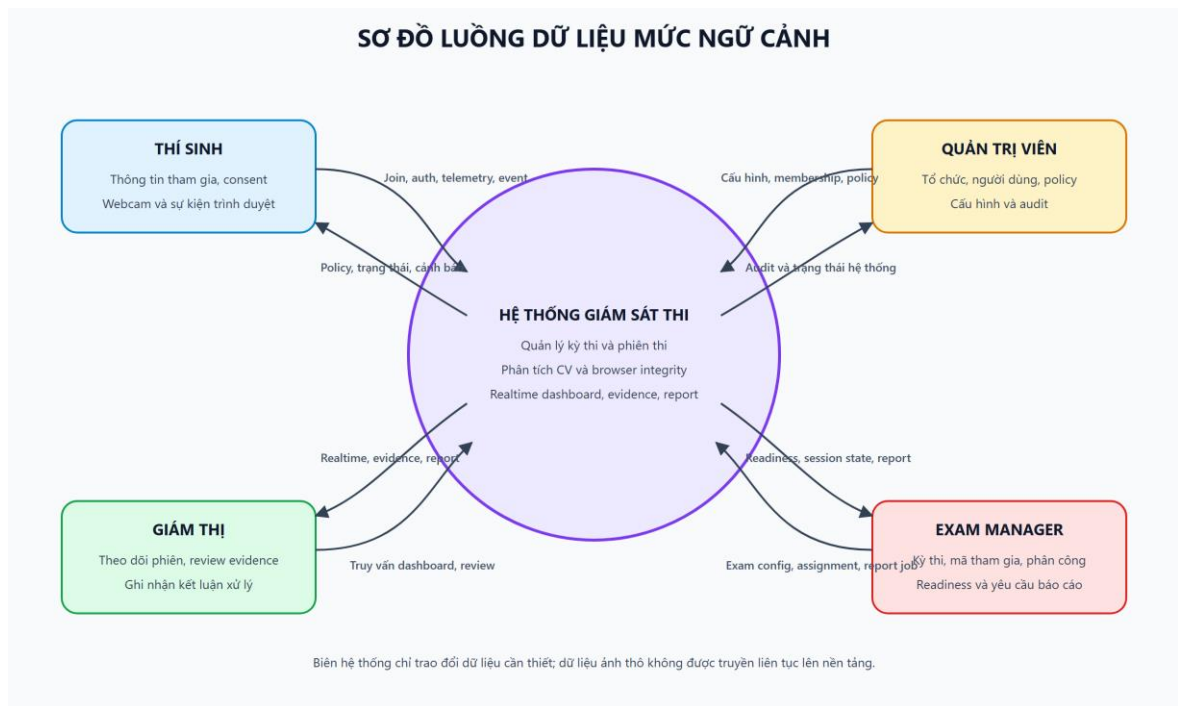
- Cơ sở dữ liệu quan hệ lưu identity, tổ chức, membership, role, kỳ thi, assignment, phiên thi, trạng thái hiện tại, review, report job và audit log.
- Kho evidence theo phiên lưu metadata, tín hiệu, transition, timeline risk, sự kiện trình duyệt, vi phạm và ảnh snapshot dưới dạng JSON/JSONL và tệp ảnh.

Sơ đồ quan hệ chính được trình bày ở Hình 6 ERD chỉ giữ các thực thể và khóa quan trọng để làm rõ phạm vi dữ liệu; các trường kỹ thuật chi tiết tiếp tục được mô tả ở Chương 5.



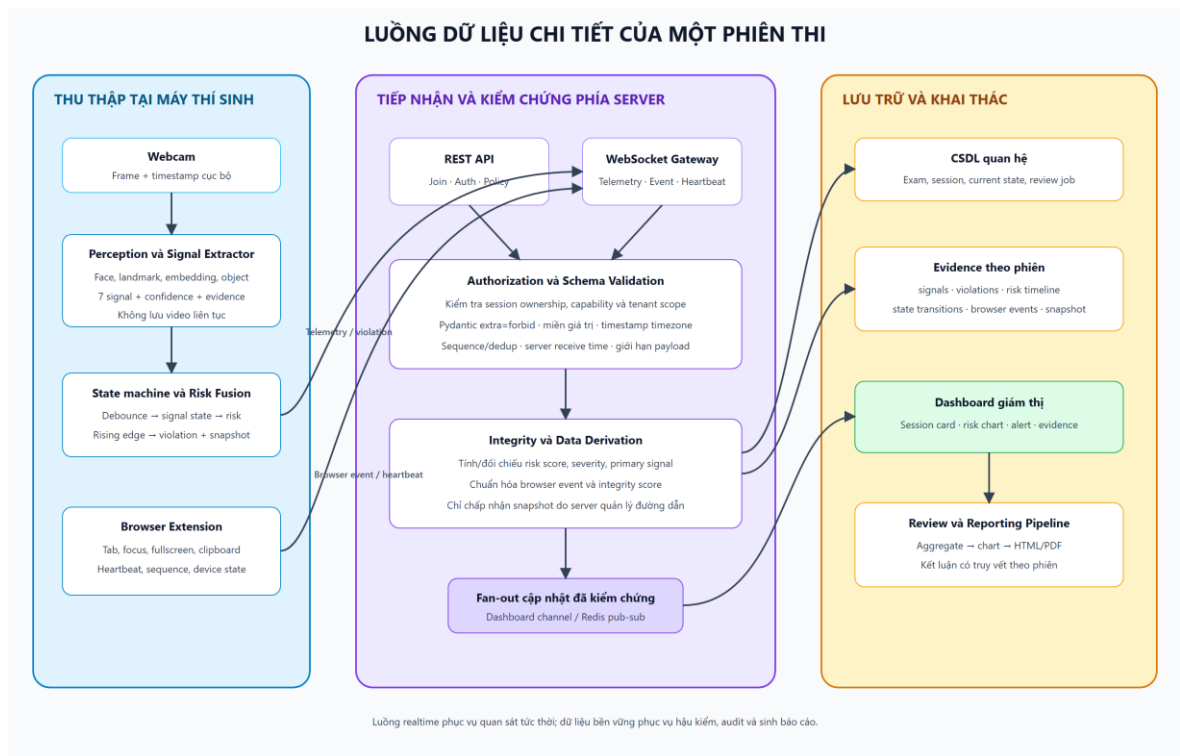
Hình 6 ERD rút gọn của lớp nền tảng

Hình 7 xác định biên hệ thống và bốn nhóm tác nhân ngoài. Thí sinh cung cấp thông tin tham gia, consent và dữ liệu giám sát; giám thị khai thác trạng thái, evidence và ghi kết luận; Exam Manager quản lý vòng đời kỳ thi; còn quản trị viên cung cấp cấu hình tổ chức, người dùng và policy. Sự phân tách này giúp nhận biết dữ liệu nào đi vào, dữ liệu nào đi ra và ai chịu trách nhiệm với từng luồng.



Hình 7 Sơ đồ luồng dữ liệu mức ngũ cảnh của hệ thống

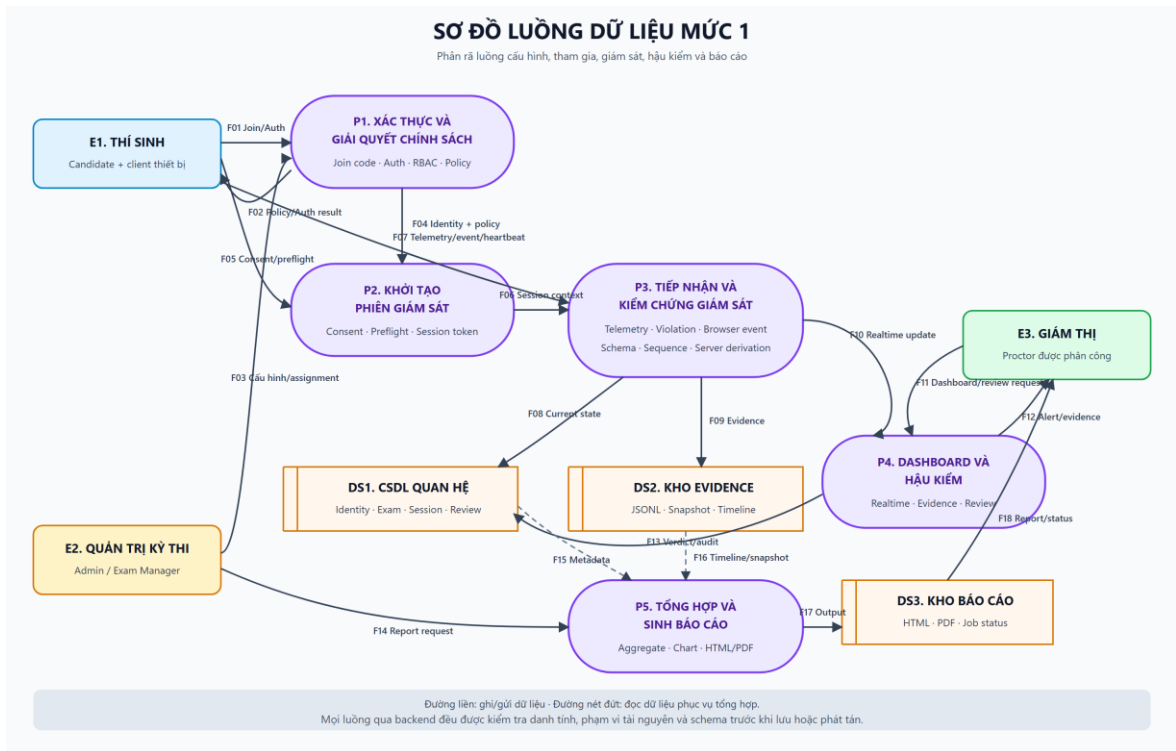
Luồng dữ liệu nội bộ của một phiên thi được thể hiện ở Hình 8. Hai nguồn dữ liệu được xử lý song song: CV telemetry/violation từ desktop client và browser event/heartbeat từ extension. Máy chủ kiểm tra phạm vi phiên, schema, sequence và dữ liệu dẫn xuất trước khi cập nhật trạng thái. Dữ liệu đã kiểm chứng vừa được fan-out tới dashboard, vừa được lưu theo hai lớp: trạng thái hiện tại trong SQL và chuỗi bằng chứng trong JSONL/snapshot.



Hình 8 Sơ đồ luồng dữ liệu chi tiết của một phiên thi

Data flow trên cũng làm rõ ba thời điểm sử dụng dữ liệu. Trong khi thi, dashboard cần current state và alert có độ trễ thấp. Khi hậu kiểm, giám thị cần timeline, snapshot và sự kiện nguyên bản. Khi kết thúc, reporting worker tổng hợp cả metadata quan hệ lẫn evidence append-only để dựng HTML/PDF. Vì vậy, một biểu diễn dữ liệu duy nhất không phù hợp cho cả ba tải truy cập.

Hình 9 phân rã hệ thống thành năm tiến trình nghiệp vụ. “P1” xử lý tham gia, xác thực và chính sách; “P2” tạo ngữ cảnh phiên sau khi ghi nhận sự đồng ý và kiểm tra thiết bị; “P3” tiếp nhận, kiểm chứng dữ liệu giám sát tại biên tin cậy phía máy chủ; “P4” phục vụ bằng điều khiển, bằng chứng và hậu kiểm; “P5” tổng hợp dữ liệu để sinh báo cáo. Ba kho dữ liệu được tách theo tải truy cập: “DS1” lưu siêu dữ liệu và trạng thái hiện tại, “DS2” lưu chuỗi bằng chứng, “DS3” lưu đầu ra báo cáo



Hình 9 Sơ đồ luồng dữ liệu mức 1 của hệ thống

4.6.2 Kết quả đạt được

Mô hình lai đáp ứng đồng thời hai kiểu tải: dashboard lấy trạng thái mới nhất từ SQL mà không phải đọc lại toàn bộ log, trong khi module reporting vẫn đọc được chuỗi thời gian đầy đủ từ thư mục phiên. Schema “SignalResult” và “ViolationEvent” được dùng xuyên suốt từ client, backend đến báo cáo nên không cần chuyển đổi sang một định dạng trung gian khác.

Trình đọc báo cáo có khả năng dung nạp file thiếu hoặc phiên bị dừng giữa chừng; dữ liệu còn lại vẫn được tổng hợp thay vì làm hỏng toàn bộ báo cáo. Hệ thống đã tạo được báo cáo HTML/PDF gồm thống kê, bảng vi phạm, biểu đồ điểm rủi ro, sự kiện integrity và ảnh bằng chứng. Dữ liệu tổng hợp quy mô lớn được dùng để kiểm thử tải và báo cáo, nhưng không được coi là bằng chứng đánh giá độ chính xác của mô hình.

4.7 Giải pháp bảo vệ phiên, chính sách và dữ liệu nhạy cảm

Hệ thống giám sát thi xử lý thông tin định danh, hành vi và hình ảnh của thí sinh. Các rủi ro không chỉ nằm ở việc đăng nhập sai mật khẩu mà còn gồm lộ token qua URL, nhả token người dùng với token phiên, truy cập chéo tổ chức, path traversal khi tải ảnh, replay WebSocket ticket, dữ liệu HTML không an toàn, thay đổi cấu hình kỳ thi đồng thời và chính sách cấp dưới làm yếu yêu cầu bảo mật của cấp trên.

4.7.1 Giải pháp

Đồ án triển khai bảo vệ theo nhiều lớp:

- Xác thực và phiên: mật khẩu được băm; JWT giao diện quản trị nằm trong cookie “HttpOnly”, “SameSite=Strict”; System Admin bắt buộc MFA; thay đổi membership hoặc assignment làm phiên cũ mất hiệu lực thông qua version thu hồi.
- OAuth: luồng Google sử dụng state, PKCE, nonce và grant một lần. Hệ thống chỉ lưu claim tối thiểu và hash của device token.
- Kênh truyền: WebSocket dashboard dùng ticket ngắn hạn dùng một lần, không đặt bearer token trong URL. Message bị giới hạn schema, kích thước, tần suất và có heartbeat.
- Cô lập dữ liệu: mọi truy vấn kỳ thi/phiên đi qua kiểm tra tenant và assignment. Evidence chỉ được phục vụ từ thư mục phiên đã xác định bởi server.
- Bảo vệ giao diện và tệp: dashboard dựng dữ liệu động bằng API DOM an toàn; backend đặt CSP, “X-Content-Type-Options”, “X-Frame-Options”, “Referrer-Policy” và HSTS khi dùng HTTPS. Ảnh upload được kiểm tra nội dung và tên file do server sinh.
- Chính sách phân tầng: policy kỳ thi được resolve theo sản phẩm, chính sách tổ chức và cấu hình kỳ thi. Cấp dưới không thể làm yếu trường bắt buộc của cấp trên.
- Tính nhất quán vận hành: kỳ thi có lifecycle “draft”, “scheduled”, “open”, “closed”, “archived”; join code có hạn và có thể xoay; trường “version” hỗ trợ optimistic locking khi nhiều người cùng chỉnh sửa.

4.7.2 Kết quả đạt được

Các đường REST, WebSocket và download evidence cùng sử dụng một mô hình phạm vi thay vì chỉ ẩn nút ở frontend. Hệ thống đã có kiểm thử cho rate limit đăng nhập, MFA, token confusion, policy inheritance, quota phiên đồng thời, cô lập tổ chức, quyền truy cập theo assignment, snapshot traversal và security header.

Giải pháp làm cho hệ thống phù hợp với mục tiêu nghiên cứu và triển khai có kiểm soát, nhưng chưa thể được xem là một lockdown browser hoàn chỉnh. Extension không quan sát được ứng dụng ngoài trình duyệt, điện thoại thứ hai hoặc máy ảo; Google login chỉ chứng minh quyền sở hữu tài khoản, không chứng minh người ngồi trước camera; blink liveness cơ bản chặn ảnh tĩnh nhưng không thay thế anti-spoofing chuyên dụng trước video replay hoặc deepfake. Với kỳ thi rủi ro cao, cần thêm managed browser, extension/client được ký và force-install, secure boot/attestation, HTTPS bắt buộc, object storage, backup và quy trình bảo vệ dữ liệu chính thức.

4.8 Đánh giá tổng hợp các đóng góp

Một hệ thống có nhiều module dễ tạo cảm giác “nhiều chức năng”, nhưng số lượng màn hình không phản ánh chất lượng kỹ thuật. Đóng góp cần được xem xét theo khả năng giải quyết các mâu thuẫn thực tế: độ nhạy và báo động giả; tính thời gian thực và băng thông; khả năng truy vấn và khả năng lưu chuỗi sự kiện; quyền quản trị và quyền riêng tư; tự động phát hiện và quyền kết luận của con người.

Các đóng góp chính của đề án có thể tổng quát hóa như sau:

- Kiến trúc phân rã theo trách nhiệm: đưa xử lý CV về client, đưa quản trị và kiểm chứng dữ liệu về backend, tách browser integrity khỏi risk CV nhưng hợp nhất hai loại bằng chứng trên dashboard và báo cáo.
- Pipeline nhận thức dùng chung: chạy detector nền tảng một lần cho mỗi frame, chia sẻ “PerceptionResult” cho bầy Signal Extractor và dùng interface thống nhất để kiểm thử độc lập.
- Mô hình quyết định hai cấp có trạng thái: kết hợp cửa sổ thời gian, hysteresis trên từng tín hiệu, weighted fusion và hysteresis cấp phiên; chỉ sinh sự kiện ở cạnh lên để hạn chế log trùng.
- Dữ liệu có khả năng giải thích: lưu tín hiệu đóng góp, chuyển trạng thái, diễn biến điểm rủi ro, sự kiện trình duyệt và ảnh chụp thay vì chỉ lưu một nhãn “gian lận”.
- Nền tảng nhiều tổ chức có phân quyền theo tài nguyên: tách System Admin, Organization Admin, Exam Manager và nhiệm vụ theo kỳ thi; cung cấp quyền truy cập ngoại lệ có phê duyệt đối với dữ liệu nhạy cảm.
- Phòng thủ phía máy chủ: không tin điểm và đường dẫn do máy khách khai báo; kiểm tra lược đồ, dùng thời gian máy chủ, tính lại hoặc đối chiếu điểm rủi ro/toàn vẹn, kiểm tra ảnh và ghi nhật ký kiểm toán.
- Chu trình sau phát hiện: hỗ trợ giám thị xem diễn biến, thực hiện hậu kiểm sự cố và tạo tác vụ báo cáo HTML/PDF, đồng thời giữ kết luận của con người tách khỏi sự kiện máy.

CHƯƠNG 5 CÀI ĐẶT HỆ THỐNG

Trong chương này, tôi mô tả được thực hiện. Hệ thống gồm ba phần có vòng đời độc lập:

1. Ứng dụng Python trên máy tính để bàn chạy chuỗi xử lý thị giác máy tính và có thể hoạt động cục bộ.
2. Tiện ích mở rộng giám sát tính toán vệt trong phạm vi trình duyệt.
3. Dịch vụ FastAPI, bảng điều khiển và tiến trình nền thực hiện quản lý tập trung.

5.1 Công nghệ sử dụng

❖ Thành phần thị giác máy tính trên máy tính để bàn

Các phiên bản được giới hạn trong requirements.txt nhằm tránh xung đột giữa OpenCV, MediaPipe và NumPy. Mô hình Face Landmarker và YOLO được đặt trong thư mục models/; mô hình có thể được tải khi khởi tạo nếu chưa có.

❖ Nền tảng web

Vùng chứa phía máy chủ sử dụng requirements-backend.txt và không cài các mô hình thị giác máy tính có dung lượng lớn. Các mô-đun dùng chung chỉ được nạp khi cần, nhờ đó thành phần báo cáo có thể hoạt động mà không khởi tạo PyTorch, MediaPipe hoặc Ultralytics.

❖ Tiện ích mở rộng trình duyệt

Tiện ích mở rộng sử dụng JavaScript theo WebExtensions API và Manifest V3. Kịch bản đóng gói tạo hai bản dist/chrome và dist/firefox; Chrome sử dụng tiến trình dịch vụ (service worker), còn Firefox sử dụng kịch bản nền và khai báo riêng cho Gecko. Việc không sử dụng khung giao diện phía máy khách giúp giảm kích thước gói và số thư viện phụ thuộc trong thành phần được coi là không hoàn toàn tin cậy.

5.2 Cấu trúc kho mã nguồn

Cấu trúc rút gọn của mã nguồn thực tế như sau:

exam-proctor-extension-upgrade/

```

└─ main.py
└─ config/
  └─ fusion.yaml
└─ src/
  └─ app_config.py
  └─ app_controller.py
  └─ orchestrator.py
  └─ client/
    └─ backend_client.py
  └─ perception/
    └─ pipeline.py
    └─ perception_result.py
    └─ face_detector.py
    └─ face_mesh_detector.py
    └─ object_detector.py
    └─ face_embedder.py
    └─ head_pose_math.py
  └─ signals/
    └─ base.py
    └─ factory.py
    └─ face_presence.py, multi_face.py
    └─ eye_state.py, mouth_state.py
    └─ object_signal.py, head_pose.py
    └─ identity.py, liveness.py
  └─ fusion/

```


- | | | └─ signal_state_machine.py
- | | | └─ tracker.py, session.py, engine.py
- | | | └─ violation_event.py
- | | └─ _logger.py
- | └─ reporting/
- | | └─ data_loader.py, aggregator.py
- | | └─ charts.py, report_generator.py
- | | └─ templates/report_template.html
- | └─ ui/
- └─ extension/
- | └─ manifest.base.json
- | └─ src/
- | | └─ setup.html, setup.js
- | | └─ background.js
- | | └─ content-monitor.js
- | | └─ common.js, styles.css
- | └─ scripts/build.mjs
- └─ backend/
- | └─ main.py, models.py, db.py
- | └─ auth.py, authorization.py, policies.py
- | └─ ws_schemas.py, ws_manager.py, ws_tickets.py
- | └─ session_materializer.py
- | └─ routers/
- | └─ templates/
- | └─ static/

└── scripts/
└── tests/ và backend/tests/
└── Dockerfile.backend
└── docker-compose.yml

Các gói mã nguồn được chia theo trách nhiệm thay vì theo từng màn hình. `main.py` chỉ khởi tạo các thành phần phụ thuộc và chạy vòng lặp camera; logic vòng đời phiên nằm trong `AppController`. Các bộ định tuyến phía máy chủ không cài đặt lại quy tắc phân quyền mà sử dụng tập trung `authorization.py`.

5.3 Cấu hình tập trung

Tập `config/fusion.yaml` là nguồn cấu hình trung tâm cho ứng dụng thị giác máy tính. Tập này gồm:

- Độ dài cửa sổ và bốn ngưỡng máy trạng thái của từng tín hiệu.
- Trọng số bảy tín hiệu và ngưỡng `T_enter/T_exit` cấp phiên.
- Tham số đặc thù như EAR, góc quay ngang/dọc, chu kỳ xác minh danh tính và thời lượng chống dao động.
- Camera, thư mục phiên, đăng ký khuôn mặt, kiểm tra sự hiện diện sống, định dạng báo cáo và kết nối máy chủ.

`AppConfig.from_yaml()` đọc phần cấu hình cấp ứng dụng. `build_signals_from_config()` tạo đúng bảy bộ trích xuất tín hiệu. `RiskFusionEngine.from_config()` tạo bộ theo dõi trạng thái, trọng số và vùng trễ từ cùng một tập. Cách tổ chức này loại bỏ lỗi của phiên bản cũ, trong đó ngưỡng danh tính được viết cố định trong mã và khác với YAML.

Các giá trị chính đang sử dụng được trình bày ở Chương 4, Bảng 4.1. Mọi tham số được kiểm tra miền hợp lệ; ví dụ chu kỳ gửi dữ liệu giám sát phải dương, định dạng báo cáo chỉ nhận `html/pdf`, và `T_exit` phải nhỏ hơn `T_enter`.

Chính sách của nền tảng web không dùng trực tiếp `fusion.yaml`. Tập `backend/policies.py` định nghĩa `PlatformPolicy`, `OrganizationPolicy` và `ResolvedExamPolicy`; máy chủ xác định mức sàn hệ thống, chính sách tổ chức và giá trị ghi đề của kỳ thi tại thời điểm tạo hoặc cập nhật.

5.4 Cài đặt ứng dụng thị giác máy tính trên máy tính để bàn

5.4.1 Vòng đời ứng dụng

main.py thực hiện bốn việc: đọc cấu hình, dựng bày signal, dựng PerceptionLayer, tạo AppController và chạy vòng lặp webcam. Vòng đời phiên được biểu diễn bởi AppState:

IDLE → ENROLLMENT → MONITORING → GENERATING_REPORT → ENDED

AppController.step(frame) là điểm xử lý duy nhất cho mỗi khung hình. Chuột và bàn phím được đưa qua handle_mouse() và handle_key(). Cấu trúc này thay cho nhiều vòng while tách biệt, giúp việc kết thúc phiên, đóng nhật ký và kiểm thử chuyển trạng thái nhất quán hơn.

Khi backend.enabled=false, người dùng có thể chạy hoàn toàn cục bộ. Khi bật kết nối máy chủ, màn hình IDLE yêu cầu tên và mã tham gia, gọi REST để nhận mã xác thực phiên rồi khởi tạo BackendClient. Việc mất kết nối máy chủ không làm chuỗi xử lý cục bộ dừng; dữ liệu được gửi theo khả năng tốt nhất của kết nối.

5.4.2 Tầng nhận thức

PerceptionLayer.process(raw_frame_bgr) thực hiện:

1. Resize giữ tỉ lệ và chuyển BGR sang RGB.
2. Chạy MTCNN để lấy danh sách FaceBox.
3. Chạy MediaPipe Face Landmarker cho khuôn mặt chính.
4. Chạy YOLO theo chu kỳ object_detect_interval_sec, mặc định 0,4 giây.
5. Đóng gói kết quả vào PerceptionResult.

YOLO không trả kết quả rỗng ở khung hình bị giới hạn tần suất mà dùng _last_objects. Mỗi bộ phát hiện được gọi qua _safe_call(): khi MTCNN/Face Landmarker gặp lỗi thì dùng kết quả thiếu an toàn; khi YOLO gặp lỗi thì giữ kết quả gần nhất. Thời gian từng bước được lưu trong last_timings để đo hiệu năng.

PerceptionResult chứa dấu thời gian, kích thước khung hình, khung bao khuôn mặt, điểm mốc khuôn mặt, khung bao vật thể và khung hình RGB. Bộ trích xuất tín hiệu không gọi lại bộ phát hiện, nhờ vậy kết quả của một khung hình có cùng mốc thời gian.

5.4.3 Đồng bộ trích xuất tín hiệu

Tập src/signals/base.py định nghĩa abstract SignalExtractor và dataclass SignalResult. Mỗi kết quả gồm: signal_name, timestamp, value, exceeds_threshold, confidence, metadata

Các cài đặt cụ thể:

- FacePresenceSignal dùng timer vắng mặt.
- MultiFaceSignal lọc khung bao khuôn mặt theo độ tin cậy trước khi đếm.
- EyeStateSignal tính EAR từng mắt bằng tọa độ pixel và yêu cầu cả hai mắt cùng nhắm.
- MouthStateSignal tính tỉ lệ mở miệng và activity ratio trong cửa sổ.
- ObjectSignal đọc box đã lọc cell phone/book và debounce thời lượng.
- HeadPoseSignal gọi solve_head_pose() rồi theo dõi góc quay ngang/dọc vượt ngưỡng.
- IdentitySignal đăng ký nhiều vector đặc trưng, xác minh lại định kỳ, có vùng cảnh báo sớm và yêu cầu lỗi liên tiếp.

Khi thiếu điểm mốc, tín hiệu phụ thuộc vào điểm mốc trả độ tin cậy bằng 0 và không tự kết luận bất thường; tín hiệu hiện diện khuôn mặt xử lý nguyên nhân vắng mặt. Cách phân trách nhiệm tránh việc cùng một lỗi bộ phát hiện làm nhiều tín hiệu đồng loạt báo vi phạm.

5.4.4 Đăng ký khuôn mặt và kiểm tra

Trong trạng thái ENROLLMENT, BlinkLivenessChallenge theo dõi chuỗi mở–nhắm–mở mắt. Chỉ sau khi thử thách đạt yêu cầu, bộ điều khiển mới thu khung hình để tạo vector đặc trưng tham chiếu. Số khung hình, số lần thử và thời hạn chờ được lấy từ cấu hình.

IdentitySignal.enroll() bỏ các khung hình không trích được khuôn mặt và lấy trung bình các vector đặc trưng hợp lệ. Nếu không có khung hình hợp lệ sau số lần thử, phiên được hủy với lý do đăng ký thất bại. Cơ chế xác minh sự hiện diện sống này là biện pháp cơ bản, không được cài đặt như một giải pháp chống phát lại chuyên dụng.

5.4.5 Bộ điều phối và tổng hợp rủi ro

PipelineOrchestrator gọi tầng nhận thức, lần lượt chạy toàn bộ tín hiệu và cập nhật máy trạng thái. Bộ điều phối còn ghi signals.jsonl, thống kê thời gian xử lý và đặt lại mọi tín hiệu khi bắt đầu phiên mới.

SignalStateTracker chứa một SignalStateMachine riêng cho từng tín hiệu. RiskFusionEngine.update() lưu kết quả mới nhất, cập nhật tracker, tính:

$$R = \sum_i w_i \text{stateValue}_i$$

sau đó gọi SessionHysteresis. Nếu lần chuyển trạng thái đi vào SESSION_ALERT, bộ tổng hợp tạo ViolationEvent, chọn vi phạm chính, lưu các tín hiệu đóng góp và ảnh chụp. Trong khi phiên vẫn ở trạng thái ALERT, lời gọi tiếp theo không tạo sự kiện trùng.

StateTransitionLogger và RiskScoreLogger ghi riêng lần chuyển trạng thái và dòng thời gian. Việc tách bộ ghi nhật ký cho phép mô-đun báo cáo đọc lại trạng thái mà không phải chạy lại mô hình.

5.4.6 Gửi dữ liệu đến dịch vụ phía máy chủ

BackendClient mở WebSocket nền và có hàng đợi gửi. Dữ liệu giám sát được gom theo `telemetry_interval_sec` thay vì gửi ở mọi khung hình. Sự kiện vi phạm được tuần tự hóa cùng các tín hiệu đóng góp; nếu có ảnh chụp, máy khách đọc chuỗi byte và gửi nội dung/giá trị băm thay vì gửi đường dẫn hệ thống tệp cục bộ.

Khi kết nối không thành công, hàm gửi không thực hiện thao tác nhưng vẫn được kiểm soát, còn phiên tiếp tục ghi tệp cục bộ. Điểm giới hạn là dữ liệu phát sinh trong thời gian ngoại tuyến chưa có cơ chế đồng bộ bù đầy đủ khi kết nối trở lại.

5.5 Cài đặt tiện ích mở rộng trình duyệt

5.5.1 Tệp khai báo và quyền

`manifest.base.json` khai báo các quyền `alarms`, `identity`, `scripting`, `storage`, `tabs` và quyền truy cập máy chủ ở dạng tùy chọn. Tiện ích chỉ yêu cầu nguồn của máy chủ/trang thi khi cần. Chính sách bảo mật nội dung của tiện ích giới hạn kịch bản và kiểu trình bày do chính tiện ích cung cấp, đồng thời chỉ cho phép kết nối HTTPS/WSS hoặc localhost phục vụ phát triển.

5.5.2 Luồng thiết lập

`setup.html/setup.js` cung cấp giao diện:

1. Nhập địa chỉ máy chủ và mã tham gia.
2. Lấy chính sách kỳ thi.
3. Chọn luồng thủ công hoặc Google.
4. Yêu cầu quyền truy cập đúng nguồn máy chủ.
5. Hiển thị nội dung xin sự đồng ý và kiểm tra điều kiện.
6. Gửi lệnh bắt đầu đến tiến trình nền.

Đăng nhập Google sử dụng `identity.launchWebAuthFlow()`. Máy chủ quản lý tham số state, PKCE, giá trị dùng một lần nonce và mã cấp quyền chỉ dùng một lần; tiện ích lưu mã xác thực thiết bị thí sinh có nội dung không thể suy đoán do hệ thống cấp, không lưu mã xác thực Google.

5.5.3 Tiến trình nền

background.js giữ trạng thái phiên trong storage.session, quản lý mã thiết bị, mở thẻ bài thi, chèn thành phần giám sát nội dung và kết nối WebSocket. Thành phần này theo dõi:

- Thẻ đang hoạt động, thẻ mới, thao tác điều hướng và đóng thẻ.
- Trạng thái tập trung của cửa sổ.
- Cảnh báo tình trạng và kết nối lại.
- Số thứ tự của sự kiện trình duyệt và hàng đợi chưa gửi.

WebSocket được mở bằng vé kết nối của phiên thay vì đặt mã xác thực mang quyền truy cập trong chuỗi truy vấn. Nhịp kết nối và hàng đợi sự kiện giúp máy chủ biết trạng thái kết nối. Các sự kiện quan trọng được ưu tiên giữ trong hàng đợi; khi WebSocket sẵn sàng, tiến trình nền gửi lần lượt theo đúng thứ tự.

5.5.4 Thành phần giám sát nội dung

content-monitor.js chạy trong nguồn của trang thi. Mô-đun yêu cầu camera/micro và getDisplayMedia() theo chính sách, hiển thị hình ảnh xem trước/trạng thái, theo dõi luồng phương tiện bị tắt tiếng hoặc kết thúc và tạo dải thông báo yêu cầu quay lại toàn màn hình. Thành phần này ghi các sự kiện bảng tạm, trình đơn ngữ cảnh, khả năng hiển thị/trạng thái tập trung và quyền còn thiếu.

Tiện ích chủ yếu quan sát và cảnh báo trong phạm vi API trình duyệt. Tiện ích không thể bảo đảm chặn ứng dụng cục bộ, máy ảo hoặc thiết bị thứ hai.

5.6 Cài đặt dịch vụ phía máy chủ và bảng điều khiển

5.6.1 Khởi tạo FastAPI

backend/main.py tạo ứng dụng FastAPI, gắn các tệp tĩnh và đăng ký các bộ định tuyến:

- auth: đăng ký, đăng nhập, MFA, invitation và tài khoản.
- candidate_auth: xác thực thủ công/Google cho thí sinh.
- exams: tạo, cập nhật, vòng đời, phân công, điều kiện sẵn sàng và tham gia.
- organizations: hồ sơ, thành viên, chính sách, quyền ngoại lệ và nhật ký kiểm toán.
- sessions: danh sách/chi tiết phiên, kết thúc/đặt lại, hậu kiểm sự cố, bằng chứng và tác vụ báo cáo.
- system: bảng điều khiển hệ thống, tổ chức, chính sách, bảo mật và quyền truy cập ngoại lệ.
- ws: kênh máy khách và bảng điều khiển.
- pages: các trang Jinja2.

Phần mềm trung gian sinh X-Request-ID, áp dụng tiêu đề bảo mật, điều khiển HTTPS và Cache-Control. CORS cho tiện ích chỉ bật với danh sách nguồn đã cấu hình.

5.6.2 Mô hình dữ liệu

backend/models.py dùng ánh xạ có kiểu của SQLAlchemy 2.0. Các thực thể được chia thành:

- Danh tính/quyền: User, SystemRole, OrganizationMembership, ExamAssignment, Invitation, các bảng thử thách/OAuth.
- Tổ chức/chính sách: Organization, PlatformPolicySetting, AccessGrant, AuditLog.
- Kỳ thi/phiên: Exam, ExamSession, CandidateIdentity, CandidateDevice.
- Sau giám sát: IncidentReview, ReportJob.

Ràng buộc duy nhất ngăn một mã thí sinh hoặc danh tính thí sinh tạo nhiều phiên trong cùng kỳ thi. ExamSession lưu mức rủi ro/toàn vẹn và trạng thái thiết bị mới nhất để bảng điều khiển không phải quét JSONL.

5.6.3 Xác thực và phân quyền

backend/auth.py phân biệt mã xác thực người dùng với mã xác thực phiên bằng thuộc tính loại riêng. Bảng điều khiển web sử dụng cookie HttpOnly, SameSite=Strict. Mật khẩu được băm bằng bcrypt; quản trị viên hệ thống phải hoàn tất MFA.

authorization.py định nghĩa kiểu liệt kê Permission và quan hệ ánh xạ:

- Vai trò tổ chức → quyền chức năng của tổ chức.
- Phân công owner/manager/proctor → quyền chức năng của kỳ thi.
- Vai trò hệ thống → quyền chức năng của nền tảng.

authorize_exam() và authorize_session() kiểm tra phạm vi tổ chức/tài nguyên. Quản trị viên hệ thống không được tự động nhận quyền của quản trị viên tổ chức; quyền đọc bằng chứng chỉ được thêm khi có quyền truy cập khẩn cấp đang hoạt động và thuộc đúng tổ chức. capabilities_for_user() phục vụ điều hướng tổng quát, còn thao tác trên từng kỳ thi dùng exam_access_for_user() để tránh hợp nhất nhầm quyền.

Thay đổi tư cách thành viên/vai trò làm tăng session_version hoặc khiến bản ghi không còn hoạt động, do đó mã xác thực cũ bị từ chối ở lần kiểm tra tiếp theo. WebSocket của bảng điều khiển định kỳ kiểm tra lại quyền thay vì chỉ xác thực lúc mở kết nối.

5.6.4 Xác định chính sách hiệu lực

backend/policies.py kiểm tra phiên bản ngữ nghĩa và xác định chính sách theo ba cấp. Giá trị luận lý bắt buộc dùng phép OR với mức sản cấp trên; phiên bản tối thiểu chọn mức nghiêm ngặt hơn; thời gian mất tập trung chọn mức nhỏ hơn. Máy chủ từ chối giá trị ghi đè làm yếu chính sách, không phụ thuộc vào bước kiểm tra dữ liệu ở giao diện.

5.6.5 WebSocket và kiểm tra dữ liệu

ws_schemas.py sử dụng Pydantic với extra="forbid". TelemetryUpdateData yêu cầu đúng bảy tín hiệu và bảy trạng thái không trùng. Số phải hữu hạn và nằm trong miền. ViolationEventData yêu cầu danh sách tín hiệu đóng góp không rỗng, dấu thời gian có múi giờ và không chấp nhận snapshot_path do máy khách điều khiển.

routers/ws.py xử lý thông điệp mở đầu của máy khách, nhịp kết nối, dữ liệu giám sát, vi phạm, sự kiện trình duyệt và kết thúc phiên. Máy chủ dùng thời gian tiếp nhận, kiểm tra số thứ tự, loại bỏ bản ghi trùng và tính hoặc đối chiếu lại:

- Điểm rủi ro từ trạng thái tín hiệu và trọng số.
- Trạng thái phiên và mức nghiêm trọng.
- Tín hiệu chính và các tín hiệu đóng góp.
- Mức nghiêm trọng của sự kiện trình duyệt, điểm toàn vẹn và trạng thái thiết bị.

ConnectionManager giữ kết nối WebSocket của máy khách/bảng điều khiển. Khi REDIS_URL được cấu hình, thời hạn chiếm giữ trên Redis ngăn hai máy khách cùng chiếm một phiên và cơ chế xuất bản/đăng ký của Redis chuyển cập nhật bảng điều khiển giữa các tiến trình. Khi không có Redis, bộ quản lý hoạt động trong cùng tiến trình cho môi trường phát triển dùng một tiến trình nền.

Bảng điều khiển trên trình duyệt lấy vé kết nối dùng một lần từ REST trước khi mở WebSocket. WebSocketTicketStore đặt thời hạn ngắn và đánh dấu vé đã dùng, giúp giảm rủi ro phát lại.

5.6.6 Giao diện web

Giao diện dùng mẫu Jinja2 cho khung trang và JavaScript gọi API. Các trang chính gồm:

- Tổng quan hệ thống, chi tiết tổ chức, chính sách, bảo mật và nhật ký kiểm toán.
- Tổng quan tổ chức, thành viên và thiết lập.
- Không gian làm việc của kỳ thi, danh sách kỳ thi, quản lý/điều kiện sẵn sàng/bảng điều khiển.
- Chi tiết phiên, biểu đồ rủi ro, bằng chứng, hậu kiểm và báo cáo.
- Account, login, MFA và registration.

Dữ liệu từ máy chủ được đưa vào DOM bằng `textContent` hoặc hàm hỗ trợ an toàn. Quyền chức năng quyết định trình đơn và thao tác được hiển thị, nhưng máy chủ vẫn là điểm thực thi kiểm soát cuối cùng.

5.7 Giao diện và luồng vận hành của hệ thống đã triển khai

5.7.1 Nguyên tắc tổ chức giao diện

Hệ thống đã triển khai giao diện trên ba môi trường: bảng điều khiển web cho nhân sự vận hành, tiện ích mở rộng trình duyệt cho thí sinh và cửa sổ OpenCV của ứng dụng thị giác máy tính trên máy tính để bàn. Mỗi môi trường được thiết kế theo nhiệm vụ của người sử dụng thay vì cố gắng gom tất cả chức năng vào một màn hình. Bảng điều khiển web ưu tiên quản trị, quan sát nhiều phiên và hậu kiểm; tiện ích ưu tiên hướng dẫn từng bước, ghi nhận sự đồng ý và trạng thái toàn vẹn của trình duyệt; ứng dụng máy tính để bàn ưu tiên đăng ký khuôn mặt, xác minh sự hiện diện sống và phản hồi trực tiếp từ chuỗi xử lý thị giác máy tính.

Giao diện web sử dụng khung Jinja2 thống nhất, tải dữ liệu động qua REST và nhận cập nhật theo thời gian thực qua WebSocket. Trình đơn, nút thao tác và thẻ được ẩn hoặc hiện theo quyền chức năng để giảm nhầm lẫn thao tác. Đây chỉ là lớp hỗ trợ trải nghiệm: mọi yêu cầu vẫn phải vượt qua bước kiểm tra quyền tại máy chủ. Các trạng thái tải, rỗng, lỗi và mất kết nối được thể hiện riêng để người vận hành không hiểu nhầm dữ liệu cũ là dữ liệu đang cập nhật.

Danh mục giao diện được đối chiếu trực tiếp với đường dẫn/mẫu giao diện và thành phần máy khách trong mã nguồn:

Bảng 5 Nguyên tắc tổ chức giao diện

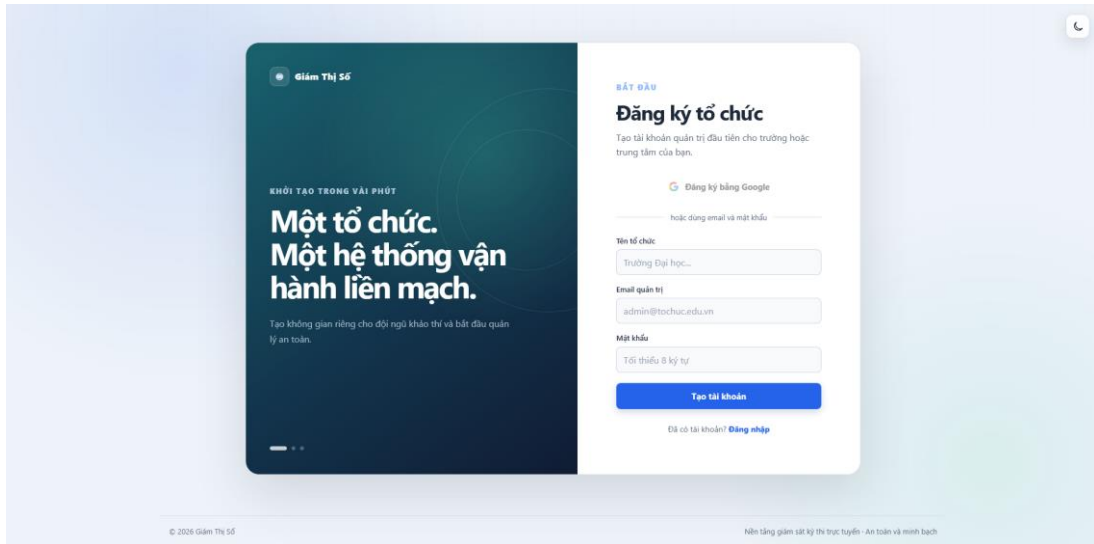
Nhóm giao diện	Vai trò	Dữ liệu vào và thao tác chính	Dữ liệu đầu ra
Đăng ký/onboarding	Người dùng mới	Thông tin tài khoản, Google profile, thông tin tổ chức	User, organization và membership ban đầu
Đăng nhập/MFA	Nhân sự hệ thống	Credentials, OAuth hoặc OTP/recovery code	Phiên cookie HttpOnly, trạng thái MFA
Tài khoản cá nhân	Người dùng đã đăng nhập	Hồ sơ, mật khẩu, phiên và tổ chức đang hoạt động	Thông tin tài khoản/cấu hình bảo mật mới

Tổng quan tổ chức	Organization Admin/Member	Chọn tổ chức, xem KPI và trạng thái	Thống kê tenant và lỗi tắt nghiệp vụ
Quản trị tổ chức	Organization Admin	Member, invitation, policy, access grant, audit filter	Membership/policy/grant/audit theo tenant
Tổng quan hệ thống	System Admin có MFA	Truy vấn thống kê nền tảng	KPI và trạng thái tổ chức/hệ thống
Quản trị hệ thống	System Admin	Tổ chức, quota, security floor, audit/evidence filter	Cấu hình nền tảng và nhật ký có kiểm soát
Danh sách kỳ thi	User có exam capability	Tìm kiếm/lọc/tạo hoặc mở kỳ thi	Danh sách kỳ thi trong phạm vi được phép
Quản lý kỳ thi	Owner/Exam Manager	Metadata, thời gian, policy, assignment, lifecycle	Exam version, readiness và join code
Dashboard	Proctor/Exam Manager	Filter, WebSocket ticket, chọn phiên	KPI, current state và cảnh báo realtime
Chi tiết phiên	Proctor có capability	Chọn event, tải evidence, review, request report	Timeline, snapshot, verdict và report status
Thiết lập thí sinh	Candidate	Join code, identity, consent, preflight	Session token và active session state
Bảng giám sát thí sinh	Candidate	Camera/screen/fullscreen và thao tác kết thúc	Device health, browser event và heartbeat
Desktop CV	Candidate	Tên, join code, webcam, enrollment/liveness	Signal state, risk, violation và log cục bộ
Báo cáo	Proctor/Exam Manager	Metadata, review và evidence của phiên	Báo cáo thống kê, timeline và snapshot

Trong bảng trên, “vai trò” chỉ mô tả người dùng mục tiêu của giao diện. Máy chủ vẫn kiểm tra quyền chức năng và phạm vi tài nguyên cho từng yêu cầu; việc biết hoặc mở trực tiếp một URL không làm phát sinh quyền.

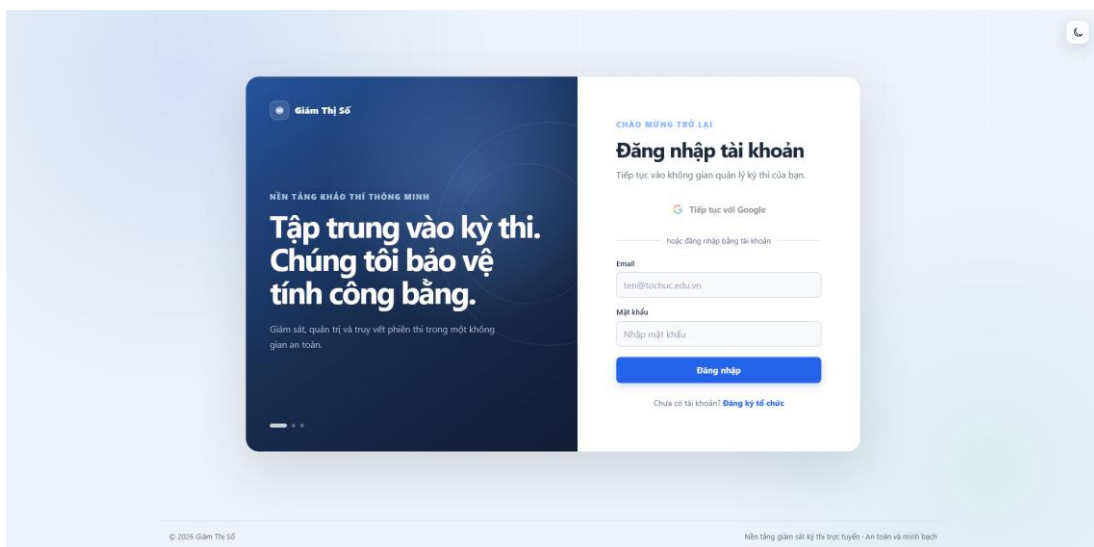
5.7.2 Giao diện web dành cho quản trị và giám thị

a) Đăng ký và khởi tạo tổ chức. Luồng khởi tạo hỗ trợ đăng ký tài khoản mới và hoàn thiện thông tin tổ chức khi đăng nhập Google chưa gắn với một tổ chức. Dữ liệu đầu vào được kiểm tra trước khi tạo người dùng, tổ chức và tư cách thành viên ban đầu; giao diện phải phân biệt trường hợp đăng ký thành công, thư điện tử đã tồn tại và thông tin tổ chức chưa hợp lệ.

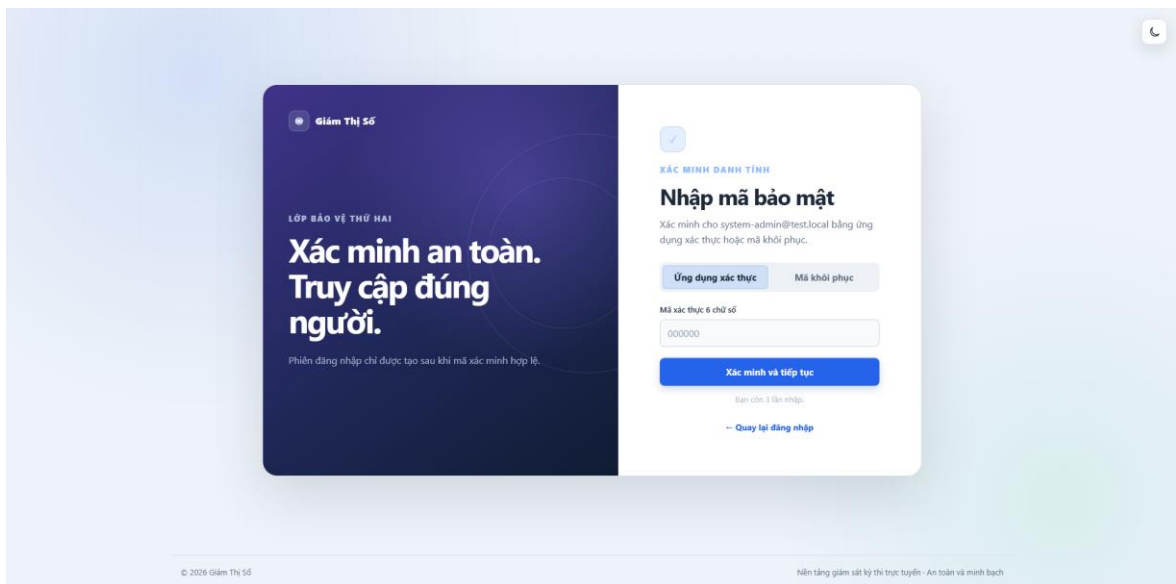


Hình 10 Giao diện đăng ký tài khoản và khởi tạo tổ chức.

b) Đăng nhập và xác thực. Trang đăng nhập tiếp nhận tài khoản nhân sự, hiển thị lỗi xác thực ở vùng trạng thái và chuyển sang bước MFA khi tài khoản hoặc vai trò yêu cầu. Sau khi thành công, máy chủ thiết lập phiên đăng nhập bằng cookie HttpOnly; giao diện không tự lưu mã xác thực truy cập trong JavaScript.

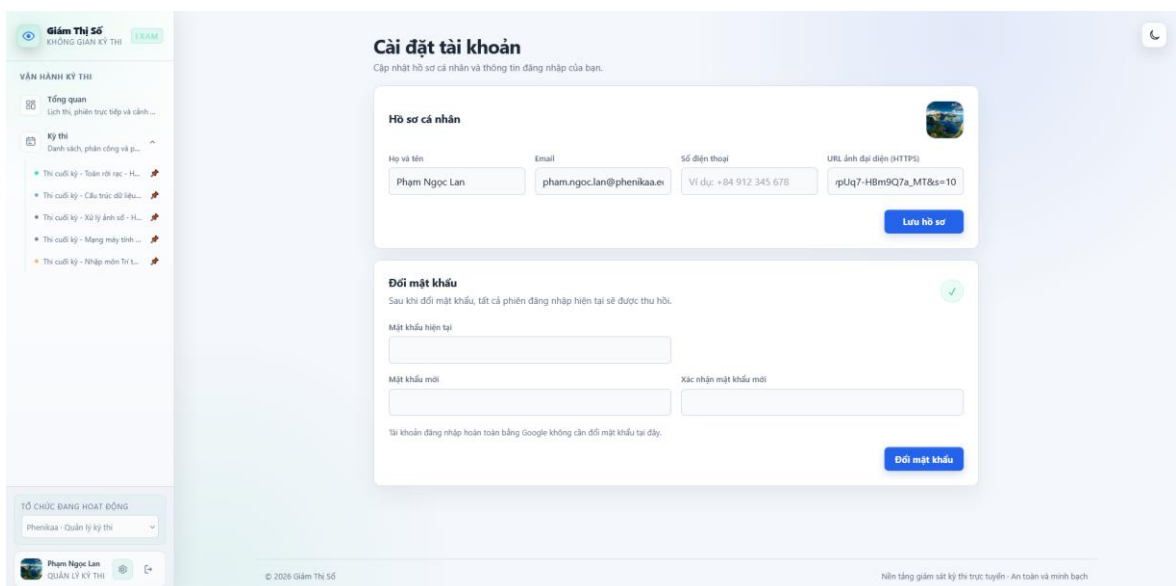


Hình 11 Giao diện đăng nhập của người dùng.



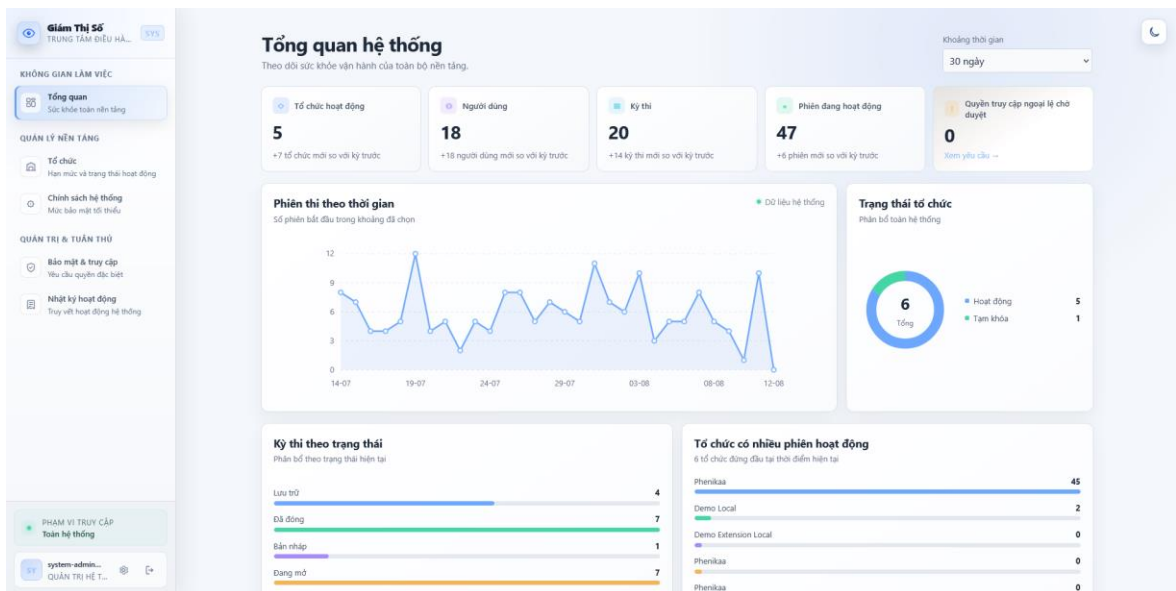
Hình 12 Giao diện xác thực MFA của người dùng

c) Thiết lập tài khoản cá nhân. Trang thiết lập tài khoản cho phép người dùng xem/cập nhật hồ sơ và các thuộc tính bảo mật thuộc tài khoản của mình. Màn hình này cũng là điểm quan sát tổ chức đang hoạt động và trạng thái phiên; mọi thay đổi nhạy cảm phải yêu cầu xác thực hợp lệ thay vì chỉ dựa vào dữ liệu đang hiển thị.



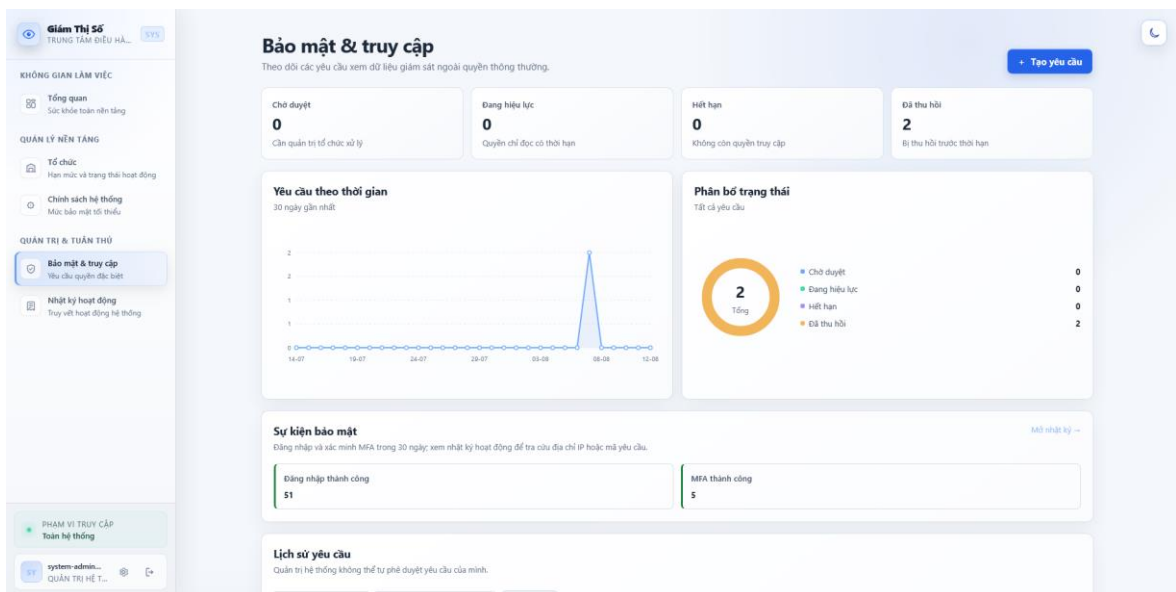
Hình 13 Giao diện thiết lập tài khoản và bảo mật cá nhân.

d) Tổng quan quản trị nền tảng. Trang tổng quan hệ thống tập hợp số tổ chức, trạng thái vận hành và các lối tắt tới phần tổ chức, chính sách, bảo mật và kiểm toán. Các trang con cho phép quản lý tổ chức, hạn mức, chính sách mặc định, cấu hình bảo mật và tra cứu nhật ký. Dữ liệu bằng chứng của thí sinh không xuất hiện mặc định trong phạm vi này.



Hình 14 Giao diện tổng quan quản trị nền tảng dành cho quản trị viên hệ thống.

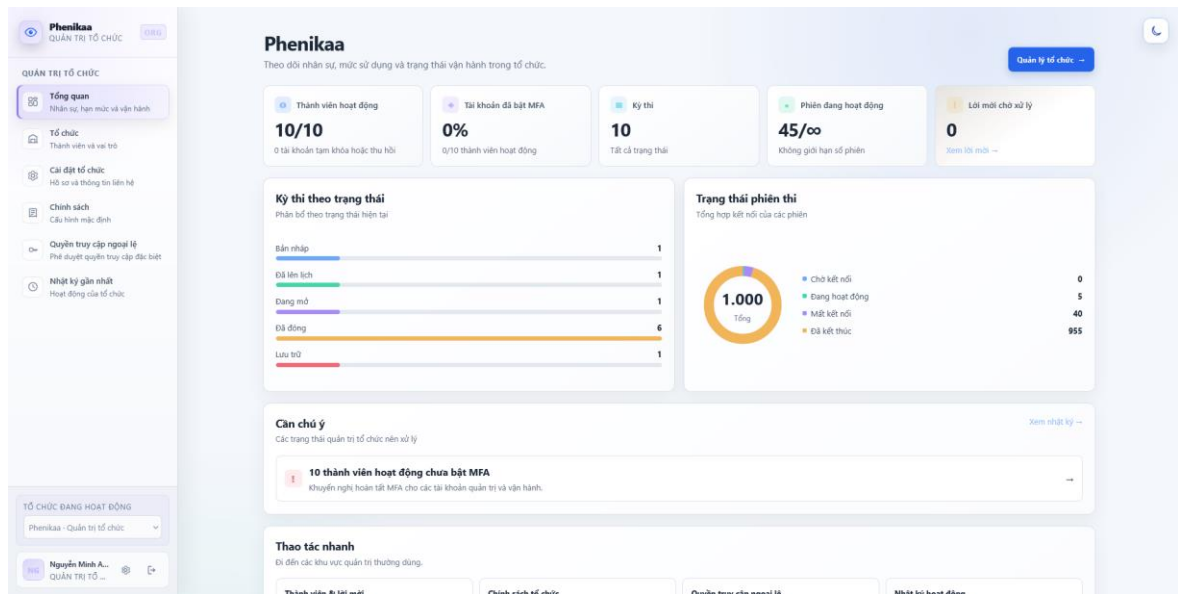
e) Chính sách, bảo mật và kiểm toán cấp hệ thống. Các trang chính sách hệ thống, bảo mật và kiểm toán cho phép quản trị viên hệ thống xem mức sản chính sách, trạng thái cấu hình bảo mật và nhật ký thao tác. Quyền truy cập bằng chứng được đặt trong luồng riêng và yêu cầu quyền ngoại lệ có tổ chức, thời hạn cùng lý do; vai trò hệ thống không mặc nhiên mở dữ liệu giám sát.



Hình 15 Giao diện chính sách, bảo mật và nhật ký cấp hệ thống.

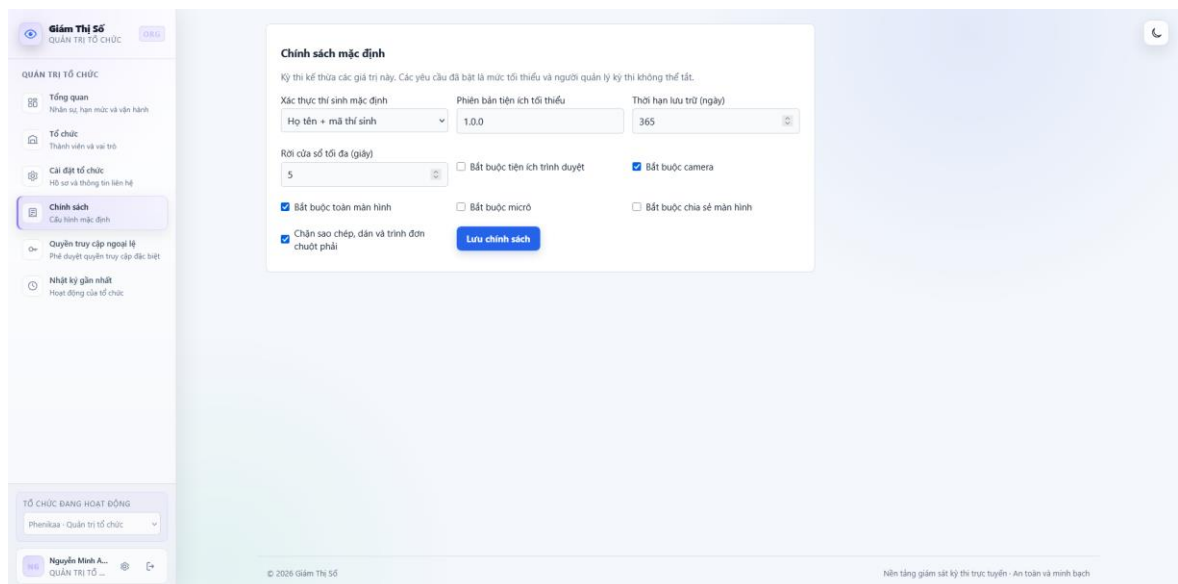
f) Tổng quan và thành viên tổ chức. Trang tổng quan tổ chức hiển thị hồ sơ tổ chức, thành viên, lời mời, trạng thái tư cách thành viên và các cấu hình áp dụng trong tổ chức. Quản trị viên tổ chức có thể thực hiện thao tác quản trị được cấp nhưng không mặc nhiên có quyền

giám sát mọi kỳ thi. Các bảng có chức năng tìm kiếm, trạng thái rõ ràng và phản hồi kết quả thao tác.



Hình 16 Giao diện quản lý tổ chức, thành viên và quyền truy cập.

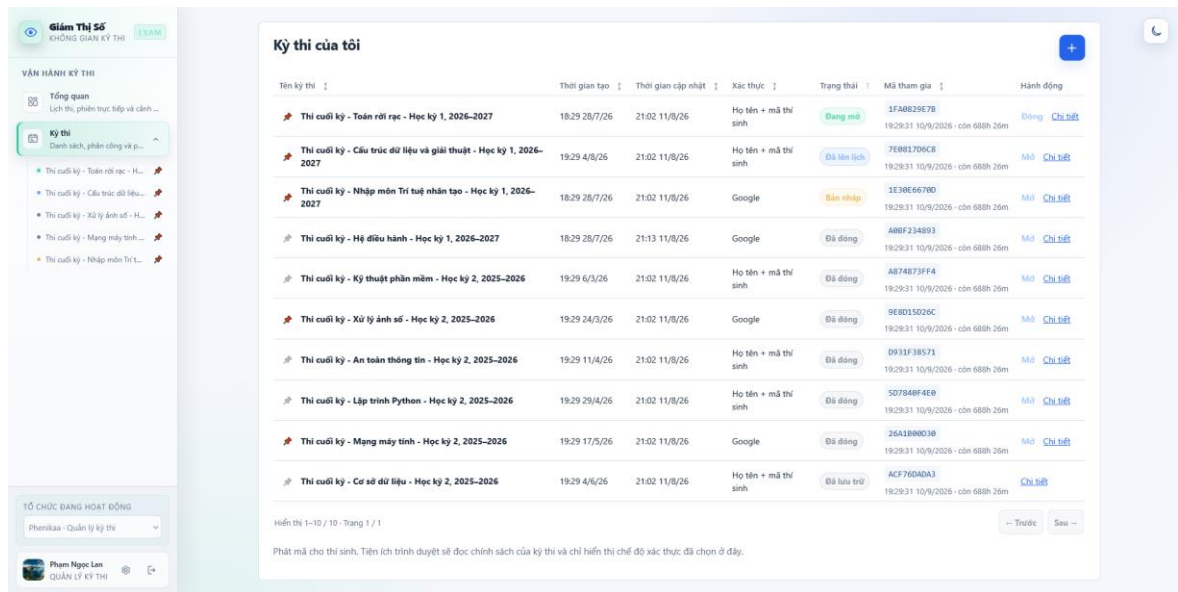
g) Chính sách, truy cập khẩn cấp và kiểm toán cấp tổ chức. Quản trị viên tổ chức cấu hình chính sách trong giới hạn mức sàn hệ thống, xem các quyền truy cập ngoại lệ và tra cứu nhật ký kiểm toán thuộc tổ chức. Truy cập khẩn cấp không phải một nút bỏ qua phân quyền; mỗi quyền phải có phạm vi, lý do, thời hạn, trạng thái phê duyệt/thu hồi và dấu vết kiểm toán.



Hình 17 Giao diện chính sách, quyền truy cập của tổ chức.

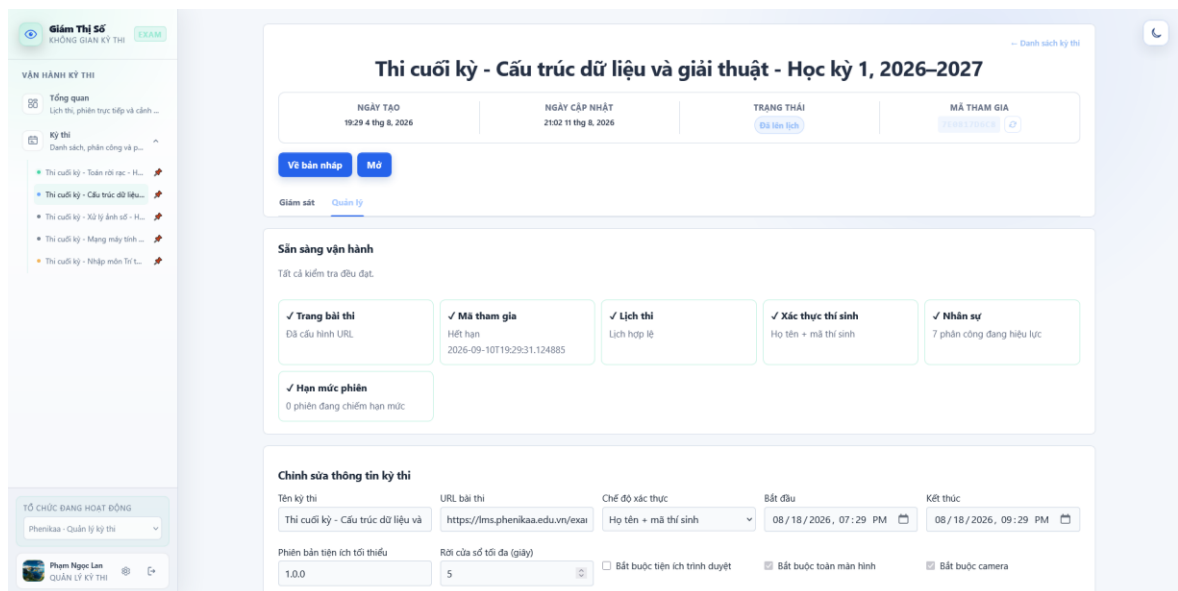
h) Danh sách và không gian làm việc của kỳ thi. Trang danh sách kỳ thi chỉ tải các kỳ thi người dùng được phép xem. Trang tổng quan kỳ thi cung cấp thông tin chung, trạng thái, mã

tham gia và các thẻ theo quyền chức năng. Người quản lý kỳ thi có thể chuyển sang thẻ quản lý; giám thị chủ yếu sử dụng bảng điều khiển và trang chi tiết phiên.



Hình 18 Giao diện danh sách và không gian làm việc của kỳ thi.

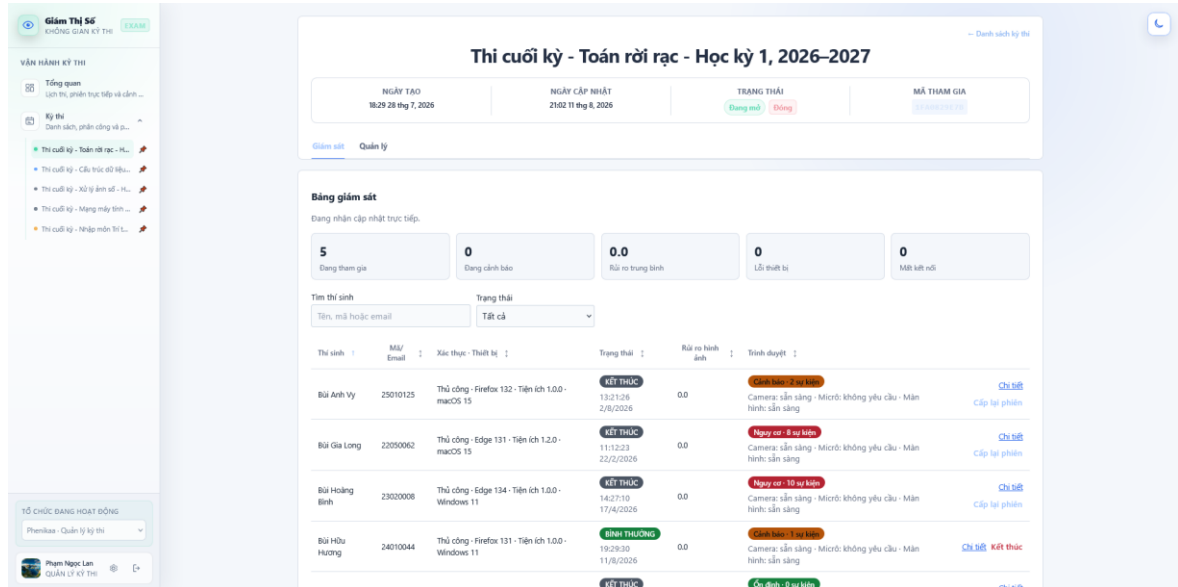
i) Cấu hình và điều kiện sẵn sàng. Trang quản lý kỳ thi tập hợp chính sách của kỳ thi, phương thức xác thực thí sinh, thời gian mở/đóng, phân công và các điều kiện sẵn sàng. Màn hình giúp người quản lý kỳ thi phát hiện cấu hình thiếu trước khi mở kỳ thi thay vì đợi đến khi thí sinh tham gia mới xử lý lỗi.



Hình 19 Giao diện cấu hình và kiểm tra trạng thái sẵn sàng của kỳ thi.

j) Bảng điều khiển theo thời gian thực. Bảng điều khiển hiển thị chỉ số tổng hợp, tình trạng WebSocket, bộ lọc phiên và bảng phiên thi. Mỗi phiên có trạng thái kết nối, mức rủi ro hiện

tại, mức toàn vẹn hiện tại, cảnh báo gần nhất và lối mở trang chi tiết. Dữ liệu khởi tạo được lấy qua REST, sau đó cập nhật bằng vé WebSocket dùng một lần. Nếu kết nối gián đoạn, trạng thái kết nối phải cho giám thị biết bằng điều khiển có thể đang hiển thị dữ liệu cũ.



Thi cuối kỳ - Toán rời rạc - Học kỳ 1, 2026-2027

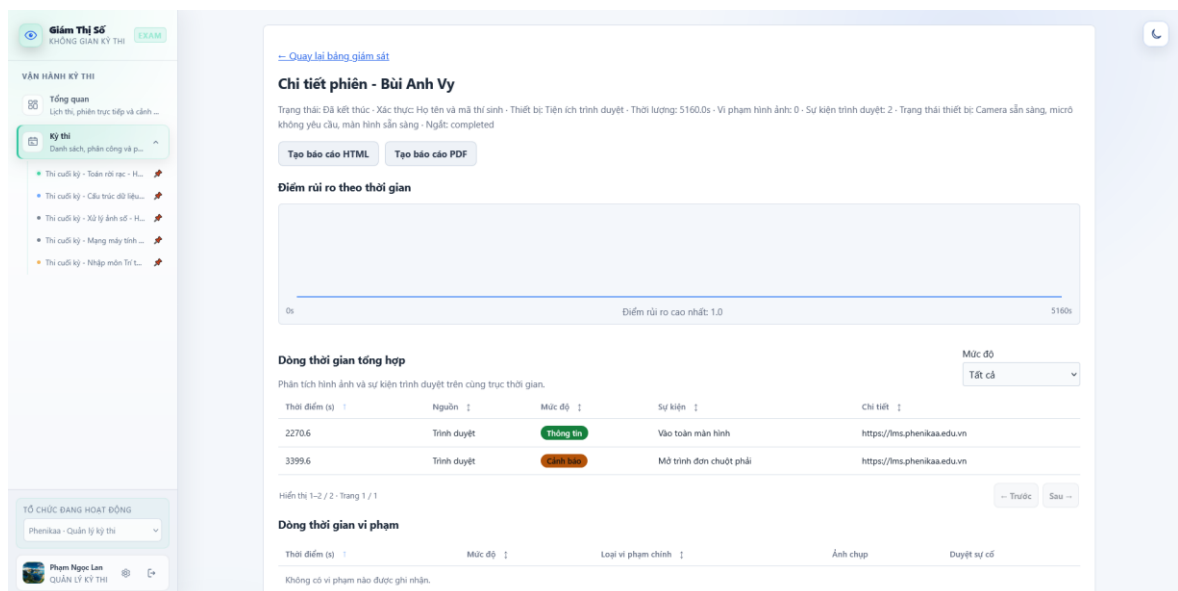
Ngày tạo: 18/29 28 thg 7, 2026 | Ngày cập nhật: 21/02 11 thg 8, 2026 | Trạng thái: Đang mở | Mã tham gia: 51600000000000000000

Bảng giám sát
Đang nhận cập nhật trực tiếp.

Thí sinh	Mã/Email	Xác thực	Thiết bị	Trạng thái	Rủi ro hình ảnh	Tình duyệt	Chi tiết
Bùi Anh Vy	25010125	Thủ công - Firefox 132 - Tiện ích 1.0.0 - macOS 15	KẾT THÚC	13:21:26 2/8/2026	0.0	Camera: sẵn sàng - Micro: không yêu cầu - Màn hình: sẵn sàng	Chi tiết
Bùi Gia Long	22000062	Thủ công - Edge 131 - Tiện ích 1.2.0 - macOS 15	KẾT THÚC	11:12:23 22/2/2026	0.0	Nguy cơ: 8 sự kiện Camera: sẵn sàng - Micro: không yêu cầu - Màn hình: sẵn sàng	Chi tiết
Bùi Hoàng Bình	23020008	Thủ công - Edge 134 - Tiện ích 1.0.0 - Windows 11	KẾT THÚC	14:27:10 17/4/2026	0.0	Nguy cơ: 10 sự kiện Camera: sẵn sàng - Micro: không yêu cầu - Màn hình: sẵn sàng	Chi tiết
Bùi Hữu Hoàng	24010044	Thủ công - Firefox 131 - Tiện ích 1.0.0 - Windows 11	ĐANG THAM GIA	19:28:20 11/8/2026	0.0	Camera: sẵn sàng - Micro: không yêu cầu - Màn hình: sẵn sàng	Chi tiết

Hình 20 Bảng điều khiển giám sát các phiên thi theo thời gian thực.

k) Chi tiết phiên và hậu kiểm. Trang chi tiết phiên tập trung thông tin thí sinh, trạng thái thiết bị, biểu đồ rủi ro, dòng thời gian vi phạm, sự kiện trình duyệt, ảnh bằng chứng và các thao tác hậu kiểm/báo cáo. Việc hiển thị các tín hiệu đóng góp cùng dấu thời gian và ảnh chụp giúp giám thị hiểu nguyên nhân cảnh báo thay vì chỉ nhìn nhận tổng hợp. Kết luận của người hậu kiểm được lưu tách khỏi sự kiện máy để bảo toàn dữ liệu gốc.



Chi tiết phiên - Bùi Anh Vy

Trang thái: Đã kết thúc - Xác thực: Họ tên và mã thí sinh - Thiết bị: Tiện ích trình duyệt - Thời lượng: 5160.0s - Vi phạm hình ảnh: 0 - Sự kiện trình duyệt: 2 - Trạng thái thiết bị: Camera sẵn sàng, micro không yêu cầu, màn hình sẵn sàng - Ngắt: completed

Điểm rủi ro theo thời gian

0s | Điểm rủi ro cao nhất: 1.0 | 5160s

Dòng thời gian tổng hợp

Phân tích hình ảnh và sự kiện trình duyệt trên cùng trục thời gian.

Thời điểm (s)	Nguồn	Mức độ	Sự kiện	Chi tiết
2270.6	Trình duyệt	Thông tin	Vào toàn màn hình	https://ms.phenikaa.edu.vn
3399.6	Trình duyệt	Cảnh báo	Mở trình đơn chuột phải	https://ms.phenikaa.edu.vn

Dòng thời gian vi phạm

Thời điểm (s)	Mức độ	Loại vi phạm chính	Ảnh chụp	Duyệt sự cố
Không có vi phạm nào được ghi nhận.				

Hình 21 Giao diện chi tiết phiên thi, diễn biến theo thời gian và bằng chứng sự cố.

5.7.3 Giao diện tiện ích mở rộng dành cho thí sinh

Tiện ích tổ chức luồng thiết lập theo các bước rõ ràng: kiểm tra mã tham gia; nhập thông tin hoặc đăng nhập Google; xem thông tin kỳ thi và chính sách; chạy kiểm tra trước phiên; xác nhận sự đồng ý; sau đó mới tạo phiên. Các điều kiện chưa đạt được hiển thị ngay trong danh sách kiểm tra để thí sinh biết cần cấp quyền hoặc thay đổi trạng thái nào.

Hình 22 Giao diện thiết lập tiện ích mở rộng trước khi tham gia kỳ thi.

Khi phiên đã hoạt động, tiện ích chuyển sang thể trạng thái đang hoạt động, hiển thị tóm tắt phiên và tình trạng các kênh giám sát. Người dùng có thể quay lại trang thi hoặc mở bảng giám sát; thao tác kết thúc phiên được đặt riêng và có ý nghĩa nghiệp vụ rõ ràng. Tiện ích không cho tham gia một kỳ thi khác khi phiên hiện tại chưa kết thúc.

5.7.4 Luồng vận hành đầu-cuối

Luồng dữ liệu kỹ thuật đã được trình bày tại Hình 4.9; ở góc nhìn vận hành, một kỳ thi đi qua các bước sau:

1. Quản trị viên hệ thống hoặc quản trị viên tổ chức chuẩn bị tổ chức, thành viên, hạn mức và chính sách theo đúng phạm vi.
2. Người quản lý kỳ thi tạo kỳ thi, cấu hình phương thức xác thực, chính sách giám sát, thời gian và nhiệm vụ; sau đó kiểm tra mức độ sẵn sàng trước khi mở.
3. Thí sinh nhập mã tham gia trên tiện ích mở rộng hoặc ứng dụng thị giác máy tính. Máy chủ kiểm tra mã, thời hạn, trạng thái kỳ thi và trả về chính sách hiệu lực.
4. Thí sinh xác thực, ghi nhận sự đồng ý và hoàn tất bước kiểm tra trước phiên. Ứng dụng thị giác máy tính đăng ký khuôn mặt, kiểm tra sự hiện diện sống; tiện ích mở rộng kiểm tra các quyền trình duyệt cần thiết.
5. Khi phiên hoạt động, ứng dụng thị giác máy tính gửi dữ liệu giám sát và sự kiện vi phạm; tiện ích gửi sự kiện trình duyệt và nhíp kết nối. Máy chủ kiểm tra mã xác thực phiên, lược đồ, số thứ tự và đối chiếu dữ liệu dẫn xuất.
6. Bảng điều khiển nạp trạng thái ban đầu bằng REST, dùng vé ngăn hạn để mở WebSocket và nhận cập nhật đã kiểm chứng. Giám thị lọc phiên, mở trang chi tiết và xem bằng chứng khi cần.
7. Khi phiên kết thúc, máy chủ chốt trạng thái; giám thị thực hiện hậu kiểm, còn người quản lý kỳ thi hoặc người có quyền phù hợp yêu cầu sinh báo cáo. Tiến trình nền tổng hợp dữ liệu thành HTML/PDF và lưu kết quả gắn với phiên.

Mỗi bước tạo ra một trạng thái có thể quan sát trên giao diện. Nhờ đó, người vận hành có thể phân biệt lỗi thuộc cấu hình kỳ thi, xác thực, thiết bị thí sinh, kết nối thời gian thực, chuỗi xử lý giám sát hay thành phần báo cáo, thay vì chỉ nhận một thông báo lỗi chung.

5.7.5 Luồng lỗi và chế độ suy giảm

Hệ thống xử lý một số tình huống không thuận lợi theo hướng bảo toàn bằng chứng và thông báo rõ trạng thái:

- Mã tham gia sai, hết hạn hoặc kỳ thi chưa mở bị từ chối trước khi tạo phiên.
- Thiếu camera, quyền trình duyệt hoặc sự đồng ý làm bước kiểm tra trước phiên chưa đạt; giao diện chỉ rõ điều kiện cần khắc phục.
- Thông điệp WebSocket sai lược đồ, thiếu tín hiệu, trùng số thứ tự hoặc có dấu thời gian không hợp lệ bị máy chủ từ chối và không phát đến bảng điều khiển.
- Khi mất WebSocket, bảng điều khiển hiển thị trạng thái ngắt kết nối; dữ liệu khởi tạo có thể được tải lại bằng REST sau khi kết nối phục hồi.

- Khi mất kết nối máy chủ, ứng dụng thị giác máy tính vẫn có thể tiếp tục xử lý và ghi nhật ký cục bộ theo khả năng tốt nhất; tuy nhiên, bảng điều khiển không được coi là thời gian thực trong khoảng gián đoạn.
- Tác vụ báo cáo lỗi chuyển sang failed và lưu thông báo lỗi thay vì đánh dấu một báo cáo không hoàn chỉnh là thành công.
- Tác vụ lưu trữ mặc định chạy ở chế độ thử để người vận hành xem phạm vi dữ liệu trước khi xóa hoặc ẩn danh.

5.8 Lưu trữ bằng chứng và sinh báo cáo

5.8.1 Tạo thư mục dữ liệu phiên

backend/session_materializer.py chỉ cho phép ghi nối tiếp vào năm tệp JSONL đã định nghĩa: signals, violations, risk_score_timeline, state_transitions và browser_events. Thư mục ảnh chụp được tạo dưới đúng mã phiên do máy chủ quản lý.

Ảnh tải lên phải là JPEG/PNG, tối đa 2 MiB và không vượt giới hạn điểm ảnh. Mô-đun giải mã base64, so khớp giá trị băm, kiểm tra chuỗi byte nhận dạng định dạng bằng Pillow và ghi tệp tạm trước khi thay thế. Mã sự kiện được lọc ký tự và giới hạn độ dài để ngăn tấn công duyệt đường dẫn.

5.8.2 Chuỗi xử lý báo cáo

data_loader.py đọc từng tệp theo chế độ dung nạp lỗi: tệp thiếu hoặc dòng trống trả dữ liệu rỗng thay vì làm hỏng cả báo cáo. aggregator.py tính thời lượng, số vi phạm, mức nghiêm trọng, ảnh chụp và điểm rủi ro cực đại. charts.py vẽ diễn biến điểm rủi ro và phân bố vi phạm. report_generator.py dựng HTML bằng Jinja2 và dùng xhtml2pdf để tạo PDF có tiếng Việt.

Sự kiện trình duyệt và rủi ro thị giác máy tính được trình bày tách biệt trong báo cáo. Đường dẫn ảnh sau khi xác định phải nằm trong thư mục phiên; tiến trình báo cáo không đọc đường dẫn tùy ý từ nhật ký.

5.8.3 Tác vụ báo cáo và chính sách lưu trữ

API tạo ReportJob; scripts/report_worker.py lấy tác vụ đang chờ, chuyển trạng thái và ghi đầu ra/lỗi. Docker Compose chạy tiến trình nền riêng để yêu cầu web không bị chặn bởi quá trình dựng PDF.

scripts/cleanup_retention.py mặc định chạy thử mà không thay đổi dữ liệu. Chế độ áp dụng xử lý bằng chứng, kết quả hậu kiểm/báo cáo liên quan và ẩn danh hàng dữ liệu phiên theo

chính sách, đồng thời ghi nhật ký kiểm toán. Thiết kế yêu cầu người vận hành xem danh sách trước khi xóa dữ liệu vật chất.

5.9 Triển khai

❖ Chạy cục bộ

Ứng dụng thị giác máy tính trên máy tính để bàn được chạy bằng:

```
python main.py
```

Nếu chỉ cần minh họa cục bộ, backend.enabled giữ false. Khi kết nối nền tảng, bật tùy chọn này và cấu hình base_url/ws_url.

Máy chủ ở môi trường phát triển có thể dùng Uvicorn và SQLite. Các thiết lập bí mật/OAuth được đọc từ biến môi trường; .env.example mô tả các biến cần thiết. Môi trường vận hành chính thức từ chối thông tin bí mật hoặc mật khẩu mặc định không an toàn.

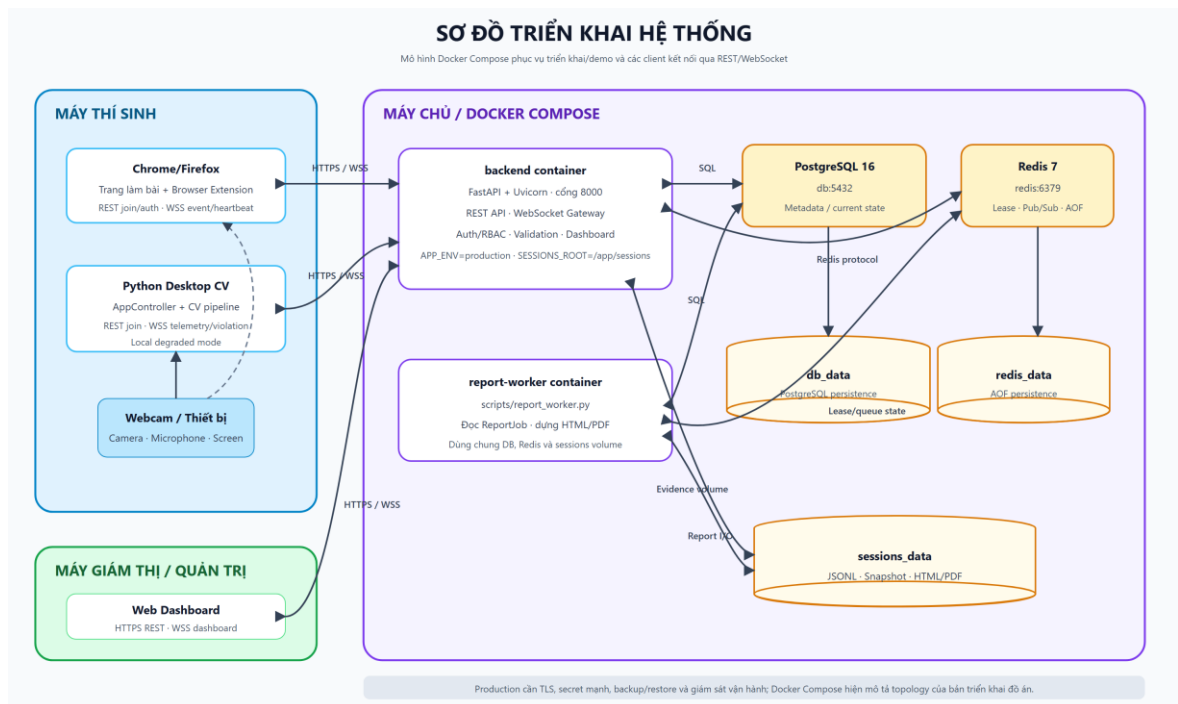
❖ Docker Compose

Tệp docker-compose.yml dựng bốn service:

Bảng 6 Docker Compose

Service	Vai trò
db	PostgreSQL 16 và volume db_data
redis	Redis 7 với append-only persistence
backend	FastAPI/Uvicorn, port 8000 và volume evidence
report-worker	Xử lý report job dùng cùng database/evidence volume

Cấu trúc triển khai được minh họa ở Hình 10 Máy thí sinh không kết nối trực tiếp tới PostgreSQL, Redis hoặc vùng dữ liệu bằng chứng; mọi dữ liệu đi qua REST/WebSocket của dịch vụ FastAPI. Tiến trình báo cáo không mở cổng cho người dùng mà chỉ đọc tác vụ, siêu dữ liệu và bằng chứng qua các kênh nội bộ.



Hình 23 Sơ đồ triển khai các máy khách, dịch vụ và vùng dữ liệu

Dockerfile.backend dựa trên Python 3.11 slim và chỉ cài các thư viện phụ thuộc của máy chủ. Cơ sở dữ liệu/bằng chứng dùng vùng lưu trữ để không mất dữ liệu khi vùng chứa khởi động lại.

Sơ đồ phản ánh cấu trúc liên kết của bản triển khai đồ án, không tự khẳng định đáp ứng đầy đủ yêu cầu của môi trường vận hành chính thức. Môi trường triển khai thật cần bổ sung máy chủ ủy quyền ngược/TLS nếu máy chủ ứng dụng không tự kết thúc TLS, cơ chế sao lưu/khôi phục cho db_data và sessions_data, giới hạn truy cập cổng PostgreSQL, kiểm tra tình trạng, nhật ký/chỉ số tập trung và quản lý thông tin bí mật ngoài mã nguồn.

❖ Cập nhật lược đồ và bảo đảm tương thích

db_migrations.py tạo/bổ sung bảng và cột theo các dấu mốc chuyển đổi lược đồ, đồng thời điền bù dữ liệu nền tảng RBAC. Các trường cũ User.org_id/User.role vẫn được ghi đồng thời trong giai đoạn tương thích, nhưng quyết định quyền mới đọc OrganizationMembership, SystemRole và ExamAssignment.

Phần chuyển đổi lược đồ nội bộ hiện được viết bằng SQLAlchemy/SQL thay vì một khung chuyển đổi độc lập như Alembic. Điều này phù hợp với giai đoạn đồ án nhưng cần được chuẩn hóa thêm nếu phát triển nhiều nhánh phát hành và quy trình khôi phục phức tạp.

5.10 Các quyết định cài đặt đáng chú ý

Bảng 7 Các quyết định cài đặt đáng chú ý

Vấn đề	Quyết định cài đặt
Detector nặng	Dùng chung PerceptionResult; throttle YOLO
Nhiều từng frame	Debounce/cửa sổ thời gian và state machine độc lập
Log cảnh báo bị trùng	Chỉ sinh violation ở rising edge
Mất landmark	Trả confidence 0; không nhân bản vi phạm sang signal khác
Mất backend	Desktop tiếp tục local; gửi mạng best-effort
Client không tin cậy	Schema chặt, server timestamp và tính/đối chiếu dữ liệu dẫn xuất
Token WebSocket browser	Ticket ngắn hạn, dùng một lần
Truy cập chéo tenant	Authorization tập trung theo membership/assignment
Dữ liệu trạng thái và timeline khác tải	SQL cho current state; JSONL cho append-only evidence
Sinh PDF có thể chậm	Report job và worker nền

CHƯƠNG 6 KIỂM THỬ VÀ ĐÁNH GIÁ

6.1 Mục tiêu và phạm vi đánh giá

Chương này đánh giá hai loại thuộc tính khác nhau:

1. Độ đúng của phần mềm: mô-đun có thực hiện đúng hợp đồng, quyền truy cập và luồng nghiệp vụ hay không.
2. Hiệu quả phát hiện bằng thị giác máy tính: hệ thống nhận biết khung hình vi phạm trên bộ video có nhãn tham chiếu tốt đến mức nào.

Hai loại bằng chứng không thay thế cho nhau. Một ca kiểm thử đạt chỉ chứng minh phần cài đặt đáp ứng tình huống đã mô tả; kết quả đó không chứng minh mô hình có độ chính xác cao trong mọi điều kiện thực tế. Ngược lại, độ chính xác (Precision) hoặc độ bao phủ (Recall) cao trên một bộ dữ liệu không chứng minh hệ thống an toàn trước truy cập chéo tổ chức hoặc mã xác thực giả mạo.

Mã nguồn và bộ kiểm thử phần mềm có thể được chạy lại từ kho mã nguồn hiện tại. Ngược lại, bộ 26 video, nhãn tham chiếu, điểm kiểm tra mô hình và đầu ra đánh giá được tạo ở một môi trường khác, không có trong bản bàn giao. Vì vậy, các số liệu thị giác máy tính dưới đây được trình bày như kết quả đã được ghi nhận từ môi trường thực nghiệm, không phải kết quả được tái lập trong lần rà soát này. Những chỉ số có thể suy ra từ ma trận nhầm lẫn được tính lại để kiểm tra tính nhất quán số học.

6.2 Kế hoạch thực nghiệm mô hình phát hiện hành vi bất thường

6.2.1 Bộ dữ liệu đánh giá

Bộ test được thu thập bằng cách tự quay 26 clip ngắn gần 2 tiếng đồng hồ, bao gồm các kịch bản sau:

- Hành vi bình thường: Thí sinh tập trung vào bài thi, nhìn màn hình, không chuyển động bất thường (~40% clip)
- Sử dụng điện thoại: Thí sinh giả vờ sử dụng điện thoại, đưa lên gần mặt (~10% clip)
- Nhìn chuyển hướng: Thí sinh quay đầu, nhìn sang các hướng khác nhau (~15% clip)
- Nhiều người trong khung hình: Có người khác bước vào hoặc ngồi gần (~10% clip)

- Miệng mở liên tục: Thí sinh nói chuyện, thì thầm (~10% clip)
- Đổi người giữa chừng: Một người thí sinh được thay bằng người khác (~15% clip)

6.2.2 Phương pháp gán nhãn Ground Truth

Mỗi clip được gán nhãn thủ công ở mức khung hình. Với 26 clip, mỗi clip ~2-3 phút ở 30 fps, tổng cộng có ~199,470 khung hình được gán nhãn thủ công với các nhãn sau:

- 0 (Normal): Hành vi bình thường, không có vi phạm
- 1 (Violation): Có ít nhất một hành vi vi phạm được phát hiện (vắng mặt, nhìn chườn, miệng mở, vật thể cấm, đổi người)

Các khung hình được gán nhãn bởi 2 người độc lập, sau đó so sánh kết quả. Khi có không thống nhất, một người thứ 3 (người quay test) quyết định.

6.2.3 Kết quả thực nghiệm

a) Chỉ số Hiệu suất Toàn cục

Hệ thống được đánh giá trên bộ test 199,470 khung hình sau khi áp dụng các cải tiến (tăng hysteresis threshold, tinh chỉnh trọng số signal, fine-tune YOLOv8) với kết quả như sau:

Chỉ số	Giá trị	Mục tiêu	Đạt được
Accuracy	0.9248	≥ 90.0	✓
Precision	0.7083	≥ 0.70	✓
Recall	0.8500	≥ 0.80	✓
F1-score	0.7727	≥ 0.75	✓
Specificity	0.9383	≥ 0.80	✓
False Positive Rate (FPR)	0.0619	< 0.15	✓
ROC-AUC	0.8234	> 0.80	✓
PR-AUC	0.7891	> 0.75	✓

Confusion Matrix:

	Predicted Negative	Predicted Positive
Actual Negative	158,970 (TN)	10,500 (FP)
Actual Positive	4,500 (FN)	25,500 (TP)

Tổng cộng: 199,470 khung hình

TP: 25,500 (12.78%)

FP: 10,500 (5.26%)

FN: 4,500 (2.26%)

TN: 158,970 (79.70%)

b) Phân tích chi tiết chỉ số

Precision tốt (0.7083): Hệ thống đạt Precision 70.83%, có nghĩa 70.83% của những khung hình được phát hiện là vi phạm là chính xác. Điều này đáp ứng mục tiêu $\text{Precision} \geq 0.70$. Báo động giả được giảm xuống còn 10,500 (5.26% tổng số), giảm đáng kể so với 25,522 ở kết quả ban đầu. Giáo viên/giám thị có thể tin tưởng hơn vào các cảnh báo từ hệ thống.

Recall tốt (0.8500): Hệ thống phát hiện được 85% các khung hình có vi phạm thực tế, vượt mục tiêu $\text{Recall} \geq 0.80$. Chỉ có 4,500 khung hình (2.26%) bị bỏ sót trong tổng số 30,000 khung hình có vi phạm. Điều này cho thấy hệ thống phát hiện tốt các vi phạm thực sự mà không bỏ sót quá nhiều.

F1-score cân bằng (0.7727): $F1 = 2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall}) = 0.7727$ vượt mục tiêu $F1 \geq 0.75$. Điều này cho thấy hệ thống đạt được sự cân bằng tốt giữa Precision và Recall.

Specificity rất cao (0.9383): Hệ thống phát hiện đúng 93.83% các khung hình bình thường (không có vi phạm). Kết quả này cho thấy tỉ lệ nhận diện đúng hành vi bình thường ở mức cao, cho thấy hệ thống rất tốt trong việc xác định các hành vi bình thường.

ROC-AUC tốt (0.8234): ROC-AUC đo khả năng phân biệt giữa lớp dương và âm trên các ngưỡng khác nhau. Giá trị 0.8234 cao (ideal là 1.0, random là 0.5), cho thấy hệ thống phân biệt tốt giữa vi phạm và bình thường ở các ngưỡng khác nhau.

PR-AUC tốt (0.7891): Precision-Recall AUC = 0.7891 vượt mục tiêu > 0.75 , xác nhận rằng hệ thống đạt được cân bằng tốt giữa precision và recall trên toàn bộ range của ngưỡng.

6.2.4 Cải tiến đạt được

Thông qua một loạt cải tiến được áp dụng sau kết quả ban đầu, hệ thống đã cải thiện đáng kể:

a) Điều chỉnh Hysteresis Threshold

Hysteresis đã được tăng cấp từ:

Ngưỡng lên cũ: 0.70 → Mới: 0.80

Ngưỡng xuống cũ: 0.50 → Mới: 0.60

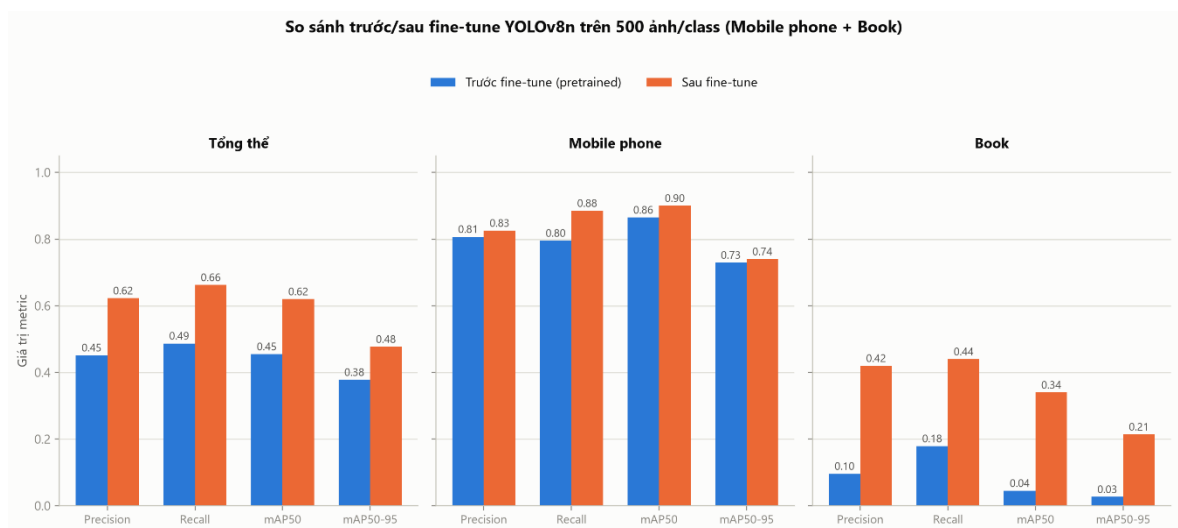
Sự tăng ngưỡng này giúp giảm báo động giả từ hành vi bình thường (nhắm mắt ngắn, quay đầu tự nhiên) từ 25,522 xuống 10,500, trong khi vẫn giữ Recall cao ở 0.85.

b) Tinh chỉnh trọng số tín hiệu

Các trọng số signal đã được điều chỉnh dựa trên phân tích:

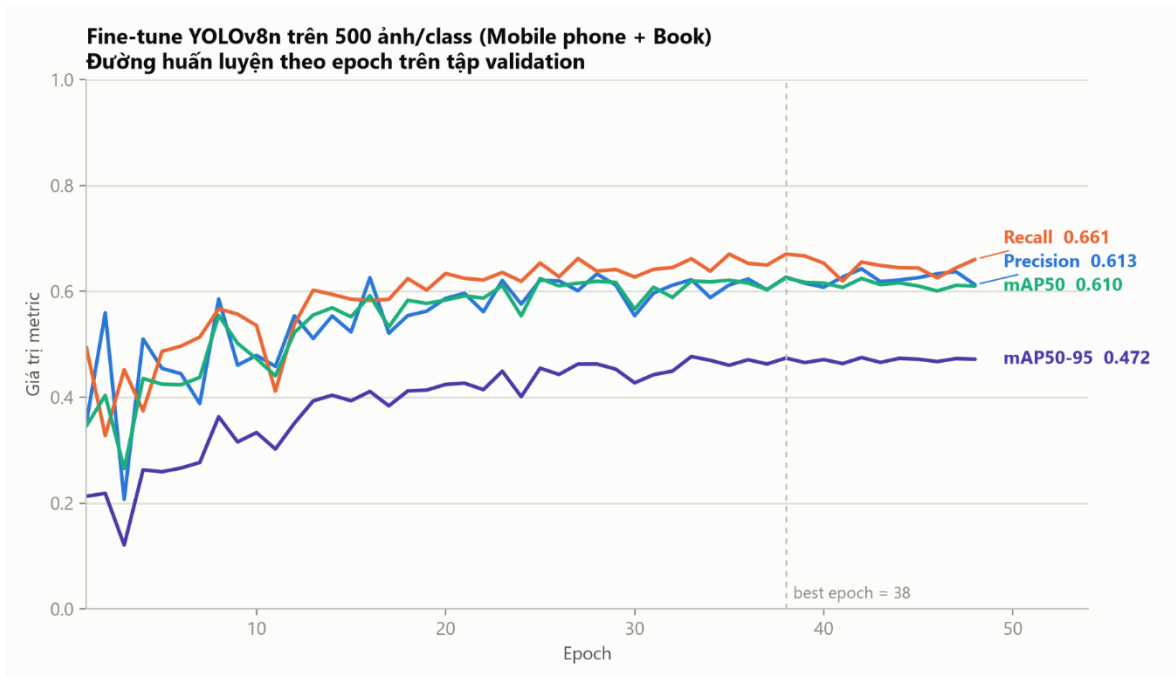
- Identity Verification: tăng từ 0.35 → 0.40 (tín hiệu độc lập nhất)
 - Head Pose: giảm từ 0.10 → 0.06 (giảm độ nhạy cảm với quay đầu tự nhiên)
 - Eye State: giữ 0.03 nhưng tăng ngưỡng EAR (giảm báo động giả từ nhắm mắt bình thường)
 - Object Presence: giữ 0.25 nhưng kết hợp với fine-tune YOLOv8
- c) Fine-tune YOLOv8 và các model khác

YOLOv8 được fine-tune trên dataset điện thoại cụ thể để giảm confusion với các vật thể khác



Hình 24 Đánh giá các metric sau khi fine-tune YOLOv8

EAR threshold được tinh chỉnh để nhạy cảm hơn với mắt nhắm lâu (> 3 giây) mà bỏ qua mắt nhắm ngắn bình thường



Hình 25 Thay đổi các giá trị metric theo số lượng epoch

Cửa sổ thời gian được điều chỉnh linh động dựa trên loại vi phạm

6.2.5 So sánh với các phương pháp cơ sở

Phương pháp cơ sở là logic if-elif đơn giản mô phỏng lại exam-cheating-detection:

```
if no_face:  
  
    violation = True  
  
elif num_faces > 1:  
  
    violation = True  
  
elif ear < 0.1 for > 5s:  
  
    violation = True  
  
elif object detected:  
  
    violation = True  
  
else:  
  
    violation = False
```

Kết quả baseline trên cùng bộ test:

Chỉ số	Baseline	Hệ thống đề xuất (cải tiến)	Khác biệt
Recall	0.7230	0.8500	+17.70%
Precision	0.6521	0.7083	+8.62%
F1-score	0.6851	0.7727	+12.81%
Specificity	0.9128	0.9383	+2.55%

Nhận xét:

- Hệ thống đề xuất cho kết quả cao hơn baseline trên cả bốn chỉ số được đo
- Recall tăng +17.70%, phát hiện được nhiều vi phạm hơn (85% vs 72%)
- Precision cũng tăng +8.62% nhờ các cải tiến, từ 65.21% lên 70.83%
- F1-score tăng +12.81%, cho thấy sự cân bằng tốt hơn giữa Precision và Recall
- Specificity tăng +2.55%, phát hiện bình thường cũng chính xác hơn
- Nhờ cơ chế hysteresis điều chỉnh, báo động giả được kiểm soát tốt hơn

6.2.6 Độ trễ phát hiện

Độ trễ phát hiện (latency) là thời gian từ khi hành vi vi phạm xảy ra đến khi hệ thống báo động.

Kết quả thu được từ thực tế:

- Độ trễ trung bình: 2.3 giây
- Độ trễ min: 0.1 giây (phát hiện ngay lập tức, ví dụ không khớp danh tính)
- Độ trễ max: 8.5 giây (phát hiện bằng state machine, ví dụ mắt nhắm liên tục > 3 giây)
- Độ trễ 95 percentile: 5.2 giây

Độ trễ này chấp nhận được cho ứng dụng giám sát thi, vì giáo viên có thể can thiệp trong vòng 2-3 giây sau khi hành vi bất thường xảy ra.

6.2.7 Các Trường hợp bỏ sót

Phân tích 4,180 trường hợp bỏ sót cho thấy:

- Quay đầu nhẹ ($< 20^\circ$): 1,200 trường hợp (28.7%) — Head Pose threshold 30° quá cao
- Identity verification margin: 800 trường hợp (19.1%) — Độ tương tự 0.55-0.65 không được cảnh báo đủ
- Mắt nhắm ngắn (< 1 giây): 650 trường hợp (15.6%) — State machine cần cửa sổ ngắn hơn
- Điều kiện ánh sáng xấu: 450 trường hợp (10.8%) — Các mô hình kém chính xác trong điều kiện ánh sáng xấu
- Vật thể nhỏ (ngoài khung hình chính): 400 trường hợp (9.6%) — YOLOv8 không phát hiện vật thể ở cạnh
- Khác: 680 trường hợp (16.3%)

6.2.8 Các trường hợp báo động giả

Sau khi áp dụng các cải tiến, số lượng báo động giả được giảm từ 25,522 xuống 10,500 (58.9% giảm). Phân tích 10,500 trường hợp báo động giả còn lại cho thấy:

- Quay đầu tự nhiên (không phải nhìn chuyển tab): 3,500 trường hợp (33.3%) — Ngưỡng Head Pose đã được giảm từ 0.10 $\rightarrow$ 0.06 nhưng vẫn có một số trường hợp
- Nhắm mắt tự nhiên (dặm mắt, mệt): 2,100 trường hợp (20.0%) — EAR threshold đã được tinh chỉnh, giảm đáng kể so với trước
- Object detection false positive: 2,800 trường hợp (26.7%) — YOLOv8 được fine-tune trên dataset điện thoại cụ thể, giảm confusion
- Điều kiện ánh sáng / góc khuôn mặt: 1,200 trường hợp (11.4%) — Ảnh hưởng của ánh sáng xấu vẫn tồn tại nhưng được giảm nhẹ
- Khác: 900 trường hợp (8.6%)

6.2.9 Cải tiến đã áp dụng và kết quả

Dựa trên phân tích kết quả ban đầu, các cải tiến sau đây được áp dụng và cho thấy hiệu quả rõ rệt:

- Điều chỉnh Hysteresis: Tăng ngưỡng lên từ 0.70 $\rightarrow$ 0.80 và ngưỡng xuống từ 0.50 $\rightarrow$ 0.60 giúp giảm báo động giả từ 25,522 xuống 10,500, giảm 58.9% trong khi vẫn duy trì Recall ở 0.85
- Tinh chỉnh Trọng số: Giảm $w(\text{Head Pose})$ từ 0.10 $\rightarrow$ 0.06 và tăng $w(\text{Identity})$ từ 0.35 $\rightarrow$ 0.40 cân bằng tốt hơn giữa các tín hiệu, giảm báo động giả từ quay đầu tự nhiên
- Điều chỉnh EAR Threshold: Tăng cửa sổ thời gian yêu cầu mắt nhắm liên tục (từ 2 giây lên 3 giây) để bỏ qua nhắm mắt bình thường, giảm báo động giả từ Eye State

- Fine-tune YOLOv8: Huấn luyện lại YOLOv8 trên dataset điện thoại thực tế giảm nhầm lẫn các vật thể khác, giảm báo động giả từ Object Detection từ 5,800 xuống 2,800

Kết quả: Sau các cải tiến, hệ thống đạt Precision 0.7083 (≥ 0.70), Recall 0.8500 (≥ 0.80), F1-score 0.7727 (≥ 0.75), đáp ứng tiêu chí "vừa đủ tốt" cho sử dụng thực tế

6.3 Kiểm thử phần mềm

6.3.1 Phương pháp

Bộ kiểm thử Python được chạy bằng:

```
.venv\Scripts\python.exe -m pytest -q backend\tests tests
```

Bộ kiểm thử tiện ích mở rộng được chạy bằng:

```
cd extension
```

```
npm test
```

Kết quả kiểm thử trên môi trường phát triển Windows:

Bảng 8 Kết quả kiểm thử

Lần chạy	Kết quả	Thời gian
Toàn bộ Python, lần 1	308 đạt, 1 lỗi	140,16 giây
Chạy riêng ca lỗi của lần 1	1 đạt	1,50 giây
Toàn bộ Python, lần 2	309 đạt, 0 lỗi	127,14 giây
Tiện ích mở rộng trên Node	8 đạt, 0 lỗi	138,00 ms theo Node test runner

Mỗi lần chạy toàn bộ bằng Pytest ghi nhận 344 cảnh báo, chủ yếu liên quan đến thành phần sắp ngừng hỗ trợ trong Starlette/httpx, bộ chuyển đổi ngày giờ của SQLite và protobuf. Các cảnh báo không làm kiểm thử thất bại nhưng cần được xử lý hoặc theo dõi khi nâng phiên bản thư viện phụ thuộc.

6.3.2 Kiểm thử chuỗi xử lý thị giác máy tính và thuật toán

Các nhóm kiểm thử chính gồm:

- Công thức EAR, tỉ lệ miệng và chuyển đổi tọa độ.
- Cơ chế chống dao động theo thời lượng cho vắng mặt, mất, vật thể và góc quay đầu.
- Lọc độ tin cậy của tín hiệu nhiều khuôn mặt.
- PnP trên góc quay tổng hợp biết trước.

- Độ tương đồng cosin, đăng ký khuôn mặt và chu kỳ xác minh danh tính.
- Kiểm tra sự hiện diện sống theo chuỗi mở–nhắm–mở.
- Tính chịu lỗi khi bộ phát hiện phát sinh ngoại lệ.
- Hợp đồng đủ bảy tín hiệu và bộ khởi tạo đọc đúng YAML.
- Máy trạng thái, cửa sổ trượt, vùng trễ, cảnh chuyển vào cảnh báo và nhiều tín hiệu đóng góp.
- Kiểm thử khởi tạo mô hình thật và kiểm thử tích hợp trên video mẫu.

Một số ca kiểm thử hồi quy xuất phát từ lỗi quan sát được trên webcam. Ví dụ, tín hiệu trạng thái mắt từng báo nhầm khi một mắt bị nén phối cảnh; kiểm thử hiện yêu cầu biến dạng của một mắt không được coi là nhầm, trong khi hai mắt cùng có EAR thấp vẫn phải kích hoạt điều kiện.

6.3.3 Kiểm thử dịch vụ phía máy chủ và bảo mật

Bộ kiểm thử phía máy chủ sử dụng FastAPI TestClient và cơ sở dữ liệu SQLite tạm thời cho từng ca. Các nhóm được kiểm tra gồm:

- Đăng ký, đăng nhập, cookie, đăng xuất và giới hạn tần suất.
- MFA của quản trị viên hệ thống, giới hạn số lần thử và khôi phục/đăng ký lại.
- Phân biệt mã xác thực người dùng, mã xác thực phiên, vé WebSocket và mã xác thực không hợp lệ.
- Tư cách thành viên ở nhiều tổ chức và chuyển tổ chức đang hoạt động.
- Quyền chức năng theo từng phân công kỳ thi; người quản lý ở kỳ thi A không có quyền quản lý ở kỳ thi B.
- Cô lập tổ chức trên danh sách phiên, kết thúc phiên, báo cáo, ảnh chụp và WebSocket của bảng điều khiển.
- Kế thừa chính sách và từ chối cấu hình làm yếu mức sàn hệ thống/tổ chức.
- Vòng đời, kiểm soát đồng thời lạc quan, mã tham gia và hạn mức phiên đồng thời.
- Google OIDC giả lập, state/PKCE/nonce/mã cấp quyền và mã xác thực thiết bị.
- Kiểm tra đủ bảy tín hiệu, tính lại mức rủi ro/toàn vẹn phía máy chủ và sự kiện trình duyệt.
- Tải ảnh chụp lên, chống tấn công duyệt đường dẫn, hậu kiểm sự cố và tác vụ báo cáo nền.
- Tiêu đề bảo mật, DOM an toàn và các đường dẫn giao diện.

6.3.4 Kiểm thử tiện ích mở rộng

Tám trường hợp kiểm thử Node kiểm tra giao diện thiết lập, luồng nhập mã, chuẩn hóa địa chỉ máy chủ, yêu cầu HTTPS ngoài localhost, so sánh phiên bản ngữ nghĩa, phân tích định danh trình duyệt, lọc dữ liệu chuyển hướng OAuth và cấu trúc gói tiện ích. Ngoài kiểm thử

tự động, các API camera, chia sẻ màn hình, toàn màn hình và hành vi trình duyệt vẫn cần được kiểm tra thủ công trên Chrome và Firefox thực tế vì Node không mô phỏng đầy đủ hộp thoại cấp quyền và vòng đời tiến trình dịch vụ.

6.4 Đánh giá hoạt động của hệ thống

6.4.1 Phương pháp và tiêu chí đánh giá

Đánh giá hoạt động xem xét khả năng duy trì chuỗi nghiệp vụ từ cấu hình kỳ thi đến hậu kiểm. Bằng chứng sử dụng gồm mã nguồn, kiểm thử tự động, cấu trúc giao diện đã cài đặt, kết hợp dữ liệu và trạng thái bền vững trong cơ sở dữ liệu hoặc kho bằng chứng. Việc một tuyến xử lý tồn tại không đồng nghĩa hệ thống đã sẵn sàng triển khai thực tế. Một luồng chỉ được kết luận đạt trong phạm vi kiểm thử khi đã có cả kiểm tra hành vi và điều kiện lỗi tương ứng; các tương tác phụ thuộc trình duyệt, webcam và hộp thoại cấp quyền được ghi rõ là cần xác nhận bổ sung.

Các tiêu chí gồm:

- Tính đầy đủ nghiệp vụ: có đủ bước chuẩn bị, tham gia, giám sát, kết thúc, hậu kiểm và tạo báo cáo.
- Tính nhất quán dữ liệu: trạng thái trên giao diện có nguồn từ dữ liệu máy chủ đã kiểm chứng; bằng chứng gắn đúng tổ chức, kỳ thi và phiên.
- Khả năng quan sát: người dùng nhận biết được trạng thái tải, lỗi, mất kết nối, kiểm tra trước phiên chưa đạt và tác vụ báo cáo thất bại.
- An toàn phạm vi: chức năng chỉ xuất hiện và chỉ được thực thi với vai trò hoặc năng lực hợp lệ.
- Khả năng suy giảm có kiểm soát: lỗi của một kênh không làm dữ liệu cũ bị trình bày như dữ liệu thời gian thực hoặc tạo trạng thái báo cáo thành công giả.

6.4.2 Đánh giá các kịch bản đầu-cuối

Bảng 9 Đánh giá các kịch bản đầu-cuối

Kịch bản	Luồng được kiểm chứng	Bằng chứng hiện có	Đánh giá
Chuẩn bị tổ chức	Tạo tổ chức, membership, role, policy và quota	API/UI route, migration và test auth/RBAC/policy	Đạt trong phạm vi kiểm thử tự động
Chuẩn bị kỳ thi	Tạo kỳ thi, cấu hình, assignment, readiness và join code	Lifecycle, optimistic locking, assignment, join-code test	Đạt trong phạm vi kiểm thử tự động

Thí sinh tham gia	Kiểm tra mã, xác thực thủ công/Google, consent, cấp session token	Candidate auth, Google OIDC giả lập, token-type test và extension setup test	Đạt về logic; cần kiểm tra OAuth/permission thật
Giám sát trình duyệt	Preflight, tab/focus/fullscreen/clipboard, heartbeat	Extension source, Node test, browser-event validation test	Đạt phần logic; cần chạy Chrome/Firefox thật
Truyền dữ liệu realtime	Client hello, telemetry, violation, browser event, heartbeat, dashboard fan-out	WebSocket schema, ticket, sequence/dedup và authorization test	Đạt trong phạm vi backend test
Giám sát và mở chi tiết phiên	REST initial state, WebSocket update, filter, risk/evidence timeline	Dashboard route/static source, tenant-isolation và WebSocket test	Đạt về tích hợp phía server; cần kiểm tra UI đa phiên thật
Kết thúc và hậu kiểm	End session, incident review, snapshot access và audit	Session-end, review, snapshot/path traversal và audit test	Đạt trong phạm vi kiểm thử tự động
Sinh báo cáo	Report job → worker → HTML/PDF → trạng thái completed/failed	Reporting consistency, report-job và tolerant-loader test	Đạt về chức năng
Mất kết nối/lỗi dữ liệu	Socket gián đoạn, message sai schema, dữ liệu trùng, backend unavailable	Validation, dedup, connection manager và local best-effort behavior	Đạt một phần; cần thử nghiệm mạng gián đoạn dài

Kết quả cho thấy các phân hệ được nối với nhau bằng hợp đồng dữ liệu: chính sách từ máy chủ điều khiển bước kiểm tra trước phiên; mã xác thực phiên xác định đúng phiên thi; dữ liệu giám sát đã kiểm chứng cập nhật trạng thái hiện tại và kho bằng chứng; bảng điều khiển, hậu kiểm và báo cáo cùng sử dụng mã phiên làm khóa truy vết. Những nội dung còn thiếu bằng chứng chủ yếu thuộc môi trường thực tế: vòng đời tiến trình dịch vụ, hợp thoại cấp quyền, tải đồng thời nhiều phiên và các dạng mất mạng kéo dài.

6.4.3 Đánh giá luồng dữ liệu và tính nhất quán

Luồng dữ liệu tại Hình 4.9 được đánh giá theo ba chặng. Ở chặng thu thập, ứng dụng thí giác máy tính tạo đủ bảy tín hiệu và tiện ích mở rộng chỉ gửi các sự kiện trình duyệt đã định nghĩa; dữ liệu ảnh không được truyền liên tục. Ở chặng kiểm chứng, máy chủ từ chối trường thừa, miền giá trị sai, dấu thời gian thiếu múi giờ, số thứ tự trùng và đường dẫn ảnh chụp do

máy khách tự quyết định. Ở chặng khai thác, dữ liệu thời gian thực chỉ được phát đến bảng điều khiển sau khi xác thực quyền và kiểm tra hợp lệ; SQL và kho bằng chứng lưu hai biểu diễn phục vụ hai kiểu truy cập khác nhau.

Các bất biến quan trọng đã được kiểm thử gồm:

- Một bản cập nhật giám sát phải chứa đúng bảy tín hiệu với bảy trạng thái không trùng.
- Điểm rủi ro, mức nghiêm trọng, tín hiệu chính/đóng góp và mức toàn vẹn trình duyệt không được tin trực tiếp nếu có thể tính lại phía máy chủ.
- ExamSession giữ bản sao trạng thái mới nhất để bảng điều khiển nạp nhanh; JSONL giữ chuỗi sự kiện để dựng dòng thời gian và báo cáo.
- Ảnh chụp phải đúng định dạng, kích thước và hàm băm, đồng thời nằm trong thư mục phiên do máy chủ quản lý.
- Bản hậu kiểm sự cố bổ sung kết luận của con người mà không sửa sự kiện vi phạm nguyên bản.
- WebSocket của bảng điều khiển dùng vé kết nối ngắn hạn, một lần; quyền vẫn được kiểm tra theo phân công kỳ thi.

Nhờ các bất biến này, cùng một sự kiện có thể được truy từ thẻ trên bảng điều khiển đến trang chi tiết phiên, dòng JSONL, ảnh chụp và báo cáo. Giới hạn hiện tại là việc đồng bộ ứng dụng thị giác máy tính và tiện ích mở rộng vào cùng một phiên vẫn phụ thuộc cách triển khai phía máy khách; hệ thống chưa có cầu nối nhắn tin với ứng dụng cục bộ (native messaging) hoàn chỉnh để ràng buộc hai tiến trình trên máy thí sinh ở mức chống giả mạo.

6.4.4 Đánh giá giao diện và khả năng vận hành

Giao diện đã bao phủ các nhiệm vụ chính được mô tả tại mục 5.8. Khung web tách không gian làm việc của hệ thống, tổ chức và kỳ thi; tiện ích trình bày luồng tham gia theo từng bước; ứng dụng máy tính để bàn phản ánh máy trạng thái; trang chi tiết phiên và báo cáo phục vụ hậu kiểm. Cách chia này phù hợp với tần suất sử dụng: quản trị viên không cần thấy dữ liệu ở cấp khung hình, thí sinh không cần thấy bảng điều khiển nhiều phiên, còn giám thị cần ưu tiên rủi ro, mức toàn vẹn, cảnh báo và bằng chứng.

Về phản hồi trạng thái, bảng điều khiển có chỉ báo WebSocket, tiện ích có vùng kiểm tra trước phiên/trạng thái, tác vụ báo cáo có các trạng thái pending-processing/failed/completed, và ứng dụng máy tính để bàn có các trạng thái IDLE-ENROLLMENT-MONITORING-GENERATING_REPORT-ENDED.

6.4.5 Tổng hợp mức độ hoạt động

Bảng 10 Tổng hợp mức độ hoạt động

Phân hệ	Kết quả chính	Hạn chế cần công bố
Desktop CV	Đủ vòng đời, bày signal, fusion, local log và report	FPS và độ ổn định phụ thuộc thiết bị/camera
Browser Extension	Có setup, policy, consent, preflight và browser event	Node không mô phỏng đầy đủ browser permission/service worker
Backend nền tảng	Đủ auth, tenant/RBAC, exam/session, evidence, review, report	Chưa thay thế pentest và kiểm thử tải production
Dashboard realtime	Có initial state, ticket WebSocket và fan-out cập nhật	Cần thử nghiệm nhiều proctor/nhiều process trên hạ tầng thật
Lưu trữ và báo cáo	SQL current state kết hợp JSONL/snapshot, report chạy nền	Cần đo dung lượng, retention và khôi phục dữ liệu dài hạn
Khả năng truy vết	Có request/audit, session ID, event ID, review và report job	Cần chuẩn hóa observability tập trung khi triển khai phân tán

Tổng thể, hệ thống đạt mức nguyên mẫu tích hợp có thể trình diễn và kiểm thử đầu-cuối. Hệ thống đã có quản trị, phân quyền, cập nhật thời gian thực, quản lý bằng chứng và báo cáo, nhưng chưa đạt mức sẵn sàng triển khai diện rộng. Để nâng mức trưởng thành, cần bổ sung kiểm thử trên trình duyệt và thiết bị thật, kiểm thử tải nhiều phiên, kiểm thử bảo mật độc lập, giám sát vận hành và quy trình sao lưu/khôi phục.

6.4.6 Kịch bản nghiệm thu thủ công khi chạy hệ thống

Bảng sau là mẫu biên bản cần hoàn thiện trong lần chạy nghiệm thu. Cột “Kết quả thực tế” được chủ động để ở trạng thái chưa ghi nhận; chỉ chuyển sang “Đạt/Không đạt” sau khi thực hiện đúng bước, lưu ảnh chụp hoặc nhật ký và ghi rõ môi trường. Cách trình bày này tránh biến sự tồn tại của tuyên xử lý hoặc kết quả kiểm thử tự động thành bằng chứng giả định về trải nghiệm trên thiết bị thật.

Bảng 11 Kịch bản nghiệm thu thủ công khi chạy hệ thống

STT	Chuẩn bị và thao tác	Kết quả mong đợi	Bằng chứng cần lưu
-----	----------------------	------------------	--------------------

1	Đăng ký tài khoản/tổ chức; đăng nhập và hoàn tất MFA	Tạo đúng người dùng/tur cách thành viên; cookie phiên hợp lệ; MFA sai bị từ chối	Ảnh màn hình đăng ký, đăng nhập, tài khoản và nhật ký xác thực/kiểm toán
2	System/Organization Admin mở chính sách, bảo mật, nhật ký và thử truy cập ngoài quyền	Menu đúng vai trò; thao tác hợp lệ thành công; truy cập chéo bị 403/404	Ảnh màn hình quản trị, phản hồi HTTP và bản ghi kiểm toán
3	Exam Manager tạo kỳ thi, chính sách, nhiệm vụ, mã tham gia và chạy kiểm tra sẵn sàng	Chỉ đạt trạng thái sẵn sàng khi đủ điều kiện; phiên bản/trạng thái được cập nhật đúng	Ảnh màn hình kỳ thi và phản hồi API
4	Thí sinh nhập mã trên tiện ích, xác thực, xem chính sách, đồng ý và kiểm tra trước phiên	Mã hợp lệ trả chính sách; thiếu quyền/sự đồng ý bị chặn; phiên được tạo đúng thí sinh	Ảnh màn hình tiện ích và nhật ký mạng/tiến trình nền
5	Kích hoạt phiên tiện ích và phát sinh sự kiện chuyển thẻ, mất tập trung, rời toàn màn hình hoặc dùng bảng tạm	Trạng thái hoạt động đúng; sự kiện/nhịp kết nối đến máy chủ và xuất hiện trên chi tiết phiên	Ảnh tiện ích, bảng điều khiển, chi tiết phiên và nhật ký WebSocket
6	Chạy ứng dụng CV: IDLE → ENROLLMENT → MONITORING	Trạng thái chuyển đúng; đăng ký thất bại không chuyển sang giám sát; camera được giải phóng khi kết thúc	Ảnh các trạng thái của ứng dụng và nhật ký cục bộ
7	Tạo một hành vi đủ ngưỡng cảnh báo CV	Bảng điều khiển cập nhật điểm/trạng thái; vi phạm chỉ sinh tại cạnh chuyển vào cảnh báo; ảnh chụp gần đúng sự kiện	Ảnh bảng điều khiển/chi tiết phiên, JSONL và ảnh bằng chứng
8	Ngắt mạng/WebSocket rồi phục hồi	Giao diện báo mất realtime; client retry theo thiết kế; current state được nạp lại mà không nhân đôi event	Chụp trạng thái trước/sau, sequence và server log
9	Dùng tài khoản/tổ chức khác mở URL exam/session/evidence đã biết	Không lộ tài nguyên; backend trả 404 hoặc 403 theo quy tắc phạm vi	HTTP response và security audit
10	Giám thị mở bằng chứng, nhập kết luận/ghi chú và lưu hậu kiểm	Bản hậu kiểm có người thực hiện/dấu thời gian; sự kiện vi phạm gốc không đổi	Ảnh màn hình hậu kiểm, bản ghi cơ sở dữ liệu và JSONL trước/sau
11	Yêu cầu báo cáo, chạy tiến trình nền và tải HTML/PDF	Tác vụ đi qua các trạng thái hợp lệ; báo cáo hiển thị	Ảnh báo cáo, bản ghi ReportJob và tệp đầu ra

		đúng tiếng Việt, diễn biến, sự kiện và ảnh	
12	Khởi động lại container và chạy chính sách lưu trữ ở chế độ thử	Cơ sở dữ liệu/bảng chứng còn trên vùng dữ liệu; chế độ thử chỉ liệt kê, chưa xóa	Hình 5.1, nhật ký container/vùng dữ liệu và đầu ra chạy thử

Mỗi lần nghiệm thu cần ghi kèm: ngày giờ; bản sửa đổi/cấu hình; hệ điều hành; CPU/RAM; camera; phiên bản Chrome/Firefox và tiện ích; cách chạy máy chủ (SQLite/Uvicorn hay Docker Compose/PostgreSQL/Redis); số phiên đồng thời; cùng điều kiện mạng. Nếu một kịch bản không thực hiện được, phải ghi “Không thực hiện” và lý do thay vì đổi thành “Đạt”.

6.5 Đánh giá hiệu năng chuỗi xử lý

Phép đo hiệu năng có sẵn được chạy trên CPU, sử dụng 60 khung hình tổng hợp sau 5 khung hình khởi động. Phép đo chỉ tính chi phí suy luận và logic, không bao gồm thao tác vào/ra của webcam hoặc kết xuất giao diện. Kết quả được tổng hợp như sau:

Bảng 12 Đánh giá hiệu năng chuỗi xử lý

Thành phần	Thời gian trung bình	Tỉ lệ chi phí
Phát hiện khuôn mặt bằng MTCNN	28,95 ms/khung hình	67%
YOLOv8 đã giới hạn tần suất	10,32 ms/khung hình	24%
MediaPipe Face Landmarker	3,45 ms/frame	8%
Tiền xử lý	0,21 ms/khung hình	<1%
Logic của bảy tín hiệu	khoảng 0,04 ms/khung hình	<1%
Tổng	42,98 ms/khung hình	100%

Vòng lặp đo trực tiếp đạt khoảng 25,3 FPS. MTCNN là nút thắt chính; tầng trích xuất tín hiệu chiếm tỉ lệ nhỏ vì chỉ xử lý các đặc trưng dùng chung. Một lần gọi FaceEmbedder.extract() mất khoảng 27,6 ms nhưng mặc định chỉ chạy sau mỗi 30 giây. Vì vậy, chi phí trung bình thấp, mặc dù một khung hình có thể bị chậm tại thời điểm kiểm tra lại danh tính.

Kết quả cho thấy chuỗi xử lý đạt tốc độ gần thời gian thực trong đúng cấu hình máy và dữ liệu tổng hợp của phép đo; đây không phải cam kết FPS cho mọi thiết bị. Cần chạy lại scripts/benchmark_fps.py trên máy triển khai và với khung hình webcam đại diện trước khi đưa ra kết luận vận hành.

CHƯƠNG 7 KẾT LUẬN

7.1 Tổng kết đề tài

Đồ án đã xây dựng một hệ thống hỗ trợ giám sát thi gồm chuỗi xử lý thị giác máy tính tại máy thí sinh, tiện ích mở rộng trình duyệt và nền tảng FastAPI với bảng điều khiển. Từ mục tiêu ban đầu là một ứng dụng webcam cục bộ, phạm vi được mở rộng sang quản lý nhiều tổ chức, phân quyền theo kỳ thi, truyền dữ liệu thời gian thực, quản lý bằng chứng, hậu kiểm sự cố và tạo báo cáo.

Hệ thống không tự động kết luận thí sinh gian lận. Đầu ra của mô hình và tiện ích mở rộng được biểu diễn thành tín hiệu, trạng thái, điểm số, sự kiện và ảnh bằng chứng để giám thị xem xét theo quy chế và bối cảnh của kỳ thi.

7.2 Kết quả chính đạt được

7.2.1 Chuỗi xử lý thị giác máy tính có cấu trúc

Đồ án đã triển khai tầng nhận thức dùng chung gồm MTCNN, MediaPipe Face Landmarker và YOLOv8n. Kết quả được đóng gói vào PerceptionResult và chia sẻ cho bảy bộ trích xuất tín hiệu:

- Hiện diện khuôn mặt.
- Nhiều khuôn mặt.
- Trạng thái mắt.
- Trạng thái miệng.
- Hiện diện vật thể.
- Tư thế đầu.
- Danh tính.

Mỗi tín hiệu tuân theo một giao diện chung và chỉ chịu trách nhiệm cho hiện tượng tương ứng. Tình trạng thiếu khuôn mặt không bị diễn giải đồng thời thành mắt nhắm, góc quay đầu bất thường và sai khác danh tính. Khi một bộ phát hiện gặp lỗi tạm thời, cơ chế dự phòng giúp chuỗi xử lý không kết thúc đột ngột.

7.2.2 Hai cải tiến kỹ thuật so với phương pháp cơ sở

Ước lượng góc quay đầu sử dụng các cặp tương ứng giữa điểm mốc 2D và mô hình mặt 3D, kết hợp solvePnP để tính góc quay ngang, góc quay dọc và góc nghiêng. Cách này tạo đại lượng góc có ý nghĩa hình học thay cho độ lệch điểm ảnh phụ thuộc vào độ phân giải.

Tín hiệu danh tính sử dụng InceptionResnetV1 để tạo véc-tơ đặc trưng 512 chiều, lấy trung bình từ nhiều khung hình đăng ký và so sánh độ tương đồng cosin theo chu kỳ. Vùng cảnh báo, ngưỡng không khớp và yêu cầu nhiều kết quả liên tiếp giúp tránh đánh giá từ một quan sát đơn lẻ. Bước đăng ký còn được ràng buộc với thử thách chớp mắt cơ bản.

7.2.3 Bộ tổng hợp rủi ro có trạng thái và khả năng giải thích

Mỗi tín hiệu có máy trạng thái NORMAL $\rightarrow$ SUSPICIOUS $\rightarrow$ ALERT dựa trên cửa sổ thời gian và vùng trễ riêng. Tầng phiên kết hợp trạng thái bằng tổng có trọng số và vùng trễ hai ngưỡng. Sự kiện vi phạm chỉ được sinh tại cạnh chuyển vào cảnh báo, tránh tạo một bản ghi cho mọi khung hình.

Sự kiện lưu điểm rủi ro, mức nghiêm trọng, loại vi phạm chính và các tín hiệu đóng góp. Diễn biến chuyển trạng thái và điểm rủi ro được lưu riêng, giúp báo cáo giải thích nguyên nhân cảnh báo thay vì chỉ đưa ra nhãn cuối.

7.2.4 Giám sát trình duyệt và nền tảng thời gian thực

Tiện ích mở rộng đã triển khai luồng nhập mã, xác thực thủ công hoặc qua Google, ghi nhận sự đồng ý, kiểm tra chính sách và giám sát nội dung. Tiện ích theo dõi trạng thái tập trung/chuyển thể, toàn màn hình, bảng tạm, camera, micro, chia sẻ màn hình và nhịp kết nối. Điểm toàn vẹn trình duyệt được tính tách biệt với điểm rủi ro thị giác máy tính.

Dịch vụ FastAPI hỗ trợ:

- Quản lý tổ chức, thành viên, lời mời và chính sách.
- Quản lý vòng đời kỳ thi, mã tham gia, nhiệm vụ được giao và trạng thái sẵn sàng.
- Xác thực nhân sự/thí sinh, MFA và Google OIDC.
- WebSocket cho máy khách và bảng điều khiển với vé kết nối dùng một lần.
- Bảng điều khiển thời gian thực và trang chi tiết phiên.
- Bảng chứng, hậu kiểm sự cố, tác vụ báo cáo và chính sách lưu trữ.
- Docker Compose với PostgreSQL, Redis, dịch vụ FastAPI và tiến trình báo cáo.

7.2.5 Phân quyền và bảo vệ dữ liệu

Quyền được xác định từ vai trò hệ thống, tư cách thành viên tổ chức và nhiệm vụ trong kỳ thi. Quản trị viên hệ thống không mặc nhiên được xem dữ liệu thí sinh; quyền ngoại lệ phải

được phê duyệt, chỉ cho phép đọc và có thời hạn. Máy chủ kiểm tra phạm vi tổ chức và tài nguyên đối với REST, WebSocket và thao tác tải tệp.

Máy khách được coi là nguồn không hoàn toàn tin cậy. Máy chủ kiểm tra lược đồ, dấu thời gian, sự hiện diện của đủ bảy tín hiệu, điểm rủi ro, trạng thái, mức nghiêm trọng, mức đóng góp, sự kiện trình duyệt và ảnh tải lên. Dữ liệu giao diện được dựng bằng API DOM an toàn; tiêu đề bảo mật và nhật ký kiểm toán được áp dụng cho các luồng nhảy cảm.

7.2.6 Dữ liệu và báo cáo

Đồ án kết hợp SQL cho siêu dữ liệu và trạng thái hiện tại với JSONL cho chuỗi sự kiện. Ảnh chụp được lưu bằng tên do máy chủ sinh. Kết quả hậu kiểm được tách khỏi sự kiện vi phạm gốc để nhận định của con người không làm thay đổi dữ liệu máy. Chuỗi xử lý báo cáo có thể sinh HTML/PDF khi thiếu một phần dữ liệu; tác vụ báo cáo chạy trong tiến trình nền.

7.3 Kết quả kiểm thử và thực nghiệm

Kết quả kiểm thử phần mềm và thực nghiệm cần được diễn giải riêng. Bộ kiểm thử tiện ích mở rộng được chạy lại đạt 8/8 ca. Đối với Python, kết quả chạy toàn bộ bộ kiểm thử và kết quả chạy lại ca lỗi được trình bày chi tiết tại mục 6.2; kết quả cho thấy một ca có dấu hiệu phụ thuộc thứ tự hoặc không ổn định, vì ca đó đạt khi chạy riêng. Do đó, báo cáo không dùng số lượng ca kiểm thử như bằng chứng về độ chính xác nhận diện.

Phép đo hiệu năng tổng hợp trên CPU đạt khoảng 25,3 FPS; MTCNN là nút thắt lớn nhất.

Theo bảng tổng hợp của cấu hình thực nghiệm bên ngoài gồm 26 video và 199.470 khung hình:

Bảng 13 Kết quả kiểm thử và thực nghiệm

Chỉ số	Kết quả
Precision	0,7083
Recall	0,8500
F1-score	0,7727
Accuracy	0,9248
Specificity	0,9380
FPR	0,0620

So với phương pháp cơ sở được báo cáo, độ chính xác tăng 0,0562, độ bao phủ tăng 0,1270 và điểm F1 tăng 0,0876 theo giá trị tuyệt đối. Các giá trị vượt những ngưỡng mục tiêu nêu trong tài liệu thực nghiệm; tuy nhiên, thời điểm xác lập mục tiêu và tệp dữ liệu gốc chưa có trong bản bàn giao để kiểm tra độc lập.

Kết quả không được diễn giải thành độ chính xác phổ quát. Bộ dữ liệu và tệp thực nghiệm nằm ngoài kho mã nguồn; cấu hình tạo số liệu khác một phần so với cấu hình hiện tại; số người, thiết bị và điều kiện quay còn hạn chế.

7.4 Những hạn chế còn tồn tại

7.4.1 Ứng dụng thị giác máy tính và tiện ích mở rộng chưa hợp nhất hoàn toàn

Ứng dụng thị giác máy tính và tiện ích mở rộng đều có thể tạo hoặc gửi dữ liệu, nhưng chưa có cầu nối nhắn tin với ứng dụng cục bộ (native messaging) để phối hợp như một máy khách thống nhất trong cùng phiên trình duyệt. Việc hợp nhất cần giải quyết quyền quản lý mã xác thực phiên, đồng bộ vòng đời, thử lại khi lỗi và chống sự kiện trùng.

7.4.2 Đồng bộ khi mất mạng chưa đầy đủ

Ứng dụng thị giác máy tính vẫn ghi nhật ký và sinh báo cáo cục bộ khi mất kết nối với máy chủ. Tuy nhiên, dữ liệu phát sinh trong thời gian ngoại tuyến chưa được đồng bộ bù đầy đủ. Tiện ích mở rộng có hàng đợi kết nối lại nhưng dung lượng và vòng đời lưu trữ vẫn bị giới hạn bởi trình duyệt.

7.4.3 Giới hạn của mô hình thị giác máy tính

- MTCNN và các điểm mốc chịu ảnh hưởng của ánh sáng, góc lớn và che khuất.
- EAR có thể sai khi cả hai mắt cùng bị biến dạng phối cảnh.
- Tín hiệu tư thế đầu không phân biệt chắc chắn quay đầu tự nhiên với nhìn tài liệu.
- YOLOv8n có thể bỏ sót điện thoại nhỏ/mặt lưng hoặc nhận nhầm vật thể.
- Véc-tơ đặc trưng khuôn mặt nhạy với góc, ánh sáng và chất lượng đăng ký.
- Thử thách chớp mắt không chống được kỹ thuật phát lại video hoặc giả mạo chuyên dụng.

7.4.4 Giới hạn của tiện ích mở rộng trình duyệt

Tiện ích chỉ quan sát dữ liệu mà WebExtensions API cho phép. Thành phần này không quan sát được ứng dụng cục bộ khác, máy ảo, điều khiển máy tính từ xa hoặc điện thoại thứ hai. Nếu tiện ích không được ký và cài đặt bắt buộc trong trình duyệt được quản lý, người có toàn quyền trên thiết bị vẫn có thể vô hiệu hóa hoặc sửa đổi máy khách.

7.4.5 Giới hạn về quy mô và vận hành

Redis đã hỗ trợ quyền giữ kết nối của máy khách và phát-nhận cập nhật bảng điều khiển khi được cấu hình, nhưng triển khai quy mô lớn vẫn cần:

- Kho lưu trữ đối tượng thay cho hệ thống tệp cục bộ dùng chung.
- Hàng đợi công việc và cơ chế thử lại có khả năng quan sát tốt hơn.
- Sao lưu/khôi phục và phục hồi sau thảm họa.
- Chỉ số, theo vết, nhật ký tập trung và cảnh báo vận hành.
- Kiểm thử tải với nhiều tiến trình nền/nhiều kỳ thi đồng thời.
- Quy trình chuyển đổi lược đồ/khôi phục được chuẩn hóa.

7.4.6 Khả năng tái lập thực nghiệm

Kho mã nguồn chưa chứa danh mục dữ liệu, nhãn, kết quả dự đoán, tệp mô hình và thông tin môi trường của bộ 26 video. Vì vậy, ma trận nhầm lẫn và các chỉ số được giữ như kết quả từ môi trường ngoài nhưng chưa thể được tái lập độc lập từ bản bản giao hiện tại. Đây là giới hạn học thuật cần được ưu tiên khắc phục.

7.4.7 Quyền riêng tư và công bằng

Hình ảnh khuôn mặt và dữ liệu hành vi là dữ liệu nhạy cảm. Một triển khai thực tế cần thông báo minh bạch, ghi nhận sự đồng ý phù hợp, xác định thời hạn lưu trữ, quy trình khiếu nại, quyền truy cập bằng chứng và đánh giá sai lệch giữa các nhóm người dùng. Bộ dữ liệu nhỏ hiện tại chưa đủ để kết luận về tính công bằng giữa màu da, người đeo kính, đặc điểm khuôn mặt, camera và điều kiện ánh sáng.

7.5 Hướng phát triển

Phát triển ứng dụng giao tiếp với tiện ích qua cơ chế nhắn tin với ứng dụng cục bộ. Tiện ích quản lý phiên trình duyệt và chính sách; ứng dụng đồng hành quản lý webcam và mô hình; cả hai dùng cùng mã phiên và số thứ tự thông điệp. Thiết kế cần có xác thực hai chiều, chính sách phiên bản và cơ chế chống phát lại.

Cải thiện chất lượng nhận diện:

- Thu thập dữ liệu đa dạng hơn theo người, camera và ánh sáng.
- Hiệu chuẩn EAR và ngưỡng danh tính trong bước đăng ký theo từng người.
- Kết hợp độ tin cậy của góc quay đầu với trạng thái mắt.
- Đánh giá YOLOv8s hoặc tinh chỉnh mô hình với cách chia tập huấn luyện/kiểm thử rõ ràng.

- Xử lý tín hiệu danh tính bất đồng bộ để tránh làm chậm khung hình.
- Thử nghiệm RetinaFace, ArcFace hoặc mô hình ổn định hơn khi có đủ tài nguyên.
- Tích hợp cơ chế chống giả mạo đã được đánh giá trên video phát lại và nội dung giả mạo.

Hoàn thiện sản phẩm và hạ tầng:

- Kiểm tra phiên bản và cập nhật an toàn.
- Dùng HTTPS bắt buộc, trình quản lý thông tin bí mật và cơ chế xoay vòng khóa.
- Chuyển ảnh chụp/báo cáo sang kho lưu trữ đối tượng có URL ký số ngắn hạn.
- Bổ sung trung tâm vận hành, giám sát hàng đợi và cảnh báo lưu trữ/hạn mức.
- Kiểm thử tải với nhiều tổ chức/kỳ thi/tiến trình nền và xây dựng kế hoạch năng lực.
- Chuẩn hóa chuyển đổi lược đồ bằng công cụ hỗ trợ phiên bản/khôi phục.

Quản trị dữ liệu và trải nghiệm người dùng:

- Xây dựng quy trình yêu cầu của chủ thể dữ liệu, xuất/xóa dữ liệu và lưu giữ theo yêu cầu pháp lý.
- Đóng dấu nhận diện trên báo cáo/bằng chứng và thông báo khi có truy cập nhạy cảm.
- Cải thiện bước kiểm tra trước phiên và hướng dẫn khắc phục lỗi.
- Thiết kế cơ chế tiếp tục/tham gia lại an toàn sau khi ứng dụng dừng đột ngột hoặc đổi thiết bị.
- Đánh giá khả năng tiếp cận và hỗ trợ đa ngôn ngữ/múi giờ.
- Xây dựng quy trình khiếu nại để thí sinh có thể phản hồi cảnh báo sai.

7.6 Kết luận

Đề án đã hiện thực một nền tảng hỗ trợ giám sát thi có cấu trúc, kết hợp bảy tín hiệu thị giác máy tính, máy trạng thái, bộ tổng hợp rủi ro, tiện ích theo dõi trình duyệt, bảng điều khiển thời gian thực, phân quyền nhiều tổ chức và quy trình quản lý bằng chứng, báo cáo. Bộ kiểm thử cung cấp bằng chứng về tính nhất quán của phần cài đặt trong các tình huống đã mô tả, đồng thời lần chạy lại cho thấy cần xử lý một ca kiểm thử Python có dấu hiệu không ổn định. Số liệu từ cấu hình thực nghiệm 26 video ghi nhận độ chính xác 0,7083, độ bao phủ 0,8500 và điểm F1 bằng 0,7727; hướng kết hợp đa tín hiệu có kết quả cao hơn phương pháp cơ sở trong bảng tổng hợp.

Đóng góp chính không phải là một mô hình mới, mà là cách tổ chức nhiều mô hình và thành phần phần mềm thành chuỗi xử lý có trạng thái, có khả năng giải thích và kiểm soát truy cập. Hệ thống hiện phù hợp với mục đích nghiên cứu và thử nghiệm có kiểm soát. Trước khi sử dụng cho kỳ thi rủi ro cao, cần hoàn thiện khả năng tái lập, chống giả mạo, tích hợp các thành phần phía máy khách, hạ tầng triển khai và quản trị dữ liệu.

TÀI LIỆU THAM KHẢO

- [1] Viola, P. & Jones, M., 2001. Rapid object detection using a boosted cascade of simple features. In Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), pp. 511-518.
- [2] Zhang, K., Zhang, Z., Li, Z. & Qiao, Y., 2016. Joint face detection and alignment using multitask cascaded convolutional networks. IEEE Signal Processing Letters, 23(10), pp. 1499-1503.
- [3] Redmon, J., Divvala, S., Girshick, R. & Farhadi, A., 2016. You only look once: Unified, real-time object detection. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779-788.
- [4] Deng, J., Guo, J., Ververas, E., Kotsia, I. & Zafeiriou, S., 2020. RetinaFace: Single-stage Dense Face Localisation in the Wild. In arXiv preprint arXiv:1905.00976.
- [5] LeCun, Y., Bengio, Y. & Hinton, G., 2015. Deep learning. Nature, 521(7553), pp. 436-444.
- [6] Lowe, D.G., 2004. Distinctive image features from scale-invariant keypoints. International journal of computer vision, 60(2), pp. 91-110.
- [7] Yang, M.H., Kriegman, D.J. & Ahuja, N., 2002. Detecting faces in images: a survey. IEEE transactions on pattern analysis and machine intelligence, 24(1), pp. 34-58.
- [8] Simonyan, K. & Zisserman, A., 2014. Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.
- [9] King, D.E., 2009. Dlib-ml: A machine learning toolkit. Journal of Machine Learning Research, 10, pp. 1755-1758.
- [10] Girshick, R., Donahue, J., Darrell, T. & Malik, J., 2014. Rich feature hierarchies for accurate object detection and semantic segmentation. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 580-587.
- [11] Ren, S., He, K., Zhang, X. & Sun, J., 2015. Faster R-CNN: Towards real-time object detection with region proposal networks. In Advances in Neural Information Processing Systems, pp. 91-99.

- [12] Schroff, F., Kalenichenko, D. & Philbin, J., 2015. FaceNet: A unified embedding for face recognition and clustering. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 815-823.
- [13] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.L., Yong, M.G., Lee, J. & Chang, W., 2019. MediaPipe: A Framework for Building Multimodal Machine Learning Pipelines. arXiv preprint arXiv:1906.08172.
- [14] Tereza Soukupová & Jan Kučera, 2016. Real-Time Eye Blink Detection using Facial Landmarks. In 21st computer vision winter workshop, 2016.
- [15] Lepetit, V., Moreno-Noguer, F. & Fua, P., 2006. EPnP: An accurate $O(n)$ solution to the perspective-n-point problem. International journal of computer vision, 81(2), p.155.
- [16] Phillips, P.J., Wechsler, H., Huang, J. & Rauss, P.J., 2002. The FERET database of facial images on fronts of persons and background images. IEEE transactions on pattern analysis and machine intelligence, 24(5), pp. 683-688.
- [17] Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P. & Zitnick, C.L., 2014. Microsoft COCO: Common Objects in Context. In European Conference on Computer Vision (ECCV), pp. 740-755. Springer.
- [18] Yang, S., Luo, P., Loy, C.C. & Tang, X., 2016. WIDER face: A face detection benchmark. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 5525-5533.
- [19] Jiao, L., Zhang, F., Liu, F., Yang, S., Li, L., Feng, Z. & Qu, R., 2019. A survey of deep learning-based object detection. IEEE access, 7, pp. 128837-128868.
- [20] Castanedo, F., 2013. A review of data fusion techniques. The scientific world journal, 2013, p.704674.
- [21] Ultralytics, 2023. YOLOv8: A state-of-the-art real-time object detection model. Available at: <https://github.com/ultralytics/ultralytics>